

"Data you can trust. Applications you can build."

Atomicity & Transactions & Replication

Reliable data. Always available. Built for real-world scale.



MongoDB.
Build what's next.

"Reliable data today. Greater possibilities tomorrow."

1 Atomicity & Transactions

Use MongoDB's atomic operations and transactions to keep your data consistent.

Single-Document Atomicity

- All write operations on a single document are atomic (all-or-nothing).
- No partial updates — either the entire operation succeeds or fails.
- Use atomic operators (e.g., \$set, \$inc) to modify fields safely.

```
db.accounts.updateOne(
  { _id: 1 },
  { $inc: { balance: 100 } }
)
```

The entire update is atomic at the document level.

ACID Transactions

MongoDB supports multi-document transactions with ACID guarantees:

- A** Atomicity — all operations succeed or none.
- C** Consistency — data remains in a valid state.
- I** Isolation — transactions are isolated from each other.
- D** Durability — committed data is permanently stored.

Transactions with Sessions

Use sessions to start and manage transactions.

```
const session = client.startSession();
session.startTransaction();
try {
  await db.collection('orders')
    .insertOne({ _id: 1, item: 'Book' }, { session });
  await db.collection('inventory')
    .updateOne({ _id: 1 }, { $inc: { stock: -1 } }, { session });
  await session.commitTransaction();
} catch (error) {
  await session.abortTransaction();
  throw error;
} finally {
  await session.endSession();
}
```

Use sessions to run multi-document transactions across collections.

Retries & Error Handling

Transactions can fail due to transient errors (e.g., network issues, elections). Use retries for reliability.

```
async function runTransactionWithRetry() {
  while (true) {
    try {
      // ... transaction logic
      await session.commitTransaction();
      break;
    } catch (error) {
      if (error.hasErrorLabel('TransientTransactionError')) {
        continue; // retry
      }
      throw error;
    }
  }
}
```

Retry transactions on transient errors to improve resilience.

When to Use Transactions?

- ✓ When you need to update multiple documents across one or more collections.
- ✓ When all operations must succeed or fail together.
- ✓ For complex business logic that requires consistency.
- ✓ Not needed for single-document operations (since they are already atomic).



Tips

- ✓ Keep transactions short and simple.
- ✓ Use appropriate write concern (e.g., "majority").
- ✓ Handle transient errors with retries.
- ✓ Avoid long-running transactions.
- ✓ Monitor transaction performance.

"Consistency builds trust. Transactions make it real."

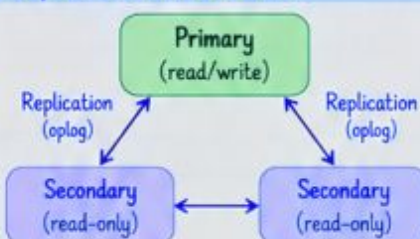
2 Replication

Use replica sets to ensure high availability, data redundancy, and automatic failover.

What is Replication?

- Replication keeps multiple copies of your data across different servers.
- MongoDB uses replica sets for replication.
- Provides high availability and fault tolerance.
- If the primary fails, a secondary is elected automatically.

Replica Set Architecture



Data is copied from the primary to secondaries asynchronously using the oplog.

Key Concepts

Replica Set	A group of MongoDB servers that maintain the same data set.
Primary	Handles all write operations.
Secondary	Replicates data from the primary (read-only by default).
Oplog	A special capped collection that records all write operations (operations log).
Elections	If the primary fails, a new primary is elected automatically.
Failover	Automatic switch to a new primary to ensure high availability.

Check Replica Set Status

Use `rs.status()` to view the replica set configuration and member states.

```
rs.status()
```

Shows the current primary, secondaries, and their status.

Example Output (Simplified)

```
{
  "set": "rs0",
  "members": [
    { "_id": 0, "name": "mongo1:27017", "stateStr": "PRIMARY" },
    { "_id": 1, "name": "mongo2:27017", "stateStr": "SECONDARY" },
    { "_id": 2, "name": "mongo3:27017", "stateStr": "SECONDARY" }
  ]
}
```

Benefits of Replication

- ✓ High availability (automatic failover).
- ✓ Data redundancy.
- ✓ Read scaling (read from secondaries).
- ✓ Disaster recovery.
- ✓ Minimal downtime.



"Resilient systems power real progress."

"More than a database. A more reliable tomorrow."



Replicate. Recover. Keep building.

"Find more than matches—find meaning."

Text Search & Vector Search

Search smarter. Find deeper. Build intelligent applications.

"Search isn't just about data. It's about opportunities."

1 Text Search

Use text search to find relevant documents based on natural language queries.

Text Indexes

Create a text index on one or more fields to enable full-text search.

```
db.articles.createIndex({
  title: "text",
  content: "text",
  tags: "text"
})
```

Text indexes tokenize words, remove stop words, and support stemming.

\$text Query

Use the \$text operator to search for a term or phrase.

```
db.articles.find({
  $text: { $search: "mongodb search tutorial" }
})
```

Returns documents that match the search terms.

Relevance and Scoring

Use \$meta to see and sort by relevance score.

```
db.articles.find(
  { $text: { $search: "database" } },
  { score: { $meta: "textScore" },
    title: 1, score: 1 }
).sort({ score: { $meta: "textScore" } })
```

Higher textScore means a more relevant match.

Autocomplete Concepts

Use prefix matching or MongoDB Search for autocomplete.

```
db.products.find({
  name: { $regex: "^lap",
    $options: "i" }
})
```

For advanced autocomplete (fuzzy matching, suggestions), use MongoDB Search (Atlas Search).

MongoDB Search (Atlas Search)

MongoDB Search provides advanced text search features like synonyms, fuzzy matching, and autocomplete.

```
db.articles.aggregate([
  {
    $search: {
      text: { query: "mongodb", path: "content" }
    }
  }
])
```

MongoDB Search is powered by Apache Lucene and available in MongoDB Atlas.

Key Features

- ✓ Full-text search across multiple fields
- ✓ Relevance scoring with textScore
- ✓ Language support and stemming
- ✓ Phrase search and negation
- ✓ Autocomplete and type-ahead (Atlas Search)
- ✓ Synonyms and custom analyzers (Atlas Search)



"Better search. Happier users."

Tips

- ✓ Choose the right fields to index.
- ✓ Use relevance scoring to improve results.
- ✓ For advanced search (autocomplete, fuzzy, synonyms), use MongoDB Search.
- ✓ Test with real user queries.
- ✓ Keep indexes up to date.
- ✓ Consider search analytics to understand user behavior.

2 Vector Search & AI Applications

Store and search vector embeddings to build AI-powered applications.

Embeddings

Convert text, images, or other data into vector embeddings (numeric arrays) using a model (e.g., OpenAI, Sentence Transformers).

```
// Example embedding (768 dims)
[ 0.023, -0.014, 0.567, ... ]
```

Embeddings capture the meaning of data in a high-dimensional vector space.

Vector Indexes

Create a vector index to enable fast similarity search.

```
db.documents.createIndex({
  embedding: {
    type: "vector",
    numDimensions: 768,
    similarity: "cosine"
  }
})
```

Vector indexes use Approximate Nearest Neighbor (ANN) algorithms for high-performance search.

Similarity Search

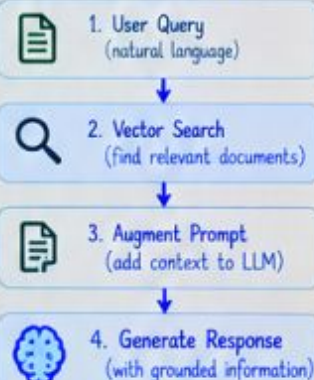
Find similar documents using a query embedding.

```
db.documents.aggregate([
  {
    $vectorSearch: {
      index: "embedding_index",
      queryVector: [ 0.023, ... ],
      path: "embedding",
      limit: 5
    }
  }
])
```

Returns the most similar documents based on vector similarity (e.g., cosine similarity).

Retrieval-Augmented Generation (RAG) Basics

Combine vector search with an LLM to ground responses in your data.



Common Use Cases

- ✓ Semantic search (find content by meaning).
- ✓ Chatbots with your data (RAG)
- ✓ Recommendation systems
- ✓ Image and multimedia search
- ✓ Knowledge bases and documentation search

"Search by meaning. Power real AI experiences."



Your data. More intelligence. Bigger possibilities."



Search deeper. Build smarter. Go further."



"Location data creates new opportunities."

Geospatial Queries & File Storage with GridFS

Store locations. Find what's near. Handle big files.

"Locations connect people, places and possibilities."

1 Geospatial Queries

Store and query location data to build location-aware applications.

GeoJSON Format

MongoDB uses GeoJSON to represent geospatial data.

```
{
  type: "Point",
  coordinates: [-73.935242, 40.730610]
}
```

Coordinates are in [longitude, latitude] order.

Geospatial Indexes

Create a 2dsphere index to support geospatial queries.

```
db.places.createIndex(
  { location: "2dsphere" }
)
```

- 2dsphere supports GeoJSON and spherical geometry (Earth).
- Required for most geospatial queries.

Nearby Searches

Find documents near a location using \$near.

```
db.places.find(
  {
    location: {
      $near: {
        $geometry: { type: "Point",
          coordinates: [-73.935242, 40.730610] },
        $maxDistance: 5000 // in meters
      }
    }
  }
)
```

Find places within 5 km of a point, sorted by distance.

Queries Within an Area

Find documents inside a specific area using \$geoWithin.

```
db.places.find(
  {
    location: {
      $geoWithin: {
        $centerSphere: [
          [-73.935242, 40.730610],
          [5 / 6378.1 // radius in radians (5 km)
        ]
      }
    }
  }
)
```

Find places within a 5 km radius (spherical area).

Common Geospatial Query Operators

Operator	Description
\$near	Finds documents near a point (returns results sorted by distance).
\$geoWithin	Finds documents within a specified area (circle, polygon, etc.).
\$geoIntersects	Finds documents that intersect with a geometry.
\$nearSphere	Like \$near but uses spherical calculation (legacy).

Supported GeoJSON Types

MongoDB supports common GeoJSON types.

Type	Description
Point	A single location (e.g., [lng, lat])
LineString	A line (e.g., a route)
Polygon	A closed area (e.g., a city boundary)
MultiPoint	Multiple points
MultiLineString	Multiple lines
MultiPolygon	Multiple polygons



Find what's near. Explore what's possible.

2 File Storage with GridFS

Store and retrieve large files using GridFS.

What is GridFS?

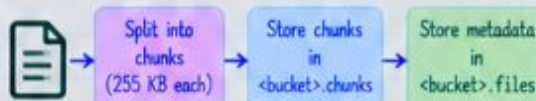
- Stores large files by splitting them into smaller chunks (default 255 KB).
- Saves file data in two collections:
 - <bucket>.files (metadata)
 - <bucket>.chunks (file chunks)
- Works with MongoDB drivers (e.g., Node.js, Python, Java, etc.).
- Ideal for large files like videos, images, backups, and documents.

Uploading a File (Node.js Example)

```
const bucket = new mongoose.mongo.GridFSBucket(
  mongoose.connection.db
);
const uploadStream = bucket.openUploadStream(
  "video.mp4",
  { metadata: { type: "video", category: "media" } }
);
fs.createReadStream("video.mp4").pipe(uploadStream);
```

Uploads a file with metadata.

How GridFS Works



GridFS automatically handles splitting, storing, and reassembling files.

Downloading a File (Node.js Example)

```
const bucket = new mongoose.mongo.GridFSBucket(
  mongoose.connection.db
);
const downloadStream = bucket.openDownloadStreamByName(
  "video.mp4"
);
downloadStream.pipe(fs.createWriteStream("video.mp4"));
```

Downloads and saves a file by filename.

When to Use GridFS?

- ✓ Files are larger than 16 MB (MongoDB's document size limit).
- ✓ You need to store files with metadata.
- ✓ You want to keep files in the same database as your application data.
- ✓ You need to stream files efficiently (e.g., for video or large downloads).

Example: Stored File Metadata

```
{
  _id: ObjectId("..."),
  filename: "video.mp4",
  length: 10485760,
  chunkSize: 255000,
  uploadDate: ISODate("2024-01-01T12:00:00Z"),
  metadata: { type: "video", category: "media" }
}
```

Metadata is stored in <bucket>.files



"Store more than just data. Store possibility."

"Maps for people. Files for progress."



Location aware. Data rich. A brighter tomorrow.

"More than queries – build powerful data solutions."

Advanced Aggregation & Views & Special Collections

Unlock advanced features. Model for real-world use cases.



MongoDB.
Build what's next.

"Same data.
More insights.
Bigger impact."

1 Advanced Aggregation

Use advanced aggregation stages to solve complex analytics and data processing problems.

\$facet – Multiple Pipelines

Run multiple aggregation pipelines in parallel on the same input data.

```
db.orders.aggregate([
  {
    $facet: {
      totalSales: [
        { $group: { _id: null, total: { $sum: "$amount" } } }
      ],
      byStatus: [
        { $group: { _id: "$status", count: { $sum: 1 } } }
      ]
    }
  }
])
```

Returns multiple result sets in a single query (e.g., total sales and count by status).

\$bucket – Group into Ranges

Group documents into specified buckets (ranges).

```
db.orders.aggregate([
  {
    $bucket: {
      groupBy: "$amount",
      boundaries: [0, 100, 500, 1000, 5000],
      default: "Other",
      output: { count: { $sum: 1 } }
    }
  }
])
```

Groups documents into custom ranges and returns a count for each bucket.

\$setWindowFields – Window Functions

Perform calculations across a range of documents (e.g., running totals, moving averages).

```
db.sales.aggregate([
  {
    $setWindowFields: {
      partitionBy: "$item",
      sortBy: { date: 1 },
      output: {
        runningTotal: {
          $sum: "$amount",
          window: { documents: [ "unbounded", "current" ] }
        }
      }
    }
  }
])
```

Adds calculated fields based on a window of documents.

\$merge – Write to a Collection

Write the aggregation results to an existing collection (merge).

```
db.orders.aggregate([
  { $group: { _id: "$item", total: { $sum: "$amount" } } },
  { $merge: { into: "item_totals", whenMatched: "merge", whenNotMatched: "insert" } }
])
```

Useful for materializing aggregation results into a collection.

\$out – Write to a New Collection

Write the aggregation results to a new collection (replaces if exists).

```
db.orders.aggregate([
  { $group: { _id: "$item", total: { $sum: "$amount" } } },
  { $out: "item_totals_backup" }
])
```

Creates a new collection with the aggregation results.



Tips for Advanced Aggregation

- ✓ Use \$facet to get multiple insights in one query.
- ✓ Use \$bucket for custom range analysis.
- ✓ Leverage \$setWindowFields for time-based calculations and trends.
- ✓ Use \$merge or \$out to persist results for reporting or downstream processing.
- ✓ Combine multiple stages to solve real-world analytics problems.

"Advanced pipelines.
Real-world results."

2 Views & Special Collections

Use views and special collections to simplify access, handle specific workloads, and model time-based data.

Views (Read-Only)

A view is a virtual collection defined by an aggregation pipeline. It is read-only and always reflects the latest data from the source collection.

```
db.createView(
  "active_orders",
  "orders",
  [ { $match: { status: "active" } },
    { $project: { _id: 1, item: 1, amount: 1 } }
  ]
)
```

Benefits:

- Simplifies complex queries.
- Provides a consistent, reusable view of data.
- Useful for access control (hide sensitive fields).
- Always up-to-date (no data duplication).

Capped Collections

Capped collections have a fixed size and maintain insertion order. When full, they automatically overwrite the oldest documents.

```
db.createCollection("logs", {
  capped: true,
  size: 10485760, // 10 MB
  max: 10000 // optional: max docs
})
```

Benefits:

- High write performance.
- Useful for logs, events, and real-time data.
- Maintains insertion order.
- Automatically removes old data.
- Cannot be resized (must recreate to change size).

Time-Series Collections

Optimized for time-series data (e.g., sensor data, metrics, IoT). Automatically manages time-based indexes and storage.

```
db.createCollection("sensor_data", {
  timeseries: {
    timeField: "timestamp",
    metaField: "deviceId",
    granularity: "hours"
  }
})
```

Benefits:

- Optimized storage and performance for time-based data.
- Automatic indexing on time field.
- Supports high-frequency insertions.
- Ideal for IoT, monitoring, and event data.



"Different data needs.
Flexible solutions."

"Model smarter.
Scale further."



Real data.
Real use cases.
Real possibilities.

"Turn data
into meaning."

Grouping & Expressions Joining & Reshaping Data

Advanced aggregations. Richer insights. More possibilities.



MongoDB.
Build what's next.

"Connect.
Transform.
Discover more
from your data."

1 Grouping & Expressions

Use \$group to aggregate data and apply powerful expressions to transform your data.

\$group - Example

Group orders by customer and calculate total amount and order count.

```
db.orders.aggregate([
  { $group: {
    _id: "$customerId",
    totalAmount: { $sum: "$amount" },
    orderCount: { $sum: 1 }
  } }
])
```

Groups documents by customerId and calculates the total amount and number of orders.

Common Accumulators (used with \$group)

Accumulator	Description	Example
\$sum	Sum of values	{ \$sum: "\$amount" }
\$avg	Average of values	{ \$avg: "\$price" }
\$min	Minimum value	{ \$min: "\$age" }
\$max	Maximum value	{ \$max: "\$age" }
\$first	First value	{ \$first: "\$name" }
\$last	Last value	{ \$last: "\$name" }
\$push	Add values to an array	{ \$push: "\$item" }
\$addToSet	Add unique values to an array	{ \$addToSet: "\$tag" }
\$count	Count documents (MongoDB 5.0+)	{ \$count: {} }



Expression Types

Use expressions to perform calculations and transformations in your pipeline.

Arithmetic

\$add, \$subtract,
\$multiply, \$divide,
\$mod

Conditional

\$cond, \$ifNull,
\$switch

String

\$concat, \$toUpper,
\$substr, \$indexOfCP

Date

\$year, \$month, \$dayOfMonth,
\$dateAdd, \$dateDiff

Array

\$arrayElemAt, \$size,
\$filter, \$map, \$in

More \$group Example (with Expressions)

```
db.orders.aggregate([
  { $group: {
    _id: { year: { $year: "$date" } },
    total: { $sum: "$amount" },
    avgPrice: { $avg: "$price" }
  } }
])
```

Groups by year using a date expression and calculates total and average price.

Tips

- ✓ Use the right accumulator for your goal.
- ✓ Combine expressions for more powerful logic.
- ✓ \$group is memory intensive – consider indexing and pipeline order.
- ✓ Explore the full list of aggregation expressions in the MongoDB docs.

2 Joining & Reshaping Data

Combine data from multiple collections and reshape documents to fit your needs.

\$lookup - Join Collections

Performs a left outer join to combine data from another collection.

```
db.orders.aggregate([
  { $lookup: {
    from: "users",
    localField: "userId",
    foreignField: "_id",
    as: "user"
  } }
])
```

Adds a user array with matching documents from the users collection.



Result: order with user details.

\$unwind - Deconstruct Arrays

Deconstructs an array field to output one document per array element.

```
db.orders.aggregate([
  { $unwind: "$items" }
])
```

If items is an array, creates a separate document for each item.

Before

```
{ _id: 1,
  items: [ "Book", "Pen" ] }
```

After

```
{ _id: 1, items: "Book" }
{ _id: 1, items: "Pen" }
```

\$unionWith - Combine Results

Combines the results of two pipelines (even from different collections).

```
db.orders.aggregate([
  { $match: { status: "open" } },
  { $unionWith: {
    coll: "returns",
    pipeline: [ { $match: {
      status: "open" } } ]
  } }
])
```

Returns open orders and open returns as a single result set.

\$replaceRoot - Reshape Documents

Replaces the current document with a specified document.

```
db.orders.aggregate([
  { $lookup: {
    from: "users",
    localField: "userId",
    foreignField: "_id",
    as: "user"
  } },
  { $unwind: "$user" },
  { $replaceRoot: { newRoot: "$user" } }
])
```

Replaces the order document with the joined user document.



"Bring data together.
See the bigger picture."

"Aggregate. Join. Reshape.
Turn complex data into
clear insights."



Same data.
New perspectives.
Greater possibilities.

Query Performance

Understand. Measure. Optimize.

"Faster queries.
Happier users."

"Good performance
turns data into
real impact."

1 Query Performance

Use `explain()`, understand how queries use indexes, and learn to optimize performance.

What is `explain()`?

The `explain()` method shows how MongoDB executes a query, including whether it uses an index or scans the whole collection.

```
db.users.find({ age: 25 }).explain("executionStats")
```

Returns details about query execution, index usage, and documents examined.

Example: `explain()` Output (Simplified)

```
{
  "queryPlanner": {
    "winningPlan": {
      "stage": "IXSCAN",
      "indexName": "age_1"
    },
    "executionStats": {
      "nReturned": 5,
      "totalDocsExamined": 5,
      "totalKeysExamined": 5
    }
  }
}
```

This query used
an index and
examined only
5 documents.

Index Scan vs Collection Scan

Index Scan (Good)

- Uses an index to find matching documents.
- Faster and more efficient.
- Look for "IXSCAN" in `explain()` output.

```
"stage": "IXSCAN",
"indexName": "age_1"
```

Reads only the relevant index entries.

Collection Scan (Bad)

- Scans every document in the collection.
- Slower and uses more resources.
- Look for "COLLSCAN" in `explain()` output.

```
"stage": "COLLSCAN",
"docsExamined": 10000
```

Reads the entire collection.

Covered Queries

A query is covered when MongoDB can answer it using only the index, without fetching documents from the collection.

```
db.users.find(
  { age: 25 },
  { _id: 0, name: 1, age: 1 }
).explain()
```

Look for "totalDocsExamined": 0 in the `explain()` output.

Index Trade-offs

- Indexes make reads faster.
- But they increase storage usage.
- They can slow down write operations (inserts, updates, deletes).
- Use indexes wisely — only on fields that are frequently queried.



Tips for Better Query Performance

- ✓ Use `explain()` to analyze slow queries.
- ✓ Create indexes on frequently queried fields.
- ✓ Avoid collection scans (COLLSCAN).
- ✓ Use covered queries when possible.
- ✓ Monitor and review indexes regularly.

"Measure.
Learn.
Optimize.
Repeat."

2 Aggregation Pipeline Basics

Transform your data with a powerful and flexible pipeline.

What is Aggregation?

The aggregation pipeline processes documents through a series of stages to transform and analyze your data.

- Each stage takes the output from the previous stage.
- You can filter, reshape, sort, group, and more.
- Very useful for analytics and reporting.



Common Pipeline Stages

- \$match** Filters documents (like `find`).
- \$project** Shapes documents (selects fields).
- \$sort** Sorts documents.
- \$limit** Limits the number of documents.

Example: Aggregation Pipeline

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $project: { _id: 0, item: 1, amount: 1 } },
  { $sort: { amount: -1 } },
  { $limit: 5 }
])
```

Find completed orders, show only item and amount, sort by amount (highest first), and return the top 5.

\$match - Filter Documents

Works like a query filter.

```
{ $match: { status: "completed" } }
```

Keeps only matching documents.

\$project - Select Fields

Include or exclude fields.

```
{ $project: { _id: 0,
  item: 1,
  amount: 1 } }
```

Reshapes each document.

\$sort - Order Documents

Sorts by one or more fields.

```
{ $sort: { amount: -1 } }
```

-1 for descending, 1 for ascending.

\$limit - Limit Results

Returns only the first N documents.

```
{ $limit: 5 }
```

Useful for top-N results.



"From raw data
to real insights."

"The right pipeline
turns data into decisions."



Same data.
Bigger possibilities.

Indexing Fundamentals & Advanced Indexes

Speed up your queries. Build for scale.

"Index today,
faster queries
tomorrow."

"Good indexes
turn big data into
fast answers."

1 Indexing Fundamentals

Indexes improve query performance by allowing MongoDB to quickly locate documents.

Why Use Indexes?

- Faster query performance
- Reduce the amount of data scanned
- Support sorting and filtering
- Improve overall application performance

Creating an Index

```
db.users.createIndex({ name: 1 })

// 1 = ascending (default)
// -1 = descending
```

Creates a single-field index on the "name" field.

List Indexes

```
db.users.getIndexes()
```

Shows all indexes for the collection.

Drop an Index

```
db.users.dropIndex("name_1")
```

Removes the specified index.

Single-Field Index

Index on one field.

```
db.users.createIndex({ email: 1 })
```

Good for queries that filter or sort by a single field.

Compound Index

Index on multiple fields.

```
db.users.createIndex({
  age: 1,
  name: -1
})
```

Improves performance for queries that use multiple fields.

Multikey Index

Index on an array field (automatically creates an index for each element).

```
db.products.createIndex({ tags: 1 })
```

Useful for querying array values.



← MongoDB automatically creates multikey indexes for array fields.

2 Advanced Indexes

Use specialized indexes to handle specific use cases and improve performance further.

Unique Index

Ensures all values in the indexed field are unique.

```
db.users.createIndex(
  { email: 1 },
  { unique: true }
)
```

Prevents duplicate values.

Partial Index

Index only documents that match a filter condition.

```
db.users.createIndex(
  { status: 1 },
  { partialFilterExpression: {
    status: "active"
  } }
)
```

Improves only documents where status is "active".

Sparse Index

Index only documents that have the indexed field.

```
db.users.createIndex(
  { phone: 1 },
  { sparse: true }
)
```

Useful when some documents don't have the field.

TTL Index (Time To Live)

Automatically removes documents after a specified time.

```
db.sessions.createIndex(
  { createdAt: 1 },
  { expireAfterSeconds: 3600 }
)
```

Deletes documents 1 hour after createdAt.

Hashed Index

Index using a hash of the field value (good for sharding).

```
db.users.createIndex(
  { userId: "hashed" }
)
```

Supports hashed sharding and even data distribution.

Wildcard Index

Index all fields in a document or a set of fields.

```
db.items.createIndex(
  { "$**": 1 }
)
```

Indexes all fields in the document. You can also specify a subpath (e.g., "metadata.\$**").



Tips & Best Practices

- ✓ Create indexes based on your most common queries.
- ✓ Avoid creating too many indexes (they use storage and slow writes).
- ✓ Use compound indexes for multi-field queries.
- ✓ Monitor index usage with explain().
- ✓ Choose the right index type for your use case.

"Right index.
Right performance.
Bigger possibilities."



Index smart.
Scale faster.

Relationships & Design Patterns & Schema Validation



MongoDB.
Build what's next.

"Design today
for a scalable
tomorrow."

Model smart. Validate data. Build robust applications.

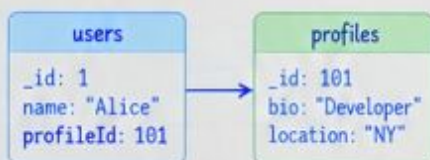
"Good design
prevents problems.
Validation keeps
your data clean."

1 Relationships & Design Patterns

Model relationships based on your application's needs. Choose the right pattern for performance and simplicity.

One-to-One

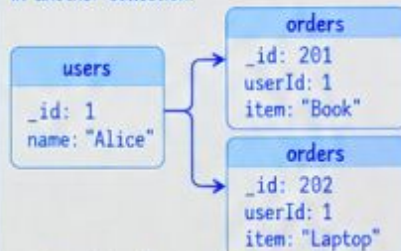
A single document is related to one document in another collection.



- Use embedding if the related data is small and always needed.
- Use referencing if the related data is large or optional.

One-to-Many

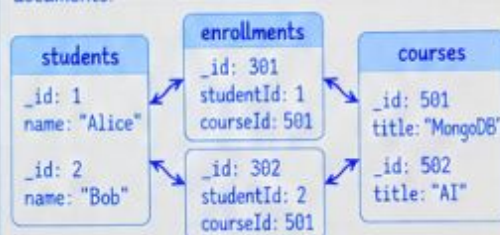
A document is related to multiple documents in another collection.



- Store the reference (e.g., `userId`) in the "many" side.
- Use embedding if the related items are small and frequently accessed together.

Many-to-Many

Multiple documents are related to multiple documents.



- Use a junction (link) collection (e.g., `enrollments`).
- Each document references the related documents.

Common Design Patterns

- Embedding related data
- Referencing with `ObjectId`
- Subset pattern (store a few important fields)
- Computed pattern (store pre-computed values)
- Bucketing pattern (group time-series data)
- Polymorphic pattern (store different types in one collection)
- Outlier pattern (separate large or infrequent data)



Common Design Mistakes

- ❌ Over-embedding (very large documents).
- ❌ Unbounded arrays (arrays that grow too large).
- ❌ Not considering query patterns.
- ❌ Duplicating data without a clear reason.
- ❌ Ignoring data growth and performance.
- ❌ Using the wrong data type (e.g., string instead of `ObjectId`).
- ❌ Not indexing fields used in relationships.

"Keep it simple.
Design for
your queries,
not just
your data."

2 Schema Validation

Use schema validation to ensure your data meets the required structure and rules.

1. Basic \$jsonSchema Example

```

db.createCollection("products", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "price"],
      properties: {
        name: { bsonType: "string" },
        price: { bsonType: "double", minimum: 0 },
        category: { bsonType: "string" }
      }
    }
  }
})
  
```

Creates a collection with validation rules using `$jsonSchema`.

2. Key Elements

- `$jsonSchema` - define the schema structure.
- `required` - specify mandatory fields.
- `bsonType` - set the expected data type (e.g., string, int, double, object, array).
- Validation rules - use keywords like `minimum`, `maximum`, `enum`, `pattern`, etc.
- Additional fields - control with `additionalProperties` (true/false).

Common bsonType Values

Type	Description
string	Text value
int	32-bit integer
long	64-bit integer
double	Floating point number
bool	Boolean value
object	Embedded document
array	Array value
date	Date value
objectId	ObjectId value

3. More Validation Rules

- Use `enum` to restrict values.
- Use `pattern` for string validation (e.g., regex).
- Use `minimum` and `maximum` for numbers.
- Use `minItems` and `maxItems` for arrays.
- Use `additionalProperties` to allow or disallow extra fields.

Example: Advanced Validation

```

validator: {
  $jsonSchema: {
    bsonType: "object",
    required: ["name", "price", "category"],
    properties: {
      name: { bsonType: "string", minLength: 2 },
      price: { bsonType: "double", minimum: 0 },
      category: { enum: ["Book", "Electronics", "Clothing"] }
    },
    additionalProperties: false
  }
}
  
```

Validates required fields, data types, and allowed values and disallows extra fields.

4. Schema Evolution

- Start with basic validation and evolve over time.
- Use `collMod` to update validation rules.
- Keep validation flexible to support future changes.
- Monitor and adjust as your application grows.

```

db.runCommand({
  collMod: "products",
  validator: { $jsonSchema: { /* new rules */ } }
})
  
```

Updates the validation rules for an existing collection.

"Valid data.
Reliable applications.
Lasting impact."



Clean data.
Stronger apps.
Brighter tomorrows.

"Model and manage your data with purpose."

Delete & Bulk Operations & Data Modeling

Clean data. Process at scale. Design for real needs.

"Good data design today. Fewer problems tomorrow."

1 Delete Documents

Use `deleteOne()` and `deleteMany()` to remove documents that match a filter.

`deleteOne()` - Delete a Single Document

Deletes the first document that matches the filter.

```
db.users.deleteOne(
  { _id: 1 }
)
```

Deletes only the first matching document.

`deleteMany()` - Delete Multiple Documents

Deletes all documents that match the filter.

```
db.users.deleteMany(
  { age: { $lt: 18 } }
)
```

Deletes all matching documents (e.g., all users younger than 18).

2 Bulk Operations

Use `bulkWrite()` to perform multiple operations (insert, update, delete) in a single request.

```
db.users.bulkWrite([
  { insertOne: { document: { name: "Alice", age: 25 } } },
  { updateOne: { filter: { name: "Bob" },
    update: { $set: { age: 26 } } } },
  { deleteMany: { filter: { age: { $lt: 18 } } } }
])
```

`bulkWrite()` can perform different write operations in one go, improving performance.

Ordered Bulk Writes (Default)

- Operations run in the order specified.
- Stops at the first error.
- Use when write order matters.

```
db.users.bulkWrite(operations,
  { ordered: true })
```

Unordered Bulk Writes

- Operations can run in parallel (in no particular order).
- Continues even if some operations fail.
- Use for better performance and independent operations.

```
db.users.bulkWrite(operations,
  { ordered: false })
```

3 Data Modeling

Design your data model around your application queries and use cases.

- Understand your application's read and write patterns.
- Choose between embedding and referencing.
- Keep your data model simple and flexible.
- Avoid over-normalization.
- Denormalize when it improves read performance.

5 When to Use What?

Scenario	Embed	Reference
One-to-few relationship	✓	—
One-to-many relationship	—	✓
Data accessed together	✓	—
Data accessed separately	—	✓
Small, bounded data	✓	—
Large, unbounded data	—	✓
High read performance needed	✓	—
Avoid data duplication	—	✓

4 Embedding vs Referencing

Choose the right approach based on your use case.

Embedded Documents

- Store related data in a single document.
- Best for one-to-few relationships.
- Fast reads (no joins).
- Simpler data model.
- Useful when related data is always accessed together.

Example: Embedded (User with Address)

```
{
  _id: 1,
  name: "Alice",
  address: {
    city: "New York",
    country: "USA"
  }
}
```

Good when a user always has one address and it is always needed.

Referenced Documents

- Store related data in separate collections and use references (e.g., ObjectId).
- Best for one-to-many or many-to-many relationships.
- Avoids data duplication.
- More flexible and scalable for large datasets.
- Useful when related data is accessed separately.

Example: Referenced (User with Orders)

```
users collection
{
  _id: 1,
  name: "Alice"
}

orders collection
{
  _id: 101,
  userId: ObjectId("1"),
  product: "Laptop",
  amount: 1000
}
```

Good when a user has many orders and they are queried separately.

6 Tips for Effective Data Modeling

- ✓ Start with your most important queries.
- ✓ Keep documents small (ideally < 16 MB).
- ✓ Use arrays for related data (e.g., tags, comments).
- ✓ Consider indexing for query performance.
- ✓ Revisit and evolve your data model as your application grows.
- ✓ There's no one-size-fits-all - choose what works best for your use case.



"Model smart. Build faster."

"The right model makes great applications possible."



Better data. Brighter possibilities.

"Keep your data
fresh and flexible."

Update & Replace Documents

Modify your data. Keep it up to date.

"Data evolves.
Your application
should too."

1 Update & Replace Documents

Use `updateOne()`, `updateMany()`, `replaceOne()`, and `upserts` to modify existing documents or insert if not found.

`updateOne()` - Update a Single Document

Updates the first document that matches the filter.

```
db.users.updateOne(
  { _id: 1 },
  { $set: { name: "Alice" } }
)
```

Updates only the first matching document.

`updateMany()` - Update Multiple Documents

Updates all documents that match the filter.

```
db.users.updateMany(
  { age: { $gte: 18 } },
  { $set: { isAdult: true } }
)
```

Updates all matching documents.

`replaceOne()` - Replace a Document

Replaces the entire document (except `_id`).

```
db.users.replaceOne(
  { _id: 1 },
  { _id: 1, name: "Alice", age: 25,
    email: "alice@example.com" }
)
```

Replaces the whole document with a new one.

Upserts - Update or Insert

If no document matches, insert a new one (using `upsert: true`).

```
db.users.updateOne(
  { email: "bob@example.com" },
  { $set: { name: "Bob", age: 28 } },
  { upsert: true }
)
```

Updates if found, otherwise inserts a new document.

2 Update Operators

Use update operators to modify specific fields without replacing the entire document.

Operator	Description	Example
\$set	Set the value of a field	{ \$set: { name: "Alice" } }
\$unset	Remove a field	{ \$unset: { age: "" } }
\$inc	Increment (or decrement) a number	{ \$inc: { age: 1 } }
\$push	Add a value to an array	{ \$push: { hobbies: "Travel" } }
\$pull	Remove a value from an array	{ \$pull: { hobbies: "Music" } }
\$addToSet	Add a value to an array only if it doesn't exist	{ \$addToSet: { hobbies: "Reading" } }
Array Filters	Update specific elements in an array	{ \$set: { "items.\$[i].qty": 5 } }, { arrayFilters: [{ "i.name": "book" }] }

3 More Examples

Use `$unset` to Remove a Field

```
db.users.updateOne(
  { _id: 1 },
  { $unset: { age: "" } }
)
```

Removes the age field.

Use `$inc` to Increment a Value

```
db.products.updateOne(
  { _id: 1 },
  { $inc: { quantity: 5 } }
)
```

Increases the quantity by 5.

Use `$push` and `$addToSet` with Arrays

```
db.users.updateOne(
  { _id: 1 },
  {
    $push: { hobbies: "Music" },
    $addToSet: { hobbies: "Reading" }
  }
)
```

Adds "Music" to the array and adds "Reading" only if it's not already present.

4 Example: Update with Array Filters

Update specific elements in an array using `arrayFilters`.

```
db.orders.updateMany(
  { "items.name": "book" },
  { $set: { "items.$[i].qty": 5 } },
  { arrayFilters: [ { "i.name": "book" } ] }
)
```

Sets `qty` to 5 for all items with name "book" in the items array.

5 Things to Remember

- ✓ `updateOne()` updates the first matching document.
- ✓ `updateMany()` updates all matching documents.
- ✓ `replaceOne()` replaces the entire document (except `_id`).
- ✓ Use `upsert: true` to insert if no document matches.
- ✓ Use update operators to modify specific fields.
- ✓ Array filters let you update specific array elements.



"Small changes.
Big impact."

"Update today for a better tomorrow."



Same data.
Greater possibilities.

Nested Documents & Array Queries

Sorting, Pagination & Counting



MongoDB.
Build what's next.

"Explore deeper, query smarter."

Handle complex data. Get precise results.

"Complex data. Simple queries. Real results."

1 Nested Documents & Array Queries

Work with embedded documents and arrays using powerful query operators.

Dot Notation (Nested Fields)

- Use dot notation to query fields inside embedded documents.
- You can also use it to query fields inside documents within arrays.

```
// Find users from a specific address city
db.users.find({
  "address.city": "New York"
})
```

Access nested fields using dot notation.

\$elemMatch (Match Array Elements)

- Use \$elemMatch to match multiple conditions on the same array element.

```
db.students.find({
  scores: {
    $elemMatch: { subject: "Math",
                  score: { $gte: 80 } }
  }
})
```

Matches an array element with both conditions.

\$all (Match All Values in Array)

- Use \$all to find documents where an array contains all specified values.

```
db.students.find({
  hobbies: { $all: ["Reading", "Music"] }
})
```

Find users who have both "Reading" and "Music" in their hobbies.

\$size (Match Array Size)

- Use \$size to match arrays with a specific number of elements.

```
db.students.find({
  hobbies: { $size: 3 }
})
```

Find users with exactly 3 hobbies.

Example Document (Nested + Array)

```
{
  _id: ObjectId("64f7c2e9b1e8a1d4f9a12345"),
  name: "Alice",
  address: {
    city: "New York",
    country: "USA"
  },
  hobbies: ["Reading", "Music", "Travel"],
  scores: [
    { subject: "Math", score: 85 },
    { subject: "English", score: 78 }
  ]
}
```

Nested document and arrays in a single document.



More Examples

Query	Description
db.users.find({ "address.city": "Boston" })	Match nested field
db.students.find({ hobbies: "Music" })	Match array value (simpler form)
db.students.find({ scores: { \$elemMatch: { subject: "English", score: { \$gte: 70 } } } })	Match multiple conditions in array element
db.students.find({ hobbies: { \$all: ["Music", "Travel"] } })	Array contains all values
db.students.find({ hobbies: { \$size: 3 } })	Array size is 3

2 Sorting, Pagination & Counting

Organize, limit, and count your results.

Sorting with sort()

- Use sort() to order the results.
- 1 for ascending, -1 for descending.

```
db.users.find().sort({ age: 1 })
```

Sort by age (ascending).

```
db.users.find().sort({ name: -1 })
```

Sort by name (descending).

Limiting Results with limit()

- Use limit() to return a specific number of documents.

```
db.users.find().limit(5)
```

Return only 5 documents.

Skipping Results with skip()

- Use skip() to skip a number of documents (useful for pagination).

```
db.users.find().skip(10).limit(5)
```

Skip 10 documents and return next 5.

Range Pagination (Page-based)

- To get page N with page size P: skip((N - 1) * P).limit(P)

```
const page = 2;
const pageSize = 5;
db.users.find()
  .skip((page - 1) * pageSize)
  .limit(pageSize)
```

Get page 2 (documents 6-10).

Counting Documents

- Use countDocuments() to count matching documents.

```
db.users.countDocuments({ age: { $gte: 18 } })
```

Count users with age >= 18.

Distinct Values with distinct()

- Use distinct() to get unique values from a field.

```
db.users.distinct("country")
```

Get unique country values.

Returns an array of distinct values (e.g., ["USA", "UK"]).

"Query smart. Handle more. Do more."



"From data to meaning."



Same data. More possibilities.

"Ask the right questions, get better data."

Read Documents & Query Operators

Find. Filter. Explore. Get the data you need.

"Powerful queries turn data into insights."

1 Read Documents

Use `find()` and related methods to read data from your collections.

- `find()` returns multiple documents.
- `findOne()` returns a single document.
- Use query filters to match specific documents.
- Use projections to select only the fields you need.
- Cursors are used to iterate through query results (efficient for large datasets).

find() - Return Multiple Documents

```
db.users.find()
{ _id: ObjectId("..."), name: "Alice", age: 25 }
{ _id: ObjectId("..."), name: "Bob", age: 28 }
{ _id: ObjectId("..."), name: "Carol", age: 22 }
```

← Returns all documents in the collection.

findOne() - Return a Single Document

```
db.users.findOne({ name: "Alice" })
{ _id: ObjectId("..."), name: "Alice", age: 25 }
```

← Returns the first matching document.

Query Filters - Find Matching Documents

```
db.users.find({ age: { $gt: 25 } })
{ _id: ..., name: "Bob", age: 28 }
{ _id: ..., name: "David", age: 30 }
```

← Find users with age greater than 25.

Projections - Select Specific Fields

```
db.users.find(
  { age: { $gte: 25 } },
  { name: 1, age: 1, _id: 0 }
)
{ name: "Bob", age: 28 }
{ name: "David", age: 30 }
```

← Include only name and age. Exclude `_id`.

Cursors - Iterate Through Results

```
const cursor = db.users.find({})
cursor.forEach(doc => {
  print(doc.name)
})
Alice
Bob
Carol
```

← Cursors help handle large result sets efficiently.

2 Query Operators

Use query operators to build powerful and flexible queries.

Comparison Operators

Compare field values.

Operator	Description	Example
<code>\$eq</code>	Equal to	<code>{ age: { \$eq: 25 } }</code>
<code>\$ne</code>	Not equal to	<code>{ age: { \$ne: 25 } }</code>
<code>\$gt</code>	Greater than	<code>{ age: { \$gt: 25 } }</code>
<code>\$gte</code>	Greater than or equal	<code>{ age: { \$gte: 25 } }</code>
<code>\$lt</code>	Less than	<code>{ age: { \$lt: 25 } }</code>
<code>\$lte</code>	Less than or equal	<code>{ age: { \$lte: 25 } }</code>

Logical Operators

Combine multiple conditions.

Operator	Description	Example
<code>\$and</code>	All conditions true	<code>{ \$and: [{ age: { \$gt: 20 } }, { age: { \$lt: 30 } }] }</code>
<code>\$or</code>	Any condition true	<code>{ \$or: [{ age: { \$lt: 20 } }, { age: { \$gt: 30 } }] }</code>
<code>\$not</code>	Invert a condition	<code>{ age: { \$not: { \$eq: 25 } } }</code>
<code>\$nor</code>	None of the conditions true	<code>{ \$nor: [{ age: 20 }, { age: 30 }] }</code>

Element Operators

Query based on field existence, type, or array elements.

Operator	Description	Example
<code>\$exists</code>	Check if field exists	<code>{ email: { \$exists: true } }</code>
<code>\$type</code>	Match field type	<code>{ age: { \$type: "int" } }</code>
<code>\$in</code>	Match any value in array	<code>{ status: { \$in: ["active", "pending"] } }</code>
<code>\$nin</code>	Not in array	<code>{ status: { \$nin: ["inactive"] } }</code>
<code>\$elemMatch</code>	Match array elements	<code>{ tags: { \$elemMatch: { \$eq: "db" } } }</code>
<code>\$size</code>	Match array size	<code>{ tags: { \$size: 3 } }</code>

Regular Expression Queries

Search for patterns in string fields.

Operator	Description	Example
<code>\$regex</code>	Match pattern	<code>{ name: { \$regex: "A" } }</code>
<code>\$options</code>	Set regex options	<code>{ name: { \$regex: "alice", \$options: "i" } }</code> (i = case insensitive)

Example: Find Users with Name Starting with 'A'

```
db.users.find({ name: { $regex: "A", $options: "i" } })
{ _id: ..., name: "Alice", age: 25 }
{ _id: ..., name: "anna", age: 27 }
```

← Find names starting with "A" (case insensitive).

"Queries today. Answers tomorrow."



Explore. Query. Discover.



"Same data. Bigger insights."



"Small commands,
big possibilities."

Database & Collection Commands

Organize your data. One command at a time.

"Databases store
possibilities.
Collections organize
them."

1 Database Commands

Create, list, rename, and drop databases.

- In MongoDB, a database is a container for collections.
- Databases are created automatically when you insert data.
- You can switch between databases using the `use` command.
- You can list, rename (copy + drop), and drop databases using simple commands.

Create / Switch to a Database

```
> use myDB
switched to db myDB
```

Creates the database if it doesn't exist and switches to it.

Rename a Database (copy data to new DB, then drop old)

```
> use oldDB
> db.copyDatabase("oldDB", "newDB")
> db.dropDatabase()
```

MongoDB doesn't have a direct rename command, so we copy and drop.

List Databases

```
> show dbs
admin      100.00 KiB
config     100.00 KiB
local      72.00 KiB
myDB       0.00 KiB
```

Drop a Database

```
> use myDB
> db.dropDatabase()
{ "dropped" : "myDB",
  "ok" : 1 }
```

2 Collection Commands

Create, list, rename, and drop collections.

- A collection is a group of documents (like a table in SQL).
- Collections are created automatically when you insert a document.
- You can also create, list, rename, and drop collections using simple commands.

Create a Collection

```
> db.createCollection("users")
{ "ok" : 1 }
```

Rename a Collection

```
> db.users.renameCollection("customers")
{ "ok" : 1 }
```

List Collections

```
> show collections
users
products
orders
```

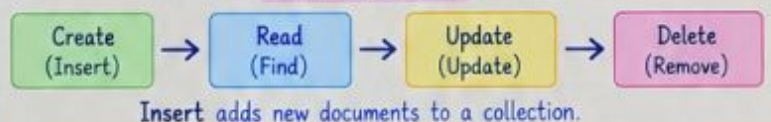
Drop a Collection

```
> db.customers.drop()
{ "ok" : 1 }
```

3 CRUD: Insert Documents

Add data to your collections.

CRUD Overview



- CRUD stands for Create, Read, Update, Delete.
- Insert operations add new documents to a collection.
- Use `insertOne()` to insert a single document.
- Use `insertMany()` to insert multiple documents.
- MongoDB automatically adds a unique `_id` (ObjectId) if not provided.

insertOne() - Insert a Single Document

```
> db.users.insertOne({
  name: "Alice",
  age: 25,
  email: "alice@example.com"
})
{
  "acknowledged" : true,
  "insertedId" : ObjectId("64f7c2e9b1e8a1d4f9a12345")
}
```

insertMany() - Insert Multiple Documents

```
> db.users.insertMany([
  { name: "Bob", age: 28 },
  { name: "Carol", age: 22 },
  { name: "David", age: 30 }
])
{
  "acknowledged" : true,
  "insertedIds" : [
    ObjectId("64f7c2e9b1e8a1d4f9a12346"),
    ObjectId("64f7c2e9b1e8a1d4f9a12347"),
    ObjectId("64f7c2e9b1e8a1d4f9a12348")
  ]
}
```

Sample Documents in Collection

```
> db.users.find().pretty()
{
  _id: ObjectId("64f7c2e9b1e8a1d4f9a12345"),
  name: "Alice",
  age: 25,
  email: "alice@example.com"
}
{
  _id: ObjectId("64f7c2e9b1e8a1d4f9a12346"),
  name: "Bob",
  age: 28
}
```

MongoDB
adds a unique
`_id` (ObjectId)
automatically.

"Insert today. Build tomorrow."



More data.
More possibilities.

"The cloud for your data and bigger ideas."

MongoDB Atlas

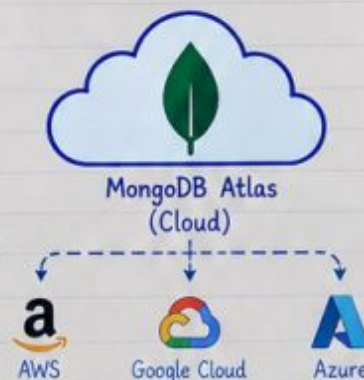
Build. Deploy. Scale. Anywhere.

MongoDB.
Build what's next.

"Managed MongoDB. More time building, less time managing."

1 What is MongoDB Atlas?

- MongoDB Atlas is a fully managed, cloud-hosted database service by MongoDB.
- It handles infrastructure, backups, security, scaling, and maintenance.
- Available on AWS, Google Cloud, and Microsoft Azure.
- Free tier available to get started.
- Access your data from anywhere, anytime.



Key Benefits

- ✓ Fully managed
- ✓ Highly available
- ✓ Secure & scalable
- ✓ Global cloud regions
- ✓ Easy to use
- ✓ Free tier

2 Cloud Database Setup

- 1 Create a MongoDB account at <https://www.mongodb.com/atlas>
- 2 Create a new project (e.g., "MyProject").
- 3 Build a cluster (choose cloud provider, region, and cluster tier).
- 4 Wait for the cluster to be deployed (a few minutes).
- 5 Your cloud database is ready to use!

The screenshot shows the "Create a Cluster" form. It has three tabs for "Provider": AWS, Google Cloud, and Azure. The "Region" dropdown is set to "Frankfurt (eu-central-1)". The "Cluster Tier" dropdown is set to "M0 (Free) - Shared RAM". A green "Create Cluster" button is at the bottom right.

Choose your cloud provider, region and cluster tier.

3 Clusters

- A cluster is a set of MongoDB servers that host your data.
- Choose a cloud provider (AWS, Google Cloud, Azure).
- Select a region close to your users.
- Start with a free M0 cluster (great for learning).
- You can scale up anytime (vertical or horizontal).

Clusters

Name	Status	Provider	Region	Tier	
myCluster	Active	AWS	Frankfurt (eu-central-1)	M0 (Free)	...

Create Cluster

4 Database Users

- Create database users to control access.
- Set a username and a strong password.
- Choose appropriate roles (e.g., readWrite, readOnly).
- Use least privilege principle for security.

Database Access

+ Add New Database User

Username	Authentication Method	Roles	Actions
myUser	Password	readWrite	

5 Network Access

- Control which IP addresses can access your cluster.
- Add your current IP address (or 0.0.0.0/0 for access from anywhere - not recommended for production).
- You can also allow access from specific IP ranges.

Network Access

+ Add IP Address

IP Address	Description	Status	
203.0.113.25	My Home IP	Active	
0.0.0.0/0	Allow from anywhere	Active	

6 Connection Strings

- Use the connection string to connect your application or MongoDB Shell (mongosh) to Atlas.
- Copy the connection string from the cluster dashboard.
- Replace <password> with your database user's password.
- Use it in your application or mongosh.

Connect to your application

Drivers

mongodb+srv://myUser:<password>@mycluster.abcde.mongodb.net/?retryWrites=true&w=majority

Replace <password> with your actual password.

Copy connection string

"From zero to global, with Atlas."



"Same data. Bigger possibilities."

"Set up today,
build tomorrow."

Installation & Tools MongoDB

Get MongoDB up and running. Explore. Build. Learn.



"Same tools.
Bigger ideas."

1 MongoDB Community Server

- Free and open-source version of MongoDB.
- Run locally for learning and development.
- Supports all core features (documents, collections, replica sets, etc.).
- Available for Windows, macOS, and Linux.
- Download from the official MongoDB website.

Download & Install

- 1 Go to <https://www.mongodb.com/try/download/community>
- 2 Choose your operating system.
- 3 Follow the installation instructions.
- 4 Verify the installation:

```
$ mongod --version
db version v7.0.0
Build Info: { "version": "7.0.0" ... }
```

Check the
installed
version.

2 MongoDB Shell (mongosh)

- mongosh is the modern MongoDB shell.
- Allows you to interact with MongoDB using JavaScript.
- Supports autocomplete, syntax highlighting, and helpful errors.
- Useful for running queries, managing databases, and performing administrative tasks.
- Included with MongoDB tools or can be installed separately.

Start Using mongosh

```
$ mongosh
Current Mongosh Log ID: 64f7c2e9b1e8...
Connecting to: mongodb://127.0.0.1:27017
Using MongoDB: 7.0.0
For mongosh info: type "help"

> test>
```

Start the
shell and
connect to
your local
MongoDB
server.

Common mongosh Commands

show dbs	Show all databases
use <db>	Switch to a database (creates if not exists)
show collections	Show all collections
db.<collection>.find()	Query documents
db.<collection>.insertOne({...})	Insert a document
db.<collection>.deleteOne({...})	Delete a document

"Explore data.
Learn by doing."

3 MongoDB Compass

- A graphical user interface (GUI) for MongoDB.
- Visualize and explore your data.
- Create, read, update, and delete documents.
- View and manage databases, collections, and indexes.
- Run queries with a visual builder.
- Available for Windows, macOS, and Linux.

Features

- ✓ Easy-to-use UI
- ✓ Visualize documents
- ✓ Query builder
- ✓ Schema insights
- ✓ Manage indexes
- ✓ Real-time statistics



MongoDB Compass
Explore your data visually.

Getting Started

- 1 Download Compass from:
<https://www.mongodb.com/try/download/compass>
- 2 Install and open Compass.
- 3 Connect to your local MongoDB server (e.g.,
mongodb://127.0.0.1:27017).
- 4 Start exploring your databases and collections.

MongoDB Compass			
My Queries Databases Performance Schema Indexes			
Databases Create Database			
Search databases...			
Database Name	Collections	Documents	Size
admin	0	0	0 B
config	1	3	12 KB
local	1	1	8 KB
test	2	5	36 KB

"Tools make it easier. Practice makes it real."



"Small steps
lead to big builds."

"Data in a format that fits your world."

JSON, BSON & Data Types

Same idea. More possibilities.

"Flexible data for real-world applications."

1 What is JSON?

- JSON (JavaScript Object Notation) is a lightweight, text-based data format.
- It is easy to read and write.
- Commonly used in web applications and APIs.
- MongoDB uses a binary version called BSON for storage and additional data types.

JSON Example

```
{
  "name": "Alice",
  "age": 25,
  "isStudent": true,
  "skills": ["MongoDB", "Python"]
}
```

Simple, text-based format (used in APIs, web apps, etc.)

2 What is BSON?

- BSON (Binary JSON) is a binary-encoded format used by MongoDB to store data.
- It supports all JSON data types and additional types (like Date, ObjectId, etc.).
- BSON is more efficient for storage, indexing, and processing within MongoDB.

BSON Example (same data, additional type)

```
{
  "_id": ObjectId("64f7c2e9b1e8a1d4f9a12345"),
  "name": "Alice",
  "age": 25,
  "isStudent": true,
  "createdAt": ISODate("2024-08-01T10:00:00Z"),
  "skills": ["MongoDB", "Python"]
}
```

BSON is binary and supports additional data types (like ObjectId, Date, etc.).

3 Common MongoDB Data Types

Data Type	Description	Example
String	Stores text data (UTF-8).	"name": "Alice"
Number	Stores numeric values (integers or decimals).	"age": 25, "price": 99.5
Boolean	Stores true or false values.	"isActive": true
Date	Stores date and time values.	"createdAt": ISODate("2024-08-01T10:00:00Z")
Array	Stores multiple values in a list.	"skills": ["MongoDB", "Python", "JavaScript"]
Nested Object	Stores embedded documents (documents inside documents).	"address": { "city": "New York", "zip": "10001" }
_id	A unique identifier for each document (created automatically).	"_id": ObjectId("64f7c2e9b1e8a1d4f9a12345")
ObjectId	A 12-byte unique identifier (includes timestamp, machine id, etc.).	ObjectId("64f7c2e9b1e8a1d4f9a12345")

4 Complete Document Example

```
{
  "_id": ObjectId("64f7c2e9b1e8a1d4f9a12345"),
  "name": "Alice",
  "age": 25,
  "isStudent": true,
  "joinedAt": ISODate("2024-08-01T10:00:00Z"),
  "skills": ["MongoDB", "Python", "JavaScript"],
  "address": {
    "city": "New York",
    "zip": "10001"
  }
}
```

← Unique identifier (ObjectId)
← String
← Number
← Boolean
← Date
← Array
} Nested object (embedded document)

5 Key Takeaways

- ✓ JSON is a human-readable format.
- ✓ BSON is the binary format used by MongoDB.
- ✓ MongoDB supports rich data types (string, number, boolean, date, array, object, ObjectId, etc.).
- ✓ Documents can be nested and flexible.
- ✓ _id is automatically generated if not provided.
- ✓ ObjectId ensures global uniqueness across collections and databases.
- ✓ Flexible data types make MongoDB suitable for real-world, evolving data.

From simple data to powerful applications.



"Same data. More possibilities. That's MongoDB."

"Big ideas
start with
simple data."

MongoDB Fundamentals



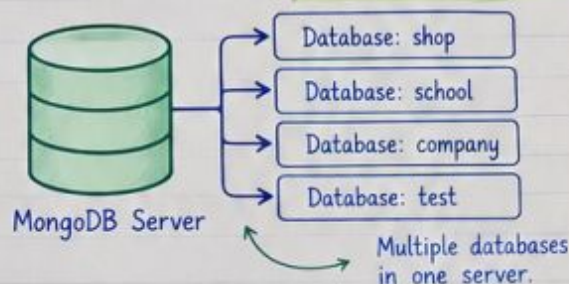
MongoDB.
Build what's next.

Flexible data.
Real possibilities.

Understand the building blocks of MongoDB.

1 What are Databases?

- A MongoDB database is a container for collections.
- It helps organize related data (like a project or application).
- You can have multiple databases in a single MongoDB server.
- Databases are created automatically when you insert data (no need to predefine).



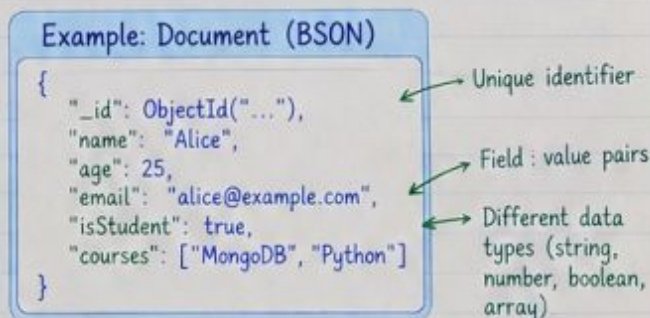
2 What are Collections?

- A collection is a group of related documents (like a table in SQL).
- Collections do not enforce a fixed schema.
- You can store documents with different fields in the same collection.
- Collections are also created automatically when you insert data.



3 What are Documents?

- A document is a record in MongoDB, stored in BSON (Binary JSON).
- Documents are similar to rows in a relational database, but more flexible.
- Each document can have a different structure.
- Documents store data in key-value pairs (fields).



4 What are Fields?

- Fields are the key-value pairs inside a document.
- A field stores a single piece of information.
- Fields can be of different data types (string, number, boolean, array, object, etc.).
- You can add new fields anytime — no schema changes needed!

Common Field Types

- String "name": "Alice"
- Number "age": 25
- Boolean "isActive": true
- Array "tags": ["db", "nosql"]
- Object "address": { "city": "NY" }
- Date "createdAt": ISODate("...")
- Null "middleName": null

5 Flexible Schemas

- MongoDB does not require a fixed schema.
- Each document in a collection can have different fields.
- You can add, remove, or modify fields without affecting existing documents.
- This flexibility makes MongoDB ideal for rapidly changing data and diverse applications.



6 Quick Summary

Concept	Purpose	Analogy (SQL)	Example
Database	Container for collections	Database	shop
Collection	Group of related documents	Table	users
Document	Single record of data	Row	{ name: "Alice", age: 25 }
Field	Key-value pair in a document	Column	name, age, email
Flexible Schema	No fixed structure, can evolve	Not applicable	Different fields in each document

"Same data.
Different structures.
More freedom."



SQL vs NoSQL

Two Approaches. Different Needs. Both Powerful.

Relational vs Non-Relational

Relational Databases (SQL)	Non-Relational Databases (NoSQL)
- Structured data - Predefined schema - Data stored in tables - Uses SQL (Structured Query Language) - Best for complex queries and transactions - Examples: MySQL, PostgreSQL, Oracle, SQL Server	- Flexible / schema-less data - Dynamic schema - Data stored in collections - Uses various query languages (e.g., MongoDB Query Language) - Best for scalability and handling large volumes of unstructured data - Examples: MongoDB, Cassandra, Redis, DynamoDB

"Different data.
Different problems.
Different solutions."

Key Differences

Feature	SQL (Relational)	NoSQL (Non-Relational)
Data model	Relational (tables)	Non-relational (collections, key-value, document, graph, wide-column)
Schema	Fixed schema	Flexible / schema-less
Query language	SQL	Varies (e.g., MongoDB Query Language)
Data storage	Tables	Collections
Data unit	Rows	Documents
Relationships	Uses joins	Embeds data or uses references
Scalability	Vertical scaling (scale up)	Horizontal scaling (scale out)
Best for	Structured data, complex transactions	Unstructured/semi-structured data, large scale applications

Tables vs Collections

SQL Table (Relational)	MongoDB Collection (Non-Relational)												
Users (Table) <table><tr><th>id</th><th>name</th><th>email</th></tr><tr><td>1</td><td>Alice</td><td>alice@example.com</td></tr><tr><td>2</td><td>Bob</td><td>bob@example.com</td></tr><tr><td>3</td><td>Carol</td><td>carol@example.com</td></tr></table>	id	name	email	1	Alice	alice@example.com	2	Bob	bob@example.com	3	Carol	carol@example.com	users (Collection) <pre>{ "_id": ObjectId("..."), "name": "Alice", "email": "alice@example.com"}, { "_id": ObjectId("..."), "name": "Bob", "email": "bob@example.com"}</pre>
id	name	email											
1	Alice	alice@example.com											
2	Bob	bob@example.com											
3	Carol	carol@example.com											

Rows vs Documents

Row (SQL)

1	Alice	alice@example.com
↑ Column (id)	↑ Column (name)	↑ Column (email)

Document (MongoDB)

<pre>{ "_id": ObjectId("..."), "name": "Alice", "email": "alice@example.com", "age": 25, "isActive": true }</pre>	Single document with flexible fields
---	--------------------------------------

When to Use What?

Use SQL (Relational) when:

- ☒ You have structured data
- ☒ You need complex queries (joins)
- ☒ Data integrity and ACID transactions are critical
- ☒ You have a fixed schema
- ☒ Example use cases: banking systems, ERP, inventory management

Use NoSQL (e.g., MongoDB) when:

- ☒ You deal with unstructured or rapidly changing data
- ☒ You need high scalability and performance
- ☒ You want flexible schema
- ☒ You handle large volumes of data
- ☒ Example use cases: real-time analytics, content management, IoT, mobile apps

"Right database.
Brighter possibilities."

"Flexible data
for a faster world."

Introduction to MongoDB

A Modern, Flexible, and Scalable NoSQL Database



"Store data
as documents.
Think flexible.
Build faster."

1 What is MongoDB?

- MongoDB is a NoSQL, document-oriented database.
- It stores data in flexible, JSON-like documents (BSON).
- Designed for modern applications that need scalability, performance, and flexibility.
- Developed by MongoDB Inc. and widely used across industries.

2 Document Databases

- MongoDB is a document database, which means it stores data as documents instead of rows in tables.
- Documents are written in a JSON-like format called BSON (Binary JSON).
- Each document can have a different structure, making it schema-flexible.
- Documents can contain nested data, arrays, and complex data types.

Example: MongoDB Document (BSON)

```
{
  "_id": ObjectId("..."),
  "name": "Alice",
  "age": 25,
  "email": "alice@example.com",
  "skills": ["MongoDB", "Python", "Cloud"],
  "address": {
    "city": "New York",
    "country": "USA"
  }
}
```

Unique ID
Field : value
pairs
Nested document
Array

3 Key Features

- ✓ Flexible Schema – No fixed structure, easy to evolve.
- ✓ High Performance – Designed for high read and write throughput.
- ✓ Horizontal Scalability – Scale out easily using sharding.
- ✓ Rich Query Language – Supports powerful queries and aggregations.
- ✓ Replication – Built-in replica sets for high availability.
- ✓ Wide Language Support – Drivers for many programming languages.
- ✓ Transactions – Supports multi-document ACID transactions (in modern versions).



Flexible
Documents



High
Performance



Scalable



Reliable

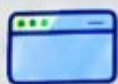
4 Advantages

- ✓ Handles unstructured and semi-structured data easily.
- ✓ Scales horizontally (scale out) to handle large volumes of data.
- ✓ Faster development with flexible schema.
- ✓ High availability and fault tolerance (replica sets).
- ✓ Supports modern data types (arrays, embedded documents, etc.).
- ✓ Large and active community.
- ✓ Ideal for modern applications (web, mobile, IoT, real-time analytics).

More flexibility.
More possibilities.



5 Common Use Cases



Web & Mobile
Applications
User data, content
management



Content Management
Systems
Articles, media,
digital assets



E-commerce
Product catalogs,
orders, user reviews



Real-time Analytics
Event data,
user behavior



IoT
Sensor data,
real-time processing



Social Networks
User profiles,
interactions

MongoDB → Flexible Data → Real-World Impact

"Ideas scale with MongoDB."

"From Data to Real-World Impact."

A Database for Modern Developers

"Better Developers Build a Brighter Tomorrow." 😊

- ✓ Learn
- ✓ Practice
- ✓ Build
- ✓ Scale
- ✓ Master

- 🗄 Document Database
- </> Flexible Schema
- ⚡ High Performance
- ☁ Cloud & On-Premise
- 👥 Developer Friendly
- 🌐 Global Community

"Data is a precious thing and will last longer than the systems themselves."

- Dwight D. Eisenhower

mongoDB

"Same Data. Bigger Opportunities."

— COMPLETE NOTES —

Learn • Practice • Build • Scale • Master



Document Model



Real-World Applications



Scalability & Performance



Security & Reliability



MongoDB Atlas



Interview & Career Ready

IDEAS

QUERIES

APPLICATIONS

OPPORTUNITIES

Modern Data for a Brighter Tomorrow."

- ✓ Learn Today.
- ✓ Build Tomorrow.
- ✓ Grow Forever.



"Code Learn Build Repeat"

Real Skills. Real Projects. Real Impact."



1 What is State Management?

State management is the way we store, update and share data in an application. It helps keep the UI in sync with the data and makes the app interactive.



State connects
your data and UI.

Key Points:

- ✓ State is data that can change over time
- ✓ When state changes, UI updates automatically
- ✓ Can be local (component) or global (app-wide)
- ✓ Essential for interactive applications
- ✓ Helps manage data flow and application logic

"State turns a static UI into a dynamic experience."

4 Props & State

Props are used to pass data from parent to child, while state is used to manage data within a component. Both work together.

```
function UserProfile({ name }) {
  const [age, setAge] = useState(18);

  return (
    <View>
      <Text>Name (from props): {name}</Text>
      <Text>Age (from state): {age}</Text>
      <Button title="Increase Age"
        onPress={() => setAge(age + 1)} />
    </View>
  );
}
```

Props come
from outside.
State lives
inside.

Key Points:

- ✓ Props are read-only
- ✓ State is mutable (can be updated)
- ✓ Props are passed from parent to child
- ✓ State is managed within the component
- ✓ Both are essential in React development

"Props pass data. State manages data."

2 Local State vs Global State

Local state is managed within a single component, while global state is shared across multiple components.

Local State	Global State
Component specific	Shared across components
Managed using useState or useReducer	Managed using Context, Redux, Zustand, etc.
Simple to use	More setup required
Good for small data	Useful for shared/complex data
Example: form input, toggle	Example: user auth, theme, cart

Key Points:

- ✓ Use local state for component-specific data
- ✓ Use global state when data is needed in multiple components
- ✓ Don't overuse global state

"Keep state local, until you need to share it."

3 React useState

useState is a React hook that lets you add state to functional components.

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <View style={{ padding: 20 }}>
      <Text>Count: {count}</Text>
      <Button title="Increase" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Simple,
powerful
and widely
used!

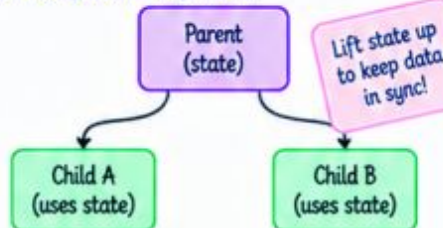
Key Points:

- ✓ Returns a state variable and a function to update it
- ✓ Initial value can be any data type
- ✓ Updating state triggers a re-render
- ✓ Can be used multiple times in a component
- ✓ Commonly used for forms, toggles, counters, etc.

"useState makes your component stateful."

5 Lifting State Up

When multiple components need to share the same state, we lift the state to their closest common parent component.



Key Points:

- ✓ Move state to the closest common parent
- ✓ Pass state down using props
- ✓ Helps keep multiple components in sync
- ✓ Avoids prop drilling in small to medium apps
- ✓ For large apps, consider Context or Redux

"Lift state up when components need to share data."

6 Context API

Context API provides a way to share state across multiple components without passing props manually at every level.

```
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

export const useTheme = () => useContext(ThemeContext);
```

Share data
across your
app easily!

Key Points:

- ✓ Avoids prop drilling
- ✓ Useful for theme, authentication, language, etc.
- ✓ Simple and built into React
- ✓ Best for medium-sized state management needs
- ✓ For complex state, consider Redux or Zustand

"Context makes sharing state simple."

7 When to Use Global State?

Use global state when the same data is needed in multiple components, across screens, or throughout the application.

Common Use Cases:

- 1 User authentication (login state)
- 2 Theme (dark/light mode)
- 3 Shopping cart (e-commerce)
- 4 App settings and preferences
- 5 Data shared across multiple screens
- 6 Real-time data (notifications, chat, etc.)

Not everything
needs global state.
Use it wisely!

Choose the right tool
for your needs:



Context API



Redux



Zustand

"Use global state when it adds real value, not by default."

Key Takeaways (Part 1)

- ✓ Understand what state management is
- ✓ Know the difference between local and global state
- ✓ Use useState for component state
- ✓ Understand props vs state
- ✓ Learn to lift state up when needed
- ✓ Use Context API for shared state
- ✓ Know when to use global state

Good state
management
leads to clean,
scalable apps!

"Manage state today,
build better apps tomorrow."



React Fundamentals & JavaScript Modules

From Basics to Components, Rendering Dynamic Data and Modules

1. MPA vs SPA

MPA (Multi Page Application)

- Multiple HTML pages
- Each page loads separately
- Page reload on navigation
- Simple and traditional

Example Structure

```
home.html
branches.html
contactus.html
```

Navigation

```
<a href="home.html">Home</a>
<a href="branches.html">Branches</a>
<a href="contactus.html">Contact us</a>
```

SPA (Single Page Application)

- Single main HTML page
- Dynamically updates content
- No complete page reload
- Faster and app-like experience

Example Structure

```
index.html
```

Navigation (dynamic content)

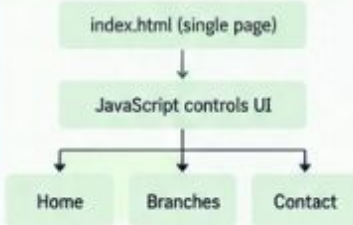
```
<button onClick="home()">Home</button>
<button onClick="branches()">Branches</button>
<button onClick="contactus()">Contact us</button>
<div id="root"></div>
```

MPA vs SPA Flow

MPA Flow



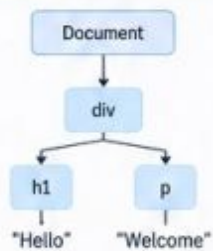
SPA Flow



Feature	MPA	SPA
HTML pages	Multiple	Single (main page)
Navigation	Loads another page	Updates current page
Page reload	Usually yes	Usually no
New experience	Traditional	App-like
JavaScript dependency	Lower	Higher
Example	Traditional websites	React applications

2. DOM and DOM Manipulation

DOM Tree



Using innerHTML

```
root.innerHTML = `
  <h1>Hello</h1>
  <p>Welcome</p>
`
```

Using DOM APIs

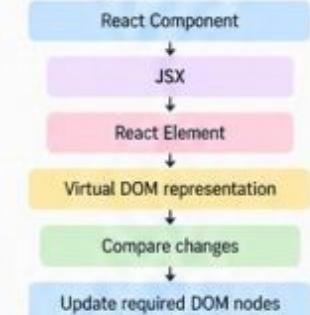
```
const h1 = document.createElement('h1')
h1.textContent = 'Hello'
root.appendChild(h1)
```

3. DOM Performance Example

```
const startTime = performance.now()
for (let i = 1; i <= 10000; i++) {
  const li = document.createElement('li')
  li.textContent = `Product - ` + i
  ol.appendChild(li)
}
const endTime = performance.now()
console.log(endTime - startTime)
```

- performance.now() provides a high-resolution timestamp useful for measuring performance.

4. React & Virtual DOM



5. React without JSX

```
const h1 = React.createElement(
  'h1',
  null,
  'Hello from My SPA'
)

const root = ReactDOM.createRoot(
  document.getElementById('root')
)
root.render(h1)
```

6. React.createElement()

```
React.createElement(
  type, // e.g. 'h1'
  props, // e.g. null
  children // e.g. 'Hello'
)
```

Example:

```
React.createElement(
  'p',
  null,
  'This is a paragraph'
)
```

7. JSX

```
const element = (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
)
```

- JSX looks like HTML but is JavaScript syntax. It is transformed into React element creation code (e.g. by Babel).

8. JSX with Babel

```
<script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
<script type="text/babel">
  const element = <h1>Hello</h1>
</script>
```

- Babel converts JSX into JavaScript that the browser can execute.

9. JSX Fragment

```
const element = (
  <>
    <h1>This is a header</h1>
    <div>
      <p>This is para 1</p>
      <p>This is para 2</p>
    </div>
  </>
)
```

10. JSX Expressions

```
const person = {
  name: 'Anil',
  age: 30
}

const element = (
  <div>
    <p>Name: {person.name}</p>
    <p>Age: {person.age}</p>
  </div>
)
```

11. React Components

```
function PersonDetails() {
  const person = {
    name: 'Anil', age: 30,
    mobile: '9876543210', city: 'pune'
  }
  return (
    <div>
      <h1>Person Details</h1>
      <p>Name: {person.name}</p>
      <p>Age: {person.age}</p>
    </div>
  )
}
```

- Component names should start with an uppercase letter.

12. Calling Components

```
createRoot(document.getElementById('root'))
  .render(
    <PersonDetails />
  )
```

13. Multiple Components

```
root.render([
  <PersonDetails />,
  <EmployeeDetails />
])
```

14. Rendering List with map()

```
const products = [
  { pid: 1, name: 'Pen', price: 10 },
  { pid: 2, name: 'Pencil', price: 5 },
  { pid: 3, name: 'Book', price: 50 },
]

const arr = products.map(p => (
  <div key={p.pid}>
    <p>Pid: {p.pid}</p>
    <p>Name: {p.name}</p>
    <p>Price: {p.price}</p>
  </div>
))

return arr
```

15. JavaScript Modules (Overview)

- Code organization
- Reusability
- Maintainability
- Avoids huge files
- Encapsulation

16. CommonJS (require & module.exports)

Math1.js

```
function add() { console.log("addition") }
function sub() { console.log("subtraction") }
function mul() { console.log("multiplication") }
module.exports = { add, subtract: sub, mul }
```

Import

```
const math = require('./Math1')
math.add()
math.subtract()
math.mul()
```

17. ES Modules (import & export)

Math3.mjs

```
export default function add() {
  console.log("addition")
}

export function sub() {
  console.log("subtraction")
}

export function mul() {
  console.log("multiplication")
}
```

Import

```
import { add, sub, mul } from './Math3.mjs'
add()
sub()
mul()
```

18. Default vs Named Export

Feature	Default Export	Named Export
Syntax	export default	export
Import braces	No	Yes
Import name	Can change	Normally same
Multiple per file	One default	Multiple named
Example	import add from './Math.js'	import { sub } from './Math.js'

19. CommonJS vs ES Modules

	CommonJS	ES Modules
Import	require()	import
Export	module.exports	export
Module system	Traditional Node.js	Standard JavaScript
File extension	.js	.js, .mjs (config dependent)

Key Takeaways

- Understand MPA vs SPA
- DOM vs React Virtual DOM
- JSX is JavaScript syntax (not HTML)
- Use components for reusability
- Use map() for rendering lists (with keys)
- Learn CommonJS and ES Modules
- Practice with React + Vite



Props + State

Pass Data | Manage Data | Build Dynamic UI



Props

Props (short for properties) are used to pass data from a parent component to a child component.



Read-only

Cannot be changed by the child component.



Passed from parent to child

Used to share data between components.



Make components reusable

Helps to create dynamic and flexible UI.

Example : Using Props

```
function Welcome({ name }) {  
  return (  
    <h2>Hello, {name}!</h2>  
  );  
}  
  
function App() {  
  return <Welcome name="Parmeshwar" />;  
}
```

Output :



State

State is used to store and manage data within a component. When state changes, the component re-renders.



Mutable (can be changed)

State can be updated inside the component.



Managed within the component

Used to handle dynamic data.



Causes re-render

When state changes, the UI automatically updates.

Example : Using State

```
import { useState } from "react";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h2>Count: {count}</h2>  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </div>  
  );  
}
```

Output :



Key Differences

Props	Feature	State
Read-only	Mutability	Can be changed
Passed from parent to child	Scope	Managed within the component
Used to share data	Purpose	Used to manage dynamic data
Does not cause re-render	Re-render	Causes re-render when updated



REACT JS – FULL COURSE NOTES

Complete Concepts • Examples • Best Practices

WHAT IS REACT?

React is a JavaScript library for building user interfaces. It is fast, declarative, component-based and maintained by Facebook (Meta).

1. INTRODUCTION

- Library for building UI
- Component Based
- Declarative
- Learn Once, Write Anywhere
- Virtual DOM
- One-way Data Binding



2. SETUP REACT PROJECT

Using Vite (Recommended)

```
# 1. Create project
npm create vite@latest my-app -- --template react
# 2. Go to project
cd my-app
# 3. Install dependencies
npm install
# 4. Run the app
npm run dev
```

Project Structure (Vite)

```
my-app/
├── node_modules/
├── public/
├── src/
│   ├── assets/
│   ├── components/
│   ├── App.jsx
│   ├── main.jsx
│   ├── index.css
│   ├── index.html
│   └── package.json
```

3. JSX (JavaScript XML)

- JSX allows writing HTML in JS.
- Transpiled to React.createElement().
- Every JSX must have a single parent element.

Example:

```
const element = (
  <div className="container">
    <h1>Hello React</h1>
    <p>This is JSX</p>
  </div>
);
```

4. COMPONENTS

Functional Component

```
function Welcome() {
  return <h2>Welcome to React</h2>;
}
export default Welcome;
```

Component Composition

```
function App() {
  return (
    <div>
      <Welcome />
      <Welcome />
    </div>
  );
}
```

5. PROPS (Properties)

- Props are read-only.
- Data is passed from parent to child.

Example:

```
function Greet(props) {
  return <h3>Hello, {props.name}!</h3>;
}
function App() {
  return <Greet name="Anchal" />;
}
```

6. STATE

- State is used for dynamic data.
- When state changes, component re-renders.

Example:

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

24. USEFUL COMMANDS

```
npm run dev      # Start development server
npm run build    # Build for production
npm run preview  # Preview production build
npm install <package> # Install a package
```

7. EVENT HANDLING

- Events in React are written in camelCase.
- Pass a function, not a string.

Example:

```
function Button() {
  function handleClick() {
    alert("Button Clicked");
  }
  return <button onClick={handleClick}>Click Me</button>;
}
```

8. CONDITIONAL RENDERING

Example:

```
function User({ isLoggedIn }) {
  if (isLoggedIn) {
    return <h3>Welcome Back!</h3>;
  } else {
    return <h3>Please Login</h3>;
  }
}
```

9. LISTS & KEYS

- Use map() to render lists.
- Key helps React identify items uniquely.

Example:

```
const fruits = ["Apple", "Banana", "Mango"];
function FruitList() {
  return (
    <ul>
      {fruits.map((item) => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  );
}
```

10. FORMS & INPUTS

Controlled Component Example:

```
import { useState } from "react";
function Form() {
  const [name, setName] = useState("");
  function handleSubmit(e) {
    e.preventDefault();
    alert("Hello " + name);
  }
  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter name"
      />
      <button type="submit">Submit</button>
    </form>
  );
}
```

11. LIFECYCLE (Class Component)

Important Methods

- constructor() – Initial setup
- componentDidMount() – After first render
- componentDidUpdate() – After update
- componentWillUnmount() – Before remove

Example:

```
class MyComponent extends React.Component {
  componentDidMount() {
    console.log("Component Mounted");
  }
  render() {
    return <h3>Lifecycle Example</h3>;
  }
}
```

12. HOOKS (Introduction)

- Hooks let you use state and other React features in functional components.

Common Hooks:

- useState – State management
- useEffect – Side effects
- useContext – Access context
- useRef – Mutable values
- useReducer – Complex state logic
- useMemo – Memoize values
- useCallback – Memoize functions

13. useState HOOK

Example:

```
import { useState } from "react";
function Example() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

14. useEffect HOOK

- Runs after render.
- Used for API calls, subscriptions, DOM updates.
- Can return a cleanup function.

Example:

```
import { useEffect, useState } from "react";
function Example() {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((res) => res.json())
      .then((data) => setData(data));
  }, []); // Empty array = run once on mount

  return () => {
    console.log("Cleanup (component unmounted)");
  };
  return <div>{data.length} posts loaded</div>;
}
```

15. useContext HOOK

- Share data across components without props drilling.

Example:

```
const ThemeContext = React.createContext();
function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}
function Toolbar() {
  return <ThemedButton />;
}
function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button>Theme: {theme}</button>;
}
```

16. useRef HOOK

- Access DOM elements or store mutable values.

Example:

```
import { useRef } from "react";
function FocusInput() {
  const inputRef = useRef(null);
  function handleClick() {
    inputRef.current.focus();
  }
  return (
    <div>
      <input ref={inputRef} />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}
```

17. useReducer HOOK

- Alternative to useState for complex state.

Example:

```
function reducer(state, action) {
  switch (action.type) {
    case "inc":
      return { count: state.count + 1 };
    case "dec":
      return { count: state.count - 1 };
    default:
      return state;
  }
}
function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: "inc" })}>Increment</button>
      <button onClick={() => dispatch({ type: "dec" })}>Decrement</button>
    </div>
  );
}
```

18. REACT ROUTER (v6)

Install

```
npm install react-router-dom
```

Example:

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
function Home() { return <h2>Home</h2>; }
function About() { return <h2>About</h2>; }

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={Home} />
        <Route path="/about" element={About} />
      </Routes>
    </BrowserRouter>
  );
}
```

19. CONTEXT API vs STATE

	STATE	CONTEXT API
Used for	Local data	Global/Shared data
Prop Drilling	Hard	Easy
Re-renders	Only that component	All consumers re-render
Complexity	Low	Slightly High

20. HIGHER ORDER COMPONENTS (HOC)

- A function that takes a component and returns a new component.

Example:

```
function withUser(WrappedComponent) {
  return function (props) {
    const user = { name: "Anchal", role: "dev" };
    return <WrappedComponent {...props} user={user} />;
  };
}
```

21. RENDER PROPS

- Share code between components using a prop which is a function.

Example:

```
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });
  return (
    <div>
      <div>
        onMouseMove={(e) =>
          setPosition({ x: e.clientX, y: e.clientY })
        }
      </div>
      {render(position)}
    </div>
  );
}
```

22. PERFORMANCE OPTIMIZATION

- React.memo() – Prevent re-render of components
- useMemo() – Memoize expensive values
- useCallback() – Memoize functions
- Lazy Loading – Load components on demand

Example (React.memo):

```
const MyComponent = React.memo(function MyComponent({ name }) {
  console.log("Rendered");
  return <h3>{name}</h3>;
});
```

23. BEST PRACTICES

- Keep components small and reusable
- Use meaningful names
- Use functional components + hooks
- Keep state as local as possible
- Avoid prop drilling (use Context)
- Handle errors & edge cases
- Keep code clean & well structured
- Use ESLint & Prettier

25. POPULAR LIBRARIES

- Axios – HTTP requests
- React Router – Routing
- Redux Toolkit – State management
- Tailwind CSS / MUI / Chakra UI – Styling
- React Hook Form – Forms
- Framer Motion – Animations

26. PROJECT IDEAS

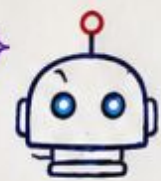
- To-Do App
- Weather App
- Notes App
- E-commerce UI
- Blog Website
- Movie App



REMEMBER

Practice daily, build projects, read documentation and keep learning. Consistency is the key to mastery! ❤️

API + List Rendering in Frontend

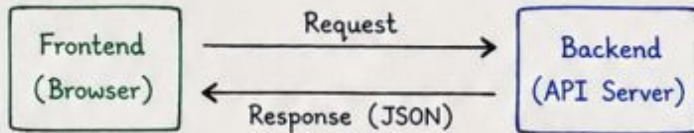


Great UI
Needs
Great Data!

Fetch Data • Render Lists • Build Dynamic UI

1) API (Application Programming Interface)

- API allows frontend to communicate with backend.
- We can Fetch (GET), Send (POST), Update (PUT), Delete (DELETE) data.
- Data is usually in JSON format.



Common HTTP Methods

GET	→ Read / Fetch Data
POST	→ Create Data
PUT	→ Update Data
DELETE	→ Delete Data

Example API Call (Using fetch)

```
fetch("https://jsonplaceholder.typicode.com/posts")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Example API Call (Using async/await)

```
async function getPosts(){
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/posts");
    const data = await res.json();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}

getPosts();
```

2) List Rendering



Codewithbharat

FOLLOW ON INSTAGRAM

- List rendering means showing multiple items dynamically using JavaScript (map function).
- We loop through data and return JSX for each item.

Example Data (JSON)

```
[
  { "id": 1, "title": "Learn React", "userId": 1 },
  { "id": 2, "title": "Learn API", "userId": 1 },
  { "id": 3, "title": "Build Projects", "userId": 1 }
]
```

React List Rendering Example

```
import { useEffect, useState } from 'react';
function PostList() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(res => res.json())
      .then(data => setPosts(data))
      .catch(err => console.error(err));
  }, []);

  return (
    <div>
      <h2>Posts</h2>
      <ul>
        {posts.map(post => (
          <li key={post.id}>
            <h3>{post.title}</h3>
            <p>User ID: {post.userId}</p>
          </li>
        ))}
      </ul>
    </div>
  );
}
export default PostList;
```

Key Points

- Always use unique 'key' in lists.
- Handle loading & error states.
- Keep components clean & reusable.

Data changes,
UI updates!
That's the
Power
😊

My Learning Approach



Tools I Use



VS Code



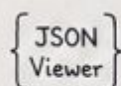
Postman



Chrome DevTools



React.js



{ JSON
Viewer }



Tailwind
CSS



Git &
GitHub

Keep Learning
Keep Building
Keep Growing



Frontend Development + AI



AI is not
Replacing
Developers.
It's empowering
Developers.

Build Better UI • Write Cleaner Code • Ship Faster with AI

1) HTML5 (Structure)

- HTML is the backbone of web pages.
- Use Semantic HTML for better SEO & accessibility.
- Important Tags:

```
<header> <nav> <main>
<section> <article> <aside>
<footer>
```

2) CSS3 (Styling)

- Makes website beautiful & responsive.
- Key Concepts:
 - Selectors
 - Box Model
 - Flexbox
 - Grid
 - Media Queries
 - Transitions & Animations

* Mobile First Approach Always!

3) JavaScript (Logic)

- Makes website interactive.
- ES6+ Features
 - let, const
 - Arrow Functions
 - Template Literals
 - Destructuring
 - Spread & Rest
 - Modules
 - Promises, async/await

4) React.js (Library)

- Component Based UI Library
- Build reusable UI components.
- Key Points:
 - JSX
 - Components
 - Props
 - State
 - useEffect()
 - React Router
 - Hooks (useState, useEffect)

5) Next.js (Framework)

- React framework for production.
- Features:
 - Server Side Rendering (SSR)
 - Static Site Generation (SSG)
 - API Routes
 - File based Routing
 - Image Optimization
 - Built-in SEO

6) Tailwind CSS (Utility First)

- Rapid UI development.
- Utility classes for styling.
- Example:

```
<button class="bg-blue-500
text-white px-4 py-2 rounded">
  Click Me
</button>
```

Fast • Consistent • Customizable

7) REST APIs (Integration)

- Communicate with backend.
- Use Fetch / Axios.
- Common Methods:

```
GET    → Read
POST   → Create
PUT    → Update
DELETE → Delete
```

8) Git & GitHub

- Version Control System
- Important Commands:

```
git init
git add .
git commit -m "message"
git push origin main
git pull origin main
```

9) AI in Frontend Development

- AI tools make developers 10x productive.
- How I use AI:

- ☒ Code Generation
- ☒ Debugging
- ☒ Explaining Concepts
- ☒ Refactoring
- ☒ UI/UX Ideas
- ☒ Writing Documentation
- ☒ Test Case Generation

AI is my
Coding
Assistant
😊

10) My Learning Approach

Learn the
Concept

Write it
by Hand

Build a
Real Project

Use AI to
Improve

Share what
I Learn

Tools I Use

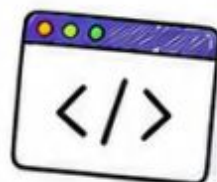
VS Code • GitHub • Chrome DevTools • Postman/Thunder Client
Tailwind CSS • Figma • ChatGPT • Vercel

Keep Learning
Keep Building
Keep Sharing





JAVASCRIPT ES6+ CHEAT SHEET



★ Most Used Features Every Developer Should Know ★

1 let & const

Block-scoped variables.

```
let name = "Saqib"; // can reassign
const age = 25;      // cannot reassign
```

3 Template Literals

Dynamic strings with backticks.

```
const name = "Saqib";
const msg = `Hello, ${name}!`;
const multiLine = `This is
                    multi-line string`;
```

5 Spread Operator (...)

Copy & merge arrays/objects.

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const mergedArr = [...arr1, ...arr2];

const obj1 = { a: 1 };
const obj2 = { b: 2 };
const mergedObj = { ...obj1, ...obj2 };
```

7 Default Parameters

Set default values for function parameters.

```
function greet(name = "Guest") {
  return `Hello, ${name}!`;
}

console.log(greet()); // Hello, Guest
console.log(greet("Saqib")); // Hello, Saqib
```

9 Promises

Handle asynchronous operations.

```
const promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) resolve("✅ Success");
  else reject("❌ Failed");
});

promise
  .then(res => console.log(res))
  .catch(err => console.log(err));
```

★ PRO TIPS

- ✓ Use const by default.
- ✓ Prefer arrow functions for callbacks.
- ✓ Use destructuring to write clean code.
- ✓ Use spread & rest operators wisely.
- ✓ Use async/await instead of nested callbacks.
- ✓ Organize code using modules.



2 Arrow Functions (=>)

Shorter function syntax.

```
const add = (a, b) => a + b;
const greet = name => `Hello, ${name}!`;
const sayHi = () => console.log("Hi!");
```

4 Destructuring

Extract values from objects & arrays.

```
const user = { name: "Saqib", age: 25 };
const { name, age } = user; // Object

const numbers = [1, 2, 3];
const [a, b, c] = numbers; // Array
```

6 Rest Operator (...)

Collect multiple arguments into an array.

```
function sum(...nums) {
  return nums.reduce((acc, curr) => acc + curr, 0);
}

console.log(sum(1, 2, 3, 4)); // 10
```

8 Modules

Use import & export to share code.

```
// math.js
export const add =
  (a, b) => a + b;
export const PI =
  3.14;
```

```
// main.js
import { add, PI }
  from "./math.js";
console.log(add(2, 3));
console.log(PI);
```

10 Async / Await

Write async code like synchronous code.

```
async function getData() {
  try {
    const res = await fetch("https://api.example.com");
    const data = await res.json();
    console.log(data);
  } catch (error) {
    console.log("Error:", error);
  }
}

getData();
```

async
await



USED IN

- React.js
- Next.js
- Mode.js
- JavaScript Projects
- React Native
- Modern Web Development



SUBSCRIBE
FOR MORE
CODING SHORTS!

FIVE BILLION CODE

LEARN • CODE • GROW • SUCCEED ❤️

FOLLOW →

@fivebillioncode



HTML ATTRIBUTES

Attributes provide additional information about HTML elements. They are always written in the opening tag. ☆

KEY POINTS

- Attributes are written in the opening tag.
- Format: `name="value"`
- Multiple attributes can be used in one tag.
- Attributes usually come in name/value pairs.

1. Anatomy of an Attribute

<tag attribute-name="value">Content</tag>		
↓	↓	↓
Element	Attribute	Value

Example

```

```

2. Global Attributes (can be used on most elements)

Attribute	Description	Example
id	Unique identifier for an element	<div id="main">
class	Specifies one or more class names for an element	<p class="note important">
style	Adds inline CSS styles	<h1 style="color: blue;">
title	Extra information on hover	
lang	Defines the language of content	<html lang="en">
data-*	Custom data attributes	<div data-id="123">

Example in Use

```
<a href="https://example.com" title="Visit Example" target="_blank" class="link" id="top-link">Example Website</a>
```

Note

id should be unique on a page.
class can be reused on multiple elements.

3. Common Attributes by Element

Element	Common Attributes	Example
<a>	href, target, title, download	
	src, alt, width, height	
<input>	type, name, value, placeholder, required	<input type="text" name="user">
<form>	action, method, enctype	<form action="submit.php" method="post">
<table>	border, cellpadding, cellspacing	<table border="1" cellspacing="0">

4. Boolean Attributes

Boolean attributes do not have a value. Presence of the attribute = true.

Examples:

- <input type="checkbox" checked>
- <input type="text" disabled>
- <option selected>
- <input type="text" required>

5. Best Practices

- ★ Use meaningful id and class names.
- ★ Use lowercase for attribute names.
- ★ Always use quotes around attribute values.
- ★ Do not use duplicate id on the same page.
- ★ Validate your HTML for better results.

REMEMBER

Clean and meaningful attributes make your code easier to read, maintain and debug.

Quick Tips

- ★ Use alt in images for accessibility and SEO.
- ★ Use title for helpful tooltips.
- ★ Use required in forms to improve user experience.

HTML5 FEATURES

HTML5 introduces new elements, attributes and APIs for modern web development.

1. New HTML5 Elements

- `<header>`
- `<nav>`
- `<main>`
- `<section>`
- `<article>`
- `<aside>`
- `<footer>`
- `<figure>`
- `<figcaption>`
- `<time>`

Semantic Elements

KEY POINTS

- HTML5 improves structure and semantics.
- Supports multimedia without plugins.
- New form input types.
- Better performance and accessibility.

2. Multimedia Elements

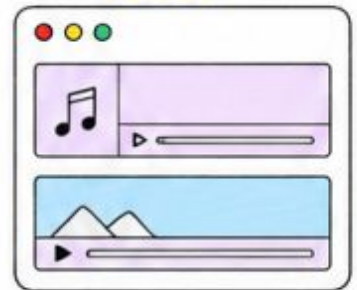
(a) Audio

```
<audio controls>
  <source src="song.mp3" type="audio/mpeg">
  Your browser does not support audio.
</audio>
```

(b) Video

```
<video width="320" controls>
  <source src="movie.mp4" type="video/mp4">
  Your browser does not support video.
</video>
```

Output



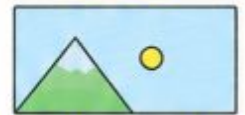
3. New Form Input Types

Type	Example	Type	Example
date	dd-mm-yyyy	email	user@example.com
time	--:--:--	url	https://example.com
datetime-local	dd-mm-yyyy, --:--	search	Search...
month	mm-yyyy	range	<input max="100" type="range" value="0"/>
week	Week --, yyyy	color	<input type="color" value="#0000FF"/>

4. Canvas Element

```
<canvas id="myCanvas"
width="200" height="100">
  Your browser does not
  support canvas.
</canvas>
```

Output



Used to draw graphics using JavaScript.

5. Other HTML5 Features

- `<svg>` → Vector graphics
- `<video>` → Video playback
- `<audio>` → Audio playback
- `<canvas>` → Drawing graphics
- `<progress>` → Progress bar
- `<meter>` → Measurement within a range
- `<details>` → Collapsible content
- `<summary>` → Visible heading for details

Example: Progress & Meter

```
<label>Progress:</label>
<progress value="70" max="100"></progress>
<br><br>
<label>Disk Usage:</label>
<meter value="0.6">60%/meter>
```

Output

Progress: 70%

Disk Usage: 60%

Remember

- Use new semantic tags for better structure.
- Provide fallbacks for older browsers.
- Test for cross-browser compatibility.



SEMANTIC HTML

Semantic HTML uses tags that describe the meaning and purpose of the content.

1. Common Semantic Elements

Tag	Purpose
<code><header></code>	Represents introductory content or a set of navigational links
<code><nav></code>	Defines navigation links
<code><main></code>	Represents the main content of the document
<code><section></code>	Defines a section in a document
<code><article></code>	Represents independent, self-contained content
<code><aside></code>	Defines content aside from the main content
<code><footer></code>	Represents footer content of a section or page

2. Example Structure

```
<header>
  <h1> My Website </h1>
  <nav>
    <a href="#"> Home </a> |
    <a href="#"> About </a> |
    <a href="#"> Contact </a>
  </nav>
</header>
<main>
  <section>
    <h2> About Us </h2>
    <p> We provide quality content. </p>
  </section>
  <article>
    <h2> Blog Post </h2>
    <p> This is an article. </p>
  </article>
  <aside>
    <h3> Related Links </h3>
    <ul> <li> Link 1 </li> <li> Link 2 </li> </ul>
  </aside>
</main>
<footer>
  <p> © 2024 My Website </p>
</footer>
```

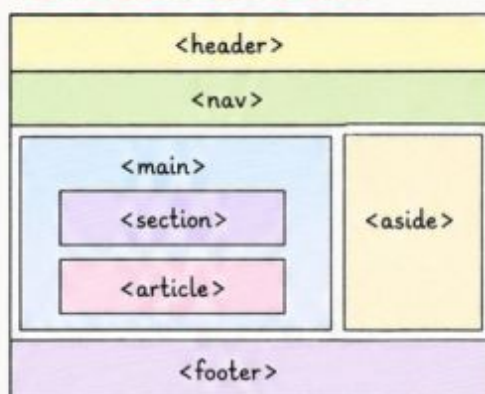
KEY POINTS

- Improves SEO
- Better accessibility
- Easier to read and maintain
- Works as a guide for browsers and developers

3. Semantic vs Non-Semantic

Non-Semantic	Semantic
<code><div></code> , <code></code>	<code><header></code> , <code><nav></code> , <code><section></code> etc.
No meaning about content	Clearly describes the meaning
Less helpful for SEO and accessibility	Better SEO and accessibility
Hard to understand large code	Easy to understand and maintain

4. Page Layout Example



NOTE

Use semantic tags whenever possible instead of `<div>` for better structure and meaning.



Quick Tips

- * Always use `<main>` only once per page.
- * Keep `<header>` and `<footer>` consistent.
- * Use `<section>` for thematically grouped content.
- * Use `<article>` for independent content like blog posts.
- * Semantic HTML makes your code future-ready.





HTML FORMS

HTML forms are used to collect user input.

1. Form Structure

```
<form action="submit.php" method="post">  
  <!-- form elements -->  
</form>
```

→ action → where to send data
→ method → get / post

2. Text Inputs

- Text : Single line text input
- Password : Hides the characters
- Email : For email input
- Number : For numeric values

```
<input type="text" placeholder="Your Name">  
<input type="password" placeholder="Password">  
<input type="email" placeholder="Email">  
<input type="number" placeholder="Age">
```

3. Radio Buttons & Checkboxes

Radio Buttons (Single Choice)

```
<input type="radio" name="gender" value="male"> Male  
<input type="radio" name="gender" value="female"> Female
```

Checkboxes (Multiple Choice)

```
<input type="checkbox" value="html"> HTML  
<input type="checkbox" value="css"> CSS  
<input type="checkbox" value="js"> JavaScript
```

Note

Radio buttons with same name allow only one selection.

4. Select Dropdown

```
<select name="course">  
  <option value="html">HTML</option>  
  <option value="css">CSS</option>  
  <option value="js">JavaScript</option>  
</select>
```

5. Textarea

```
<textarea rows="4" cols="30"  
  placeholder="Your Message">  
</textarea>
```

Your Message

6. File Input

```
<input type="file">
```

Choose File No file chosen

7. Submit & Reset Buttons

```
<input type="submit" value="Submit">  
<input type="reset" value="Reset">
```

Submit

Reset

Quick Tips

- Use placeholder to give hint inside input fields.
- Use labels with for attribute for accessibility.
- Always validate form data on the server side.

KEY POINTS

- Forms are used to collect user input.
- form tag is the container for all form elements.
- Use method="post" for security.
- Add labels for better user experience.

INPUT TYPES

- text → Single line text
- password → Hidden text
- email → Email input
- number → Numeric input
- radio → Single choice
- checkbox → Multiple choice
- date → Date picker
- file → Upload file
- submit → Submit form
- reset → Reset form
- button → Clickable button

Example Output

HTML TABLES

HTML tables are used to display data in rows and columns.

1. Syntax

```
<table>
  <tr>
    <th> Header </th>
    <th> Header </th>
  </tr>
  <tr>
    <td> Data </td>
    <td> Data </td>
  </tr>
</table>
```

Table Row (Headings)

Table Row (Data)

2. Example

```
<table border="1">
  <tr>
    <th> Name </th>
    <th> Age </th>
    <th> City </th>
  </tr>
  <tr>
    <td> Rahul </td>
    <td> 20 </td>
    <td> Delhi </td>
  </tr>
  <tr>
    <td> Neha </td>
    <td> 21 </td>
    <td> Mumbai </td>
  </tr>
</table>
```

KEY POINTS

- Table data is arranged in rows (tr) and columns (td).
- th is used for table headings.
- Tables are used for tabular data only.

3. Table Tags

- <table> → Defines a table
- <tr> → Defines a table row
- <th> → Defines a header cell
- <td> → Defines a data cell
- <caption> → Defines table caption
- <thead> → Groups header content
- <tbody> → Groups body content
- <tfoot> → Groups footer content

4. Example Output

Name	Age	City
Rahul	20	Delhi
Neha	21	Mumbai

border="1" adds border around table.

5. Common Attributes

- border** → Adds border to table
- cellpadding** → Space inside the cell
- cellspacing** → Space between cells
- width** → Sets table width
- align** → Aligns table (left/center/right)
- bgcolor** → Sets background color

6. Colspan & Rowspan

Colspan (Merge Columns)

Merged Cell		
A	B	C

```
<th colspan="3">
  Merged Cell </th>
```

Rowspan (Merge Rows)

Merged Cell	A	B
	C	D

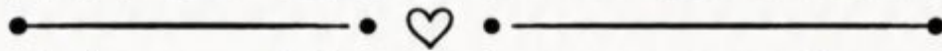
```
<td rowspan="2">
  Merged Cell </td>
```

Quick Tips

- * Use tables only for tabular data, not for layout.
- * Use <th> for headings (better for accessibility).
- * Use <caption> to add a title to the table.
- * Keep tables simple and easy to read.



HTML LINKS & IMAGES



Links are used to connect one page to another.

Images are used to add visuals to web pages. ★

* 1. Links (Anchor Tag <a>)

Syntax

```
<a href = "URL">Link Text</a>
```

Example

```
<a href = "https://example.com">
  Visit Example Website
</a>
```

Important Attributes

- href = "url" → Specifies the destination URL
- target = "_blank" → Opens link in a new tab
- title = "text" → Shows extra information on hover
- download → Downloads the linked file
- rel = "noopener" → Security for target = "_blank"

Example with Attributes

```
<a href = "notes.pdf"
  target = "_blank"
  title = "Download Notes"
  download>
  Download Notes
</a>
```



* 2. Images (Tag)

Syntax

```
<img src = "image.jpg" alt = "Description">
```

Example

```
<img src = "nature.jpg" alt = "Beautiful Nature"
  width = "400" height = "250">
```

Important Attributes

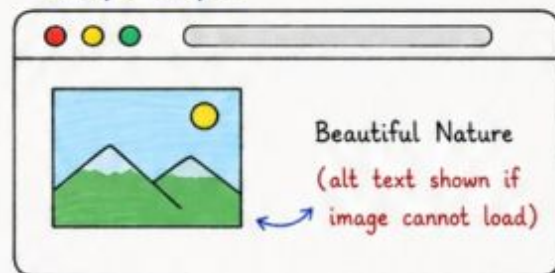
- src → Image source (URL or path)
- alt → Alternative text for image
- width → Sets width of image
- height → Sets height of image
- title → Extra info on hover
- loading → lazy | eager (for performance)

Image Types Supported

- JPG / JPEG → Photos
- PNG → Images with transparency
- GIF → Animations
- SVG → Vector Graphics



Example Output



Quick Tips

- * Always use meaningful link text.
- * Always add alt text for images (good for SEO & accessibility).
- * Use target = "_blank" carefully.
- * Optimize images for faster loading.





HTML LISTS

HTML lists are used to group related items in a specific order or structure.

* 1. Ordered List (ol)

Items are displayed in a specific order (numbers).

```
<ol>
  <li>Apple</li>
  <li>Mango</li>
  <li>Banana</li>
</ol>
```



Output

1. Apple
2. Mango
3. Banana

KEY POINTS

- `<ul>` for unordered list
- `<ol>` for ordered list
- `<dl>` for description list
- `<li>` defines list items
- Lists can be nested.

* 2. Unordered List (ul)

Items are displayed with bullets.

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```



Output

- HTML
- CSS
- JavaScript

Bullet Types

- disc (•) → Default
circle (o) → `<ul type="circle">`
square (■) → `<ul type="square">`

* 3. Description List (dl)

Used to define terms and their descriptions.

```
<dl>
  <dt>HTML</dt>
  <dd>Hyper Text Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```



Output

HTML
Hyper Text Markup Language
CSS
Cascading Style Sheets

Tags Used

- `<ol>` : Ordered List
`<ul>` : Unordered List
`<li>` : List Item
`<dl>` : Description List
`<dt>` : Term
`<dd>` : Description



Quick Notes

- Use `<ol>` when order is important.
- Use `<ul>` when order is not important.
- Use `<dl>` for terms and explanations.
- Lists can be nested inside each other. 😊

Nested List Example

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Mango</li>
    </ul>
  </li>
  <li>Vegetables</li>
</ul>
```



Output

- Fruits
 - o Apple
 - o Mango
- Vegetables





TEXT FORMATTING

HTML provides several tags to format text and make it more meaningful.

Formatting Tags

Tag	Example	Output	Use
<code> </code>	<code>Bold</code>	Bold	Bold text
<code> </code>	<code>Important</code>	Important	Important text
<code><i> </i></code>	<code><i>Italic</i></code>	<i>Italic</i>	Italic text
<code> </code>	<code>Emphasis</code>	<i>Emphasis</i>	Emphasized text
<code><u> </u></code>	<code><u>Underline</u></code>	<u>Underline</u>	Underlined text
<code><mark> </mark></code>	<code><mark>Highlight</mark></code>	Highlight	Highlighted text
<code><small> </small></code>	<code><small>Small Text</small></code>	Small Text	Smaller text
<code> </code>	<code>Deleted</code>	Deleted	Deleted text
<code><ins> </ins></code>	<code><ins>Inserted</ins></code>	<u>Inserted</u>	Inserted text
<code><sup> </sup></code>	<code>x<sup>2</sup></code>	x ²	Superscript
<code><sub> </sub></code>	<code>H<sub>2</sub>O</code>	H ₂ O	Subscript

Combination Example

`<p> This is <b>bold</b>, <i>italic</i> and <mark>highlighted</mark> text. </p>`



This is **bold**, *italic* and **highlighted** text.

Example Output

Bold Text

Important Text

Italic Text

Emphasized Text

Underline Text

Highlight Text

Small Text

~~Deleted Text~~

Inserted Text

x² (Superscript)

H₂O (Subscript)

Note

- Use `<strong>` instead of `<b>` for importance.
- Use `<em>` instead of `<i>` for emphasis.
- These tags help in better SEO and accessibility. 😊



Quick Tips

- * Use formatting tags to improve readability.
- * Do not use tags for styling (use CSS for design).
- * Keep your code semantic and meaningful.





HTML BASIC TAGS

HTML tags are the building blocks used to create and structure web pages. ☆

* Heading Tags

```
<h1>Heading</h1>
<h2>Sub Heading</h2>
...
<h6>Smallest Heading</h6>
```

Used to define headings.
h1 is largest and h6 is smallest.

* Paragraph Tag

```
<p>This is a paragraph.</p>
```

Used to define paragraph.

* Line Break Tag

```
<br>
```

Used to break a line.

* Horizontal Line

```
<hr>
```

Used to create a horizontal line.

* Comment Tag

```
<!-- This is a comment -->
```

Used to add comments.

* Bold Tag

```
<b>Bold Text</b>
```

Used to bold the text.

* Italic Tag

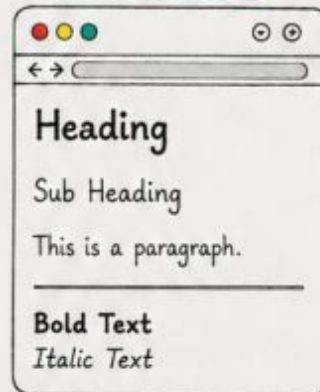
```
<i>Italic Text</i>
```

Used to italic the text.

KEY POINTS

- Tags are written inside angle brackets < >
- Most tags have an opening and closing tag.
- HTML is not case sensitive, but lowercase is recommended.

EXAMPLE OUTPUT



Quick Notes

- Use meaningful headings (h1 → h6).
- Use
 for line break, not for spacing.
- Use comments to explain your code. 😊

EMPTY TAGS (No Closing Tag)

 <hr> <input>
<meta> <link>

These tags do not have closing tags. ☆



HTML

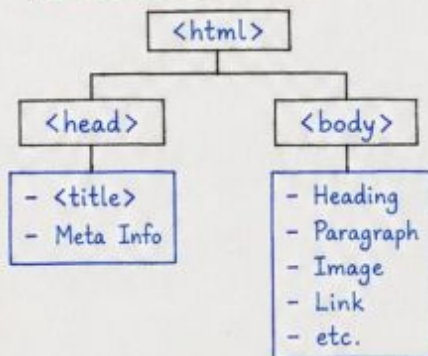
Basic to Advanced Notes

Complete handwritten notes for beginners to advanced learners.

What is HTML?

HTML (HyperText Markup Language) is the standard markup language used to create and structure web pages. 😊

HTML Page Structure



Example (Simple HTML Page)

```
<!DOCTYPE html>
<html>
  <head>
    <title> My First Page</title>
  </head>
  <body>
    <h1> Hello World! </h1>
    <p> Welcome to HTML.</p>
  </body>
</html>
```

Key Features

- * Easy to learn
- * Platform independent
- * Used to create web pages
- * Works with CSS & JavaScript
- * Supported by all browsers

★ Topics Covered

- Meaning of HTML
- HTML Document Structure
- Basic Tags
- Headings & Paragraphs
- Links & Images
- Lists
- Tables
- Forms
- Semantic HTML
- HTML5 Tags



Code.
Learn.
Build.
Repeat.





React.JS Notes

Page - 15

1. React Lazy() and Suspense

- React.lazy() is used for code splitting (lazy loading).
- Suspense shows fallback UI while component is loading.

Example :

```
const About = React.lazy(() => import("../About"));
function App() {
  return (
    <Suspense fallback={<h2>Loading...</h2>}>
      <About />
    </Suspense>
  );
}
```

Improves performance by loading components on demand.



2. useRef Hook

- useRef is used to access DOM elements or store mutable values.
- It does not cause re-renders when value changes.

Example :

```
import { useRef } from "react";
function FocusInput() {
  const inputRef = useRef(null);
  const handleClick = () => {
    inputRef.current.focus();
  };
  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}
```

useRef gives us a reference without re-rendering!



3. useReducer Hook

- An alternative to useState for complex state logic.
- Follows reducer pattern (state, action) => new state.

Example :

```
const initialState = { count: 0 };
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    default:
      return state;
  }
}
const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: "increment" });
```

Great for complex state with multiple actions!



4. React.memo()

- React.memo() memoizes functional components.
- Prevents re-render when props are same.
- Improves performance.

Example :

```
const Button = React.memo(function Button({ onClick, label }) {
  console.log("Button rendered");
  return <button onClick={onClick}>{label}</button>;
});
```

Memoization = Better Performance.



5. useEffect() Cleanup

- Cleanup function runs before next effect or when component unmounts.
- Use for subscriptions, timers, listeners.

Example :

```
import { useEffect } from "react";
function Timer() {
  useEffect(() => {
    const id = setInterval(() => {
      console.log("Tick");
    }, 1000);
    return () => clearInterval(id); // cleanup
  }, []);
  return <p>Timer is running...</p>;
}
```

Always clean up to prevent memory leaks.



6. useImperativeHandle()

- Customize the instance value that is exposed to parent when using forwardRef.
- Useful for exposing specific methods.

Example :

```
import { forwardRef, useImperativeHandle, useRef } from "react";
const Input = forwardRef((props, ref) => {
  const inputRef = useRef();
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => (inputRef.current.value = ""),
  }));
  return <input ref={inputRef} />;
});
```

Exposes only what parent needs!



7. React Portals

- Portals render children into a DOM node outside the parent hierarchy.
- Useful for modals, tooltips, popovers.

Example :

```
import { createPortal } from "react-dom";
function Modal({ children }) {
  return createPortal(
    <div className="modal">{children}</div>,
    document.getElementById("modal-root")
  );
}
```

Portals break out of parent containers.



8. Default Props in Function Components

- Provide default values for props.
- Use ES6 default parameters or defaultProps.

Example (ES6) :

```
function Greeting({ name = "Guest" }) {
  return <h2>Hello, {name}</h2>;
}
```

Example (defaultProps) :

```
function Greeting(props) {
  return <h2>Hello, {props.name}</h2>;
}
Greeting.defaultProps = { name: "Guest" };
```

Helps in handling missing props.



9. React Performance Best Practices

- | | |
|---|------------------------------------|
| ✓ Use React.memo() to memoize components. | ✓ Use key prop in lists. |
| ✓ Use useCallback() for functions. | ✓ Keep state as local as possible. |
| ✓ Use useMemo() for expensive calculations. | ✓ Lazy load routes and components. |
| ✓ Avoid inline functions in JSX. | ✓ Optimize re-renders. |



Small optimizations lead to Big impact!



10. React Interview Quick Q&A

- | | |
|--|-----------------------------|
| 1. What is React? | 8. What is React Router? |
| 2. What is JSX? | 9. What is Context API? |
| 3. What are Props and State? | 10. What is React.memo()? |
| 4. What is Virtual DOM? | 11. What is useCallback()? |
| 5. What are Hooks? | 12. What is Error Boundary? |
| 6. What is useEffect()? | 13. What is useMemo()? |
| 7. Difference between useState and useRef? | 14. What are Keys in React? |
| | 15. What is React Portal? |



Keep Practicing Keep Growing!



Summary

- ★ React makes building UI simple and efficient.
- ★ Components are reusable building blocks.
- ★ State & Props make UI dynamic.
- ★ Hooks let us use state and features in functions.
- ★ Performance tuning makes apps fast.
- ★ Practice consistently and build real projects!



Practice + Learn = Master React



Remember

- ✓ Learn a little every day.
- ✓ Build small projects.
- ✓ Read docs & explore more.
- ✓ Debug & refactor your code.
- ✓ Never stop learning!





React.JS Notes

Page - 14

1. React Fragments (<> </>)

- Fragments let us group multiple elements without adding extra nodes to the DOM.
- Use <> </> or <React.Fragment> </React.Fragment>.

Example :

```
import React from "react";
function Demo() {
  return (
    <>
      <h1>Hello</h1>
      <p>This is a fragment</p>
    </>
  );
}
export default Demo;
```

Keeps
DOM clean
and code
simple



2. Conditional Rendering

- Render different UI based on conditions.
- Use if, ternary (? :), &&, or switch.

Example :

```
function Greeting({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (
        <h2>Welcome Back!</h2>
      ) : (
        <h2>Please Login</h2>
      )}
    </div>
  );
}
```

Show only
what is
needed



3. State vs Props

State	Props
• Managed within a component. • Mutable (can change). • Triggers re-render. • Used for component data. • Example: useState()	• Passed from parent to child. • Immutable (read-only). • Does not change in child. • Used for passing data. • Example: <Child name="Saquib"/>

Props move data down, State handles changes.



4. Lists and Keys

- Use .map() to render lists.
- Keys help React identify items and optimize re-renders.

Example :

```
const items = ["Apple", "Banana", "Mango"];
function FruitList() {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item}</li>
      ))}
    </ul>
  );
}
```

Always
use unique
keys



5. Controlled vs Uncontrolled Components

Controlled	Uncontrolled
• Form data handled by React state. • We use value and onChange. • More control & validation. • Example : <pre><input type="text" value={name} onChange={(e) => setName(e.target.value)} /></pre>	• Form data handled by DOM. • We use ref to access value. • Less re-render, easier for quick forms. • Example : <pre><input type="text" ref={inputRef} /></pre>

Controlled
for complex
forms,
Uncontrolled
for simple
ones.



6. Custom Hook

- Extract component logic into reusable functions.
- Name must start with "use".

Example : useTitle hook

```
import { useEffect } from "react";
function useTitle(title) {
  useEffect(() => {
    document.title = title;
  }, [title]);
}
export default useTitle;
```

Custom Hooks
make code
clean,
reusable &
easy to
maintain



7. React.memo vs useCallback vs useMemo

Hook	Purpose	Use When	Returns
React.memo()	Memoize component	Component re-renders with same props	Memoized Component
useCallback()	Memoize function	Passing callbacks to child components	Memoized Function
useMemo()	Memoize value	Expensive calculations or heavy operations	Memoized Value

★ Use these to optimize performance and avoid unnecessary re-renders.



9. React Error Boundaries

- Catch JavaScript errors in child components.
- Prevent whole app from crashing.

Example :

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError() {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
```

Better
user
experience
with
graceful
error
handling



8. React Context API

- Share data across components without prop drilling.
- Great for theme, auth, language, etc.

Example :

```
const ThemeContext = React.createContext();
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = React.useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

Context
solves
prop drilling
like a
pro!



10. React Interview Quick Q&A

1. What is React?
2. What is JSX?
3. What are Props and State?
4. What is Virtual DOM?
5. What are Hooks?
6. What is useEffect()? Difference between useState and useRef?
8. What is React Router?
9. What is Context API?
10. What is React.memo()? Practice makes Perfect!
11. What is useCallback()?
12. What is Error Boundary?
13. What is useMemo()?
14. What are Keys in React?

Summary

- ✓ React is a component based UI library.
- ✓ Components are reusable building blocks.
- ✓ State makes components dynamic.
- ✓ Props make components configurable.
- ✓ Clean code, reusable logic & performance matter.
- ✓ Practice consistently & build real projects!

Code with Passion
Learn with Consistency
Build with Confidence



Remember

- ✓ Learn a little every day.
- ✓ Build small projects.
- ✓ Read docs & explore more.
- ✓ Debug & refactor your code.
- ✓ Never stop learning!



Keep Practicing • Keep Building • Keep Growing!





React.JS Notes

Page - 13

1. React Keys

- Keys help React identify which items have changed, been added, or removed.
- Keys should be unique among siblings.

Example :

```
const fruits = ["Apple", "Banana", "Mango"];
return (
  <ul>
    {fruits.map((fruit, index) => (
      <li key={index}>{fruit}</li>
    ))}
  </ul>
);
```

Keys
give React
a stable
identity.



3. React Forward Ref

- Forward ref allows a parent component to access DOM elements of a child component.
- Useful for focus, animations, measurements, etc.

Example :

```
const Input = React.forwardRef((props, ref) => {
  return <input ref={ref} {...props} />;
});
export default Input;
```

ForwardRef
bridges the
gap between
parent & child.



2. React.memo vs useMemo vs useCallback

Feature	React.memo	useMemo	useCallback
Type	Higher Order Component	Hook	Hook
Purpose	Memoize functional components	Memoize computed value	Memoize function reference
When to use	When component re-renders often with same props	When expensive calculation is repeated	When passing functions to child components
Returns	Memoized Component	Memoized Value	Memoized Function

★ Use these tools to optimize performance and avoid unnecessary re-renders.



4. React Portals Use Case

- Portals render children into a DOM node outside the parent hierarchy.
- Commonly used for modals, tooltips, dropdowns.

Example (Modal Portal) :

```
const modalRoot = document.getElementById("modal-root");
function Modal({ children }) {
  return ReactDOM.createPortal(children, modalRoot);
}
```

Perfect
for modals
& overlays.



5. Error Boundaries

- Error Boundaries catch JavaScript errors in child components, log them, and display fallback UI.
- They do NOT catch errors in event handlers, async code, or SSR.

Example :

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong.</h2>;
    }
    return this.props.children;
  }
}
```

Better
User
Experience
with graceful
error
handling.



6. React Context + useReducer (Advanced State)

- Context is used to avoid prop drilling.
- useReducer manages complex state logic.

Example :

```
// 1. Create Context
const AppContext = React.createContext();
// 2. Reducer
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    default:
      return state;
  }
}
// 3. Provider
function AppProvider({ children }) {
  const [state, dispatch] = React.useReducer(reducer, { count: 0 });
  return (
    <AppContext.Provider value={{ state, dispatch }}>
      {children}
    </AppContext.Provider>
  );
}
```

Powerful
combination
for scalable
state
management.



7. React Testing Library (RTL)

- RTL encourages testing components the way users interact with them.
- Focus is on behavior, not implementation.

Common Queries :

Query	Use
getByRole()	Find element by role (button, heading, etc.)
getByText()	Find element by text content
getByLabelText()	Find form elements by label
getByPlaceholderText()	Find by placeholder text
getByTestId()	Find element by data-testid

Test
like a
user,
not like a
developer.



8. React Router v6 - Nested Routes

- Nested routes help in building complex UI.
- Use <Outlet /> to render child routes.

Example :

```
<Routes>
  <Route path="/dashboard" element={ <Dashboard /> }>
    <Route index element={ <Overview /> } />
    <Route path="analytics" element={ <Analytics /> } />
    <Route path="settings" element={ <Settings /> } />
  </Route>
</Routes>
```

Nested
Routes =
Clean &
Organized
Navigation.



9. React Performance Checklist

- ✓ Use React.memo() for component memoization
- ✓ Use useCallback() for stable functions
- ✓ Use useMemo() for expensive calculations
- ✓ Lazy load components using React.lazy()
- ✓ Avoid inline functions in JSX.
- ✓ Keep component state local as much as possible
- ✓ Use keys properly in lists



Small
improvements
make a big
impact!



10. Common Interview Questions (Quick List)

1. What is React?
2. What is JSX?
3. What are Props and State?
4. What is Virtual DOM?
5. What are Hooks?
6. What is useEffect()? Difference between useState and useRef?
7. What is React Router?
8. What is Context API?
9. What is React.memo()?
10. What is useCallback()?
11. What is an Error Boundary?
12. When to use useMemo()?
13. What are Keys in React?



Summary

- ★ React is a powerful JavaScript library.
- ★ Components are reusable building blocks.
- ★ Hooks make state & side effects easy.
- ★ Performance & clean code are important.
- ★ Practice consistently and build real projects!



Practice
+
Build
=
Master React



Remember

- ✓ Learn a little every day
- ✓ Build small projects
- ✓ Debug & refactor your code
- ✓ Never stop learning



Code with Passion • Learn with Consistency • Build with Confidence





1. React Fragments

- Fragments let us group multiple elements without adding extra nodes to the DOM.
- Use `<>` or `<React.Fragment>` `</React.Fragment>`.

Example :

```
import React from "react";
function List() {
  return (
    <>
      <h2>Items</h2>
      <ul>
        <li>Apple</li>
        <li>Banana</li>
      </ul>
    </>
  );
}
export default List;
```

Fragments
keep DOM
clean and
improve
readability



2. React Portals

- Portals render children into a DOM node outside the parent component.
- Useful for modals, tooltips, popovers.

Example :

```
import { createPortal } from "react-dom";
function Modal({ children }) {
  return createPortal(
    <div className="modal">{children}</div>,
    document.getElementById("modal-root")
  );
}
```

Portals
help to break
out of parent
containers.



3. React.memo()

- React.memo() memoizes functional components.
- Prevents re-render when props are same.
- Improves performance.

Example :

```
const Button = React.memo(function Button({ onClick, label }) {
  console.log("Button rendered");
  return <button onClick={onClick}>{label}</button>;
});
```

Memoization
= Better
Performance.



4. useCallback() Hook

- Returns memoized callback function.
- Prevents unnecessary re-creation of functions.
- Useful when passing functions to child components.

Example :

```
import { useState, useCallback } from "react";
function Parent() {
  const [count, setCount] = useState(0);
  const increment = useCallback(() => {
    setCount(c => c + 1);
  }, []);
  return <Child onIncrement={increment} />;
}
```

useCallback
keeps function
reference
stable.



5. useMemo() Hook

- Memoizes expensive calculations.
- Re-computes only when dependencies change.

Example :

```
import { useMemo } from "react";
function Expensive({ num }) {
  const result = useMemo(() => {
    console.log("Calculating...");
    return num * num * num; // heavy work
  }, [num]);
  return <h3>Result: {result}</h3>;
}
```

useMemo
saves time
by avoiding
unnecessary
re-computation.



6. useEffect() Cleanup

- Cleanup function runs before next effect or when component unmounts.
- Use for subscriptions, timers, listeners.

Example :

```
import { useEffect } from "react";
function Timer() {
  useEffect(() => {
    const id = setInterval(() => {
      console.log("Tick");
    }, 1000);
    return () => clearInterval(id); // cleanup
  }, []);
  return <p>Timer is running...</p>;
}
```

Always
clean up to
prevent
memory leaks.



7. Prop Drilling vs Context API

Prop Drilling	Context API
- Pass props through many levels. - Hard to manage. - Not scalable. - Example: theme, user.	- Share data without passing props manually. - Easy to manage. - Scalable solution. - Use Context + useContext.

Use Context
for global
or shared
data.



8. Destructuring Props

- Destructuring makes code clean and readable.
- Extract props directly in function parameter.

Example :

```
// Without destructuring
function User(props) {
  return <h2>{props.name}</h2>;
}
// With destructuring
function User({ name, age, city }) {
  return <h2>{name} - {age} - {city}</h2>;
}
```

Clean code
is happy
code.



9. React Error Boundaries

- Catch JavaScript errors anywhere in the child component tree.
- Prevents whole app from crashing.

Example :

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
```

Error
Boundaries
= Better
User
Experience.



10. React Interview Quick Q&A

- Q1. What is React?
A: A JavaScript library for building user interfaces.
- Q2. What is JSX?
A: JavaScript XML syntax extension used in React.
- Q3. What is Virtual DOM?
A: Lightweight copy of real DOM to improve performance.
- Q4. Difference between State and Props?
A: State is mutable & internal. Props are read-only & external.
- Q5. What is useEffect()?
A: Hook for sideEffects like API calls, subscriptions.
- Q6. When to useContext()?
A: When data needs to be shared across many components.

Practice
Questions
make you
Perfect



Summary

- React is component based UI library.
- Hooks make functional components powerful.
- Optimize performance with memo, callback & memo.
- Clean code, reusability & best practices make you a better developer.



Code.
Learn.
Improve.
Repeat.



Remember

- Don't Repeat Yourself (DRY)
- Think before you code
- Break UI into small components
- Test your code
- Keep Learning, Keep Building



You've
Got This! 🚀



Keep Practicing , Keep Building , Keep Growing!





React .JS Notes

Page - 11

1. React Context API

- Context API helps to share data between components without passing props manually at every level.
- It is useful for theme, authentication, language, etc.

Example :

```
// 1. Create Context
export const ThemeContext = React.createContext();

// 2. Provider
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = React.useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

// 3. Use in Component
const { theme, setTheme } = React.useContext(ThemeContext);
```

Great for
global data
like theme,
user, etc.



2. React.memo() vs useCallback() vs useMemo()

Hook / API	Purpose	When to Use	Returns
React.memo()	Memoize component	When component re-renders with same props	Memoized Component
useCallback()	Memoize function	When passing callbacks to child components	Memoized Function
useMemo()	Memoize value	When calculation is expensive	Memoized Value

Example (useCallback):

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, [dependency]);
```

Use these
hooks to
improve
performance



3. React Lifecycle (with Hooks)

- Class components had lifecycle methods.
- In functional components, we use Hooks.

Lifecycle (Class)	Hook Equivalent	Purpose
componentDidMount	useEffect(() => {}, [])	Run once on mount
componentDidUpdate	useEffect(() => {}, [])	Run on update
componentWillUnmount	useEffect(() => { return () => {} , [])	Cleanup on unmount

useEffect
is the heart
of lifecycle
in Hooks.



Example :

```
useEffect(() => {
  console.log("Component Mounted");
  return () => console.log("Component Unmounted");
}, []);
```

4. React Router v6 - Basics

- Used for routing and navigation in React apps.
- v6 uses <Routes> and <Route> instead of <Switch>.

Example :

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home </Link> |
        <Link to="/about">About </Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

v6 is
simpler,
faster &
more
powerful!



5. Controlled vs Uncontrolled Components

Controlled Component	Uncontrolled Component
- Form data is handled by React state. - We use <code>value</code> and <code>onChange</code>. - More control and validation. Example : <pre><input type="text" value={name} onChange={(e) => setName(e.target.value)} /></pre>	- Form data is handled by DOM. - We use <code>ref</code> to access value. - Less re-render, easier for quick forms. Example : <pre>const inputRef = useRef(); <input type="text" ref={inputRef} /> console.log(inputRef.current.value);</pre>

Use
Controlled
for complex
forms.
Uncontrolled
for simple
ones.



7. React.lazy() and Suspense

- React.lazy() is used for code splitting (lazy loading).
- Suspense shows fallback UI while component is loading.

Example :

```
const About = React.lazy(() => import("../About"));

function App() {
  return (
    <Suspense fallback={<h2>Loading...</h2>}>
      <About />
    </Suspense>
  );
}
```

Improves
performance
by loading
components
on demand.



6. useReducer() Hook

- An alternative to useState for complex state logic.
- Follows reducer pattern (state, action) => new state.

Example :

```
const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case "increment": return { count: state.count + 1 };
    case "decrement": return { count: state.count - 1 };
    default: return state;
  }
}

const [state, dispatch] = useReducer(reducer, initialState);
dispatch({ type: "increment" });
```

Great for
complex
state with
multiple
actions!



8. Custom Hook

- Custom Hooks let us extract and reuse logic.
- Name must start with "use".

Example : - useFetch custom hook

```
function useFetch(url) {
  const [data, setData] = React.useState(null);
  React.useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(json => setData(json));
  }, [url]);
  return data;
}
```

Custom Hooks
make code
clean, reusable
and easy to
maintain.



9. React Performance Best Practices

- Use React.memo() to memoize components.
- Use useCallback() for functions.
- Use useMemo() for expensive calculations.
- Avoid inline functions in JSX.
- Use key prop in lists.
- Keep state as local as possible.
- Lazy load routes and heavy components.

Small
optimizations
Big impact!



10. Common Commands (npm)

Command	Purpose
npm start	Start development server
npm run build	Build for production
npm test	Run tests
npm install <package>	Install a package
npm uninstall <package>	Uninstall a package
npm run eject (CRA)	Eject from Create React App

Know
your tools,
Build like a
PRO!





1. React Router (v6)

- React Router is used for navigation in React apps.
- We define routes and link components.

Example :

```
import { BrowserRouter, Routes, Route, Link }
  from "react-router-dom";
function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home </Link> |
        <Link to="/about">About </Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Use
React Router
to build
single page
applications
easily !



2. useEffect Hook

- useEffect is used to perform side effects in components.
- It runs after every render by default.
- We can control when it runs using dependency array.

Example :

```
import { useState, useEffect } from "react";
function Example() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    console.log("Component rendered");
  }, [count]); // runs only when count changes
  return (
    <div>
      <p>Count : {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

useEffect
keeps our
components
in sync
with the
outside world
!



3. useState Hook

- useState is used to add state to functional components.
- It returns an array with current value and updater function.

Example :

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count : {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}
```

State makes
components
dynamic and
interactive
!



4. useRef Hook

- useRef is used to access DOM elements or store mutable values.
- It does not cause re-renders when value changes.

Example :

```
import { useRef } from "react";
function FocusInput() {
  const InputRef = useRef(null);
  const handleClick = () => {
    InputRef.current.focus();
  };
  return (
    <div>
      <input ref={InputRef} type="text" />
      <button onClick={handleClick}>Focus Input </button>
    </div>
  );
}
```

useRef
gives us
a reference
without
re-rendering
!



5. Conditional Rendering

- We can render components conditionally.
- Using if, ternary operator (?:), or && operator.

Example :

```
function Welcome({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <h2>Welcome Back! </h2>
        : <h2>Please Login </h2>}
    </div>
  );
}
```

Other ways:

- if condition
- Ternary (?:)
- Logical &&



6. Lists and Keys

- Use map() to render lists.
- Always give a unique key to each item.

Example :

```
const items = ["Apple", "Banana", "Mango"];
function ListItems() {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item} </li>
      ))}
    </ul>
  );
}
```

Keys help
React identify
which items
changed,
added or
removed !



7. Props vs State

Props	State
Passed from parent component	Managed inside component
Read-only	Can be changed using setState (useState)
Used to communicate between components	Used to manage component data
Cannot be updated	Can be updated
Example: <Child name="Saquib" />	Example: const [count, setCount] = useState(0)

Props go
down,
State stays
inside !



8. React Component Types

- React has two types of components.
- Functional Components and Class Components.

Functional Component	Class Component
Uses functions	Uses ES6 classes
Uses Hooks (useState, useEffect, etc.)	Uses state and lifecycle methods
Short and easy to write	More code
Preferred in modern React	Older approach, still used in some cases

Use
Functional
components
as much
as possible
!



Summary

- React helps us build fast, reusable and scalable UI.
- Components are the building blocks of React.
- State makes UI dynamic and Props make it reusable.
- Hooks make functional components powerful.
- Always write clean, readable and maintainable code.



Keep
Learning
Keep
Growing



Remember

- Practice daily
- Build small projects
- Read documentation
- Use tools & extensions
- Keep code clean
- Never stop learning



You've
Got This !





React.JS Notes

Page - 9

1. React Forms

- We can handle forms in React using state.
- Controlled components are usually used in forms.

Example :

```
import { useState } from "react";
function FormExample() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const handleSubmit = (e) => {
    e.preventDefault();
    alert('Name: ${name}, Email: ${email}');
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" placeholder="Name"
        value={name} onChange={(e) => setName(e.target.value)} />
      <input type="email" placeholder="Email"
        value={email} onChange={(e) => setEmail(e.target.value)} />
      <button type="submit"> Submit </button>
    </form>
  );
}
export default FormExample;
```

Forms in React are easy with state and events 😊

2. React Fragment

- Fragment lets us group multiple elements without adding an extra node to the DOM.
- Use <> </> or <React.Fragment> </React.Fragment>.

Example :

```
import React from "react";
function FragmentExample() {
  return (
    <>
      <h1>Hello React</h1>
      <p>This is a fragment</p>
    </>
  );
}
export default FragmentExample;
```

Use Fragment to keep DOM clean and lightweight ★

3. React Portals (Advanced)

- Portals render children into a DOM node outside the parent hierarchy.
- Useful for modals, tooltips, notifications.

Example :

```
import { createPortal } from "react-dom";
function Portal({ children }) {
  return createPortal(children, document.getElementById("portal-root"));
}
export default Portal;
```

Portals help to break out of parent containers ★

4. Error Handling (try...catch in Components)

- We can handle errors in lifecycle methods or async code using try...catch.

Example :

```
import { useEffect, useState } from "react";
function ErrorExample() {
  const [data, setData] = useState(null);
  useEffect(() => {
    const fetchData = async () => {
      try {
        const res = await fetch("https://api.example.com/data");
        const json = await res.json();
        setData(json);
      } catch (error) {
        console.log("Error:", error);
      }
    };
    fetchData();
  }, []);
  return <div>{data ? "Data Loaded" : "Loading..."}</div>;
}
export default ErrorExample;
```

Always handle errors gracefully 😊

5. Custom Hooks

- Custom Hooks let us extract component logic into reusable functions.
- Name must start with "use".

Example : create a useTitle hook.

```
import { useEffect } from "react";
function useTitle(title) {
  useEffect(() => {
    document.title = title;
  }, [title]);
}
export default useTitle;
// Usage in component
function MyComponent() {
  useTitle("My React App");
  return <h2>Check the title in tab!</h2>;
}
```

Custom Hooks make code clean, reusable and easy to maintain ❤️

6. React Performance - Advanced

- Optimize re-renders and improve performance.
- Tools: React.memo, useMemo, useCallback.

Technique	Purpose	Example
React.memo	Prevents re-render of functional components if props are same	const MemoComp = React.memo(MyComp);
useMemo	Memoize expensive calculations	const value = useMemo(() => expensiveFunc(a, b), [a, b]);
useCallback	Memoize functions to prevent re-creation	const handleClick = useCallback(() => { ... }, [deps]);
Virtualization	Render only visible items in large lists	Use libraries like react-window / react-virtualized

7. React Best Practices

- ☑ Keep components small and focused.
- ☑ Use meaningful names for variables and components.
- ☑ Follow single responsibility principle.
- ☑ Avoid inline functions and objects in JSX.
- ☑ Use keys in lists.
- ☑ Handle errors and edge cases.
- ☑ Write reusable custom hooks.
- ☑ Keep your code DRY (Don't Repeat Yourself).

Good Code
Happy Developer
😊

8. Summary

- ★ React is a powerful JavaScript library for building UI.
- ★ Components are building blocks of React applications.
- ★ Props, State and Hooks are core concepts.
- ★ React Router helps in navigation.
- ★ Performance optimization makes apps faster.
- ★ Best practices and clean code are important.
- ★ Keep learning and building projects! ❤️



React.JS Notes

Page - 8

1. React Testing Basics

- Testing helps to verify that components work as expected.
- Popular library: Jest + React Testing Library

Example:

```
import { render, screen } from '@testing-library/react';
import App from './App';

test('renders heading', () => {
  render(<App />);
  const heading = screen.getByRole('heading');
  expect(heading).toBeInTheDocument();
});
```

★ Test what user sees, not implementation! 😊

2. React Developer Tools (Browser Extension)

- Official extension for Chrome and Firefox.
- Helps to inspect component hierarchy, props, state and performance.

• Features:

- Components tab
- Profiler tab
- State & Props view
- Highlight updates

Must have tool for every React developer!

3. Higher Order Components (HOC)

- HOC is a function that takes a component and returns a new component.
- Used for code reusability and logic sharing.

Example:

```
function withLogin(WrappedComponent) {
  return function (props) {
    const isLoggedIn = true;
    if(!isLoggedIn) return <h2>Please Login</h2>;
    return <WrappedComponent {...props} />;
  }
}
export default withLogin;
```

HOC is a pattern, not a part of React itself. 😊

4. Render Props

- Technique for sharing code using a prop whose value is a function.
- Provides more flexibility than HOC.

Example:

```
function DataProvider({ render }) {
  const data = "Hello React";
  return <div>{render(data)}</div>;
}
// Usage
<DataProvider render={({data}) =>
  <h2>{data}</h2>
} />
```

Render Props uses a function to share logic. ★

5. React Portals

- Portals help to render children into a DOM node that exists outside the parent component.
- Useful for modals, tooltips, and overlays.

Example:

```
import { createPortal } from "react-dom";

function PortalExample() {
  return createPortal(
    <div className="modal">Hello Portal</div>,
    document.getElementById('portal-root')
  );
}
```

HTML:

```
<div id="root"></div>
<div id="portal-root">
  </div>
```

♥

6. Code Splitting with React.lazy()

- Code splitting helps to reduce the bundle size.
- Components are loaded only when needed.

Example:

```
const Home = React.lazy(() => import("./pages/Home"));
const About = React.lazy(() => import("./pages/About"));

function App() {
  return (
    <Suspense fallback=<h2>Loading...</h2>>
      <Home />
    </Suspense>
  );
}
```

Use Suspense as a fallback. 😊

7. Performance Optimization (Advanced Tips)

- ✓ Use React.memo() to prevent unnecessary re-renders.
- ✓ Use useCallback() for function memoization.
- ✓ Use useMemo() for expensive calculations.
- ✓ Avoid inline functions in JSX.
- ✓ Keep state as local as possible.
- ✓ Use React Developer Tools Profiler.
- ✓ Lazy load routes and heavy components.



Small optimizations, Big impact!

8. Common React Interview Topics

1. What is React?
2. What is JSX?
3. Props vs State?
4. What is Virtual DOM?
5. Lifecycle of a component?
6. What are Hooks?
7. useState vs useRef?
8. useEffect and dependency array?
9. Controlled vs Uncontrolled components?
10. How does React Router work?
11. What is code splitting?
12. How to optimize performance?
13. Error Boundaries?

Summary

- React is all about building reusable UI components.
 - Understand core concepts deeply.
 - Use Hooks and modern patterns.
 - Write clean, efficient and testable code.
 - Keep learning and building projects.
- ♥

Remember :

- ✓ Code
- ✓ Learn
- ✓ Build
- ✓ Debug
- ✓ Repeat

Consistent Practice is the Key to Mastery!



Happy Coding! Keep Building Amazing Things with React





React.JS Notes

Page - 7

1. Controlled vs Uncontrolled Components

- In controlled components, form data is handled by React state.
- In uncontrolled components, form data is handled by the DOM itself using refs.

Feature	Controlled	Uncontrolled
Data	Handled by React state	Handled by DOM
Access	Easier (state values)	Using refs
Validation	Easy	Difficult
Re-renders	More	Less
Use Case	Most of the time	File inputs, small forms

Example (Controlled Input):

```
const [name, setName] = useState("");
return (
  <input
    type="text"
    value={name}
    onChange={(e) => setName(e.target.value)}
  />
);
```

Controlled components give more control!

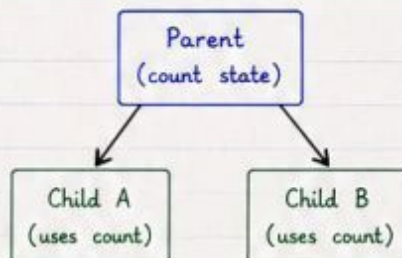
2. Lifting State Up

- Sometimes, multiple components need to share the same state.
- In that case, we lift the state up to their common parent component.

Example:

```
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <ChildA count={count} setCount={setCount}/>
      <ChildB count={count} setCount={setCount}/>
    </div>
  );
}
```



State flows down as props, events flow up!

3. React.lazy() and Suspense

- React.lazy() is used for code splitting (lazy loading).
- Suspense shows a fallback UI while the component is loading.

Example:

```
const About = React.lazy(() => import('./About'));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <About />
    </Suspense>
  );
}
```

Improves performance by loading components on demand.

4. Error Handling with Error Boundary

- Error Boundaries catch errors in children components.
- They prevent the whole app from crashing.

Example:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong.</h2>;
    }
    return this.props.children;
  }
}
```

Use Error Boundary for better user experience.

5. Performance Optimization Tips

- Use React.memo() to memoize functional components.
- Use useMemo() to memoize expensive calculations.
- Use useCallback() to memoize functions.
- Avoid inline functions in JSX.
- Use key prop in lists.
- Code split using React.lazy().
- Avoid unnecessary state updates.

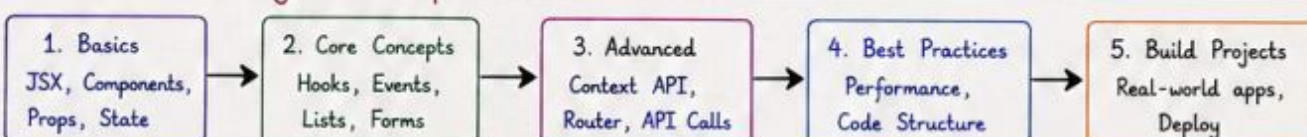


Optimize Smartly, Not Hard!

6. Useful React Snippets

Task	Snippet
Functional Components	<code>const MyComp = () => { return (<div></div>); }</code>
useState	<code>const [state, setState] = useState(initialValue);</code>
useEffect	<code>useEffect(() => { /* code */ }, [deps]);</code>
Conditional Rendering	<code>{condition ? <A /> : }</code>
List Rendering	<code>{items.map(item => <Item key={item.id} />)}</code>
Event Handling	<code>onClick={() => handleClick()}</code>
Passing Props	<code><Child name={name} age={age} /></code>

7. React Learning Roadmap



Keep Learning, Keep Building, Keep Growing!





React.JS Notes

Page - 6

1. Context API (State Management)

- Context API is used to share data between components without passing props manually at every level.
- It helps in avoiding "props drilling".

Example :

```
// 1. Create Context
import { createContext, useContext, useState } from "react";
export const ThemeContext = createContext();

// 2. Provider Component
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

// 3. Use in any component
export const useTheme = () => useContext(ThemeContext);
```

★ Use Context API only when needed, not for every state. ★

2. React.memo()

- React.memo() is a higher-order component.
- It prevents unnecessary re-renders by memoizing the component.
- It only re-renders when props actually change.

Example :

```
const MyComponent = React.memo(({ user }) => {
  console.log("Rendering MyComponent");
  return <h3>{user.name}</h3>;
});
```

Use React.memo() for functional components that receive the same props frequently.



3. useCallback vs useMemo

Hook	Purpose	Returns	Use Case
useCallback	Memoizes a function	Memoized function	When passing functions as props to child components
useMemo	Memoizes a value	Memoized value	When calculation is expensive

Example :

useCallback

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, [dependencies]);
```

useMemo

```
const total = useMemo(() => {
  return a + b;
}, [a, b]);
```

♥ Use useCallback for functions, ♥ useMemo for values.

4. Error Boundary

- Error Boundaries are React components that catch JavaScript errors anywhere in their child component tree.
- They log errors and display fallback UI instead of crashing the whole app.

Example :

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true };
  }
  render() {
    if (this.state.hasError) {
      return <h2>Something went wrong. Please try again.</h2>;
    }
    return this.props.children;
  }
}
```

Wrap your app or routes with ErrorBoundary for better UX.



5. React Developer Tools

- React Developer Tools is a browser extension.
- It helps to inspect React components, state, props and performance.
- Available for Chrome and Firefox.

Things you can do :

- Inspect component hierarchy
- See props and state
- Highlight updates
- Measure performance



Install it for better debugging !

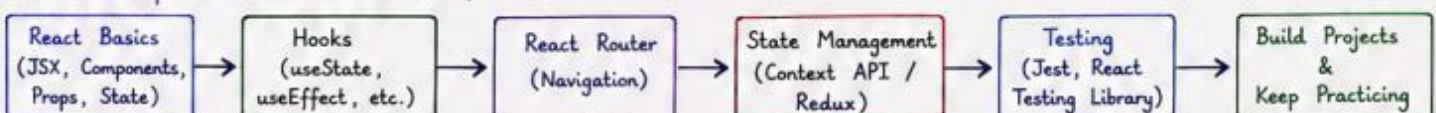
6. Best Practices

- ★ Keep components small and reusable.
- ★ Use meaningful variable and component names.
- ★ Avoid inline styles, use CSS or styled-components.
- ★ Use keys in lists.
- ★ Keep state as local as possible.
- ★ Remove unused code and dependencies.
- ★ Follow single responsibility principle.
- ★ Use React.memo, useCallback, useMemo when required.
- ★ Write clean, readable and maintainable code.



Clean Code
Better Code
Best Code

7. Roadmap (What to Learn Next)



★ Stay Consistent, Keep Learning, Never Stop Building! ♥ 😊



React.JS Notes

Page - 5

1. React Router (Navigation)

- React Router is used to build single page applications with navigation.
- It provides `<BrowserRouter>`, `<Routes>`, `<Route>`, `<Link>`, etc.

Example :

```
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";

function App() {
  return (
    <Router>
      <nav>
        <Link to="/">Home </Link> |
        <Link to="/about">About </Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </Router>
  );
}
export default App;
```

2. API Calls in React

- We use `fetch()` or `axios` to make API calls.
- API calls are usually made inside `useEffect`.

Example using `fetch` :

```
import { useEffect, useState } from "react";
function Users() {
  const [users, setUsers] = useState([]);
  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then(res => res.json())
      .then(data => setUsers(data))
      .catch(err => console.log(err));
  }, []);
  return (
    <div>
      <h2>Users </h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>{user.name}</li>
        ))}
      </ul>
    </div>
  );
}
export default Users;
```

Don't forget
to handle
loading & error
states!



3. Project Structure (Best Practice)

my-react-app/

```
├── public/
│   └── index.html
├── src/
│   ├── assets/      # images, icons
│   ├── components/  # reusable components
│   ├── pages/       # page components
│   ├── hooks/       # custom hooks
│   ├── context/     # context API
│   ├── services/    # API calls
│   ├── utils/       # helper functions
│   ├── App.js
│   ├── index.js
│   └── styles/      # global styles
├── .gitignore
├── package.json
└── README.md
```

★
Keep components
small and
reusable.
Follow single
responsibility
principle. ♥

4. Interview Questions (Most Asked)

1. What is React?
2. What are the features of React?
3. What is JSX?
4. Difference between Props and State?
5. What are React Hooks? Explain `useState` and `useEffect`.
6. What is Virtual DOM?
7. What is the use of `key` in lists?
8. What is the difference between controlled and uncontrolled components?
9. What is lifting state up?
10. What is React Router?
11. How does `useEffect` cleanup work?
12. What is the use of `useRef`?



5. Summary

- React is a JavaScript library for building user interfaces.
- It is component based, fast and maintainable.
- JSX makes writing UI easier.
- Props are used to pass data. State is used to manage dynamic data.
- Hooks let us use state and other React features in functional components.
- React Router is used for navigation.

Remember :

- ☒ Practice daily
- ☒ Build projects
- ☒ Read documentation
- ☒ Stay consistent
- ☒ You will become a great developer! ♥



Code. Learn. Build. Repeat.



Keep Coding and Keep Growing!



React.JS Notes

Page - 4

1. React Hooks

- Hooks are special functions that let us use React features in functional components.
- Hooks start with "use" (useState, useEffect, etc.)

1.1 useState Hook

- useState is used to add state to functional components.
- It returns an array with two values: the current state and a function to update it.

Example :

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
export default Counter;
```

1.2 useEffect Hook

- useEffect is used to perform side effects in functional components.
- It runs after every render.
- We can control when it runs using dependency array.

Example :

```
import { useState, useEffect } from "react";
function Timer() {
  const [time, setTime] = useState(0);
  useEffect(() => {
    const interval = setInterval(() => {
      setTime(time => time + 1);
    }, 1000);
    return () => clearInterval(interval); // cleanup
  }, []); // empty array -> run once when component mounts

  return <h2>Time: {time}s</h2>;
}
export default Timer;
```

1.3 useRef Hook

- useRef is used to access DOM elements directly.
- It also helps to persist values between renders without causing re-renders.

Example :

```
import { useRef } from "react";
function FocusInput() {
  const inputRef = useRef(null);

  const handleClick = () => {
    inputRef.current.focus();
  };
  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Enter text" />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}
export default FocusInput;
```

1.4 useMemo Hook

- useMemo is used to memoize (cache) expensive calculations.
- It only recalculates when dependencies change.

Example :

```
import { useMemo } from "react";
function ExpensiveCalculation({ num }) {
  const result = useMemo(() => {
    console.log("Calculating...");
    let sum = 0;
    for (let i = 1; i <= num; i++) sum += i;
    return sum;
  }, [num]); // runs only when 'num' changes

  return <h3>Sum: {result}</h3>;
}
export default ExpensiveCalculation;
```

1.5 useCallback Hook

- useCallback returns a memoized callback function.
- It prevents unnecessary re-creation of functions.

Example :

```
import { useState, useCallback } from "react";
function Parent() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    console.log("Button clicked");
  }, []); // function will not be recreated

  return <Child onClick={handleClick} />;
}
export default Parent;
```

1.6 Summary of Hooks

Hook	Purpose	Returns	Dependencies
useState	Manage State	[state, setState]	No
useEffect	Side Effects	Nothing	Optional
useRef	Access DOM / Persist Value	{ current }	No
useMemo	Memoize Value	Memoized Value	Yes (Array)
useCallback	Memoize Function	Memoized Function	Yes (Array)



Hooks make our components powerful and efficient.



Keep Practicing!

1. State in React

- State is a built-in object that stores data that can change over time.
- When state changes, component re-renders.
- useState Hook is used to manage state in functional components.

Example :

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment</button>
    </div>
  );
}
export default Counter;
```

4. Lists and Keys

- We can render lists using map().
- Each item in the list should have a unique "key".

Example :

```
function FruitList() {
  const fruits = ["Apple", "Banana", "Mango"];
  return (
    <ul>
      {fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
export default FruitList;
```

2. Event Handling

- Events are written in camelCase.
- Pass a function as an event handler.

Example :

```
function Button() {
  function handleClick() {
    alert("Button Clicked!");
  }
  return (
    <button onClick={handleClick}>Click Me</button>
  );
}
export default Button;
```

3. Conditional Rendering

- We can render elements conditionally using if statement or ternary operator.

Example :

```
function Status({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? (<p>Welcome Back!</p>) :
        (<p>Please Login.</p>)}
    </div>
  );
}
export default Status;
```

Output :

- Apple
- Banana
- Mango

5. Forms in React

- We can handle form inputs using state.
- onChange event is used to update state.

Example :

```
function FormExample() {
  const [name, setName] = useState("");
  const handleChange = (e) => {
    setName(e.target.value);
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    alert("Name: " + name);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" value={name} onChange={handleChange}
        placeholder="Enter your name" />
      <button type="submit">Submit</button>
    </form>
  );
}
export default FormExample;
```

6. Fragment in React

- Fragment is used to group multiple elements without adding extra nodes to the DOM.

Example :

```
import React, { Fragment } from "react";
function MyComponent() {
  return (
    <Fragment>
      <h1>Hello!</h1>
      <p>This is a fragment example.</p>
    </Fragment>
  );
}
export default MyComponent;
```

Important Point :

State = Data that can change
Props = Data that is passed

1. JSX (JavaScript XML)

- JSX allows us to write HTML inside JavaScript.
- JSX is not valid HTML in the browser.
- It is converted into JavaScript by Babel before rendering.
- Every JSX element must have a single parent element.

Example:

```
const element = (  
  <div>  
    <h1>Hello React!</h1>  
    <p>This is JSX.</p>  
  </div>  
)  
;  
// This JSX will be rendered as HTML in browser.
```

2. Components

- Components are the building blocks of React.
- A component is a JavaScript function that returns JSX.
- Components can be reused.
- Component name must start with a capital letter.

Example - Functional Component

```
function Welcome() {  
  return (  
    <div>  
      <h2>Welcome to React!</h2>  
      <p>Reusable Component</p>  
    </div>  
  );  
}  
export default Welcome;
```

3. Props (Properties)

- Props are used to pass data from one component to another.
- Props are read-only (cannot be modified).
- Props make components dynamic and reusable.

Example:

Parent Component

```
import Greet from "./Greet";  
function App() {  
  return (  
    <div>  
      <Greet name="Saqib" age={24} />  
    </div>  
  );  
}  
export default App;
```

Child Component (Greet.js)

```
function Greet(props) {  
  return (  
    <div>  
      <h2>Hello, {props.name}!</h2>  
      <p>Age: {props.age}</p>  
    </div>  
  );  
}  
export default Greet;
```

Output:

Hello, Saqib!
Age: 24

4. Default Props

- We can set default values for props.
- If no value is passed, default value will be used.

Example:

```
function Greet(props) {  
  return <h2>Hello, {props.name}</h2>;  
}  
Greet.defaultProps = {  
  name: "Guest"  
};  
// If name prop is not provided, "Guest" will be shown.
```

5. PropTypes (Type Checking)

- PropTypes helps in type checking of props.
- It is a useful tool during development.

Example:

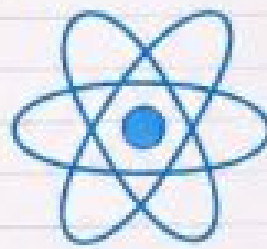
```
import PropTypes from "prop-types";  
function Greet(props) {  
  return <h2>Hello, {props.name}</h2>;  
}  
Greet.propTypes = {  
  name: PropTypes.string.isRequired  
};  
// "name" prop is required and should be a string.
```

React.JS Notes

Page - 1

1. What is React?

- React is a JavaScript library used to build user interfaces, especially single page applications.
- It was developed and maintained by Facebook (now Meta).
- It is fast, scalable and component based.



React

Build UIs
with components

2. Features of React

- ★ Component Based - Reusable UI components
- ★ Declarative - Describe what UI should look like.
- ★ Virtual DOM - Improves performance.
- ★ Single Direction Data Flow - Easier to debug.
- ★ JSX - Write HTML inside JavaScript.
- ★ Strong Community - Lots of support and libraries.

3. Installation

Prerequisite - Node.js and npm

Create React App using Vite (Recommended)

```
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

- This will start the development server.
- Open <http://localhost:5173> in browser.

4. What is JSX?

- JSX stands for JavaScript XML.
- It allows us to write HTML-like code inside JavaScript.
- JSX makes code more readable.

Example:

```
const element = (
  <h1>Hello React!</h1>
);
```

6. Props

- Props (properties) are used to pass data from one component to another.
- Props are read-only.

Example:

```
function Greet(props) {
  return <h3>Hello, {props.name}!</h3>;
}
```

Usage:

```
<Greet name="Saquib" />
```

5. Components

- Components are the building blocks of React.
- There are two types of components:
 1. Functional Components
 2. Class Components (old way)

Example - Functional Component

```
function Welcome() {
  return <h2>Welcome to React!</h2>;
}
export default Welcome;
```

7. Key Points

- ⊙ React is a library, not a framework.
- ⊙ It uses components to build UI.
- ⊙ JSX makes writing UI easier.
- ⊙ Props help in passing data.
- ⊙ Install Node.js before using React.

"Learn, Practice, Build
and Become a Better Developer."



MongoDB + mongoose

MongoDB + Mongoose

Connect Database with Node.js

Store • Manage • Build Scalable Applications

Learn
Build
Grow



What is MongoDB?



MongoDB

- NoSQL database
- Stores data in flexible, JSON-like documents
- No fixed schema
- Fast, scalable and highly available



Think of MongoDB as a database where data is stored in documents (similar to JSON).



What is Mongoose?

mongoose

- ODM (Object Data Modeling) library
- Provides schema, model and query features
- Makes it easier to work with MongoDB in Node.js



Think of Mongoose as a bridge between your Node.js application and MongoDB.



How It Works (Flow)



Node.js
(Your App)



mongoose

Mongoose
(ODM)



MongoDB
(Database)



Data
Stored

Node.js + Mongoose = Easy way to connect and work with MongoDB.



Simple Code Example

```

1 const mongoose = require('mongoose');
2
3 // Connect to MongoDB
4 mongoose.connect('mongodb://localhost:27017/myapp', {
5   useNewUrlParser: true,
6   useUnifiedTopology: true,
7 })
8 .then(() => console.log('MongoDB connected'))
9 .catch(err => console.log('MongoDB connection error:', err));
10
11 // Define a schema
12 const userSchema = new mongoose.Schema({
13   name: { type: String, required: true },
14   email: { type: String, required: true }
15 });
16
17 // Create a model
18 const User = mongoose.model('User', userSchema);

```



Working with Data (Example)

1

Create (Save data)

```

const user = new User({
  name: "Mahesh",
  email: "mahesh@example.com"
});
user.save();

```

2

Read (Find data)

```

User.find().then(users => {
  console.log(users);
});

```

3

Update (Update data)

```

User.findByIdAndUpdate(id, { name: "New Name" });

```

4

Delete (Remove data)

```

User.findByIdAndDelete(id);

```



Common Mongoose Methods

save()

Save a new document

find()

Get all documents

findById()

Get document by ID

updateOne()

Update a document

deleteOne()

Delete a document



Key Points to Remember

- ✓ MongoDB stores data in a flexible JSON format.
- ✓ Mongoose helps to work with MongoDB using schemas, models and queries.
- ✓ It makes code simpler and more organized.
- ✓ It's an important part of the MERN stack.



MERN Stack



MongoDB
(Database)

+

mongoose

Mongoose
(ODM)

+



Node.js
(Backend)

+



React
(Frontend)



Next Step

Now I will practice more with MongoDB queries, create routes and build a complete CRUD application using the MERN stack.

One step closer
to my dream



Connect with me on LinkedIn
<https://www.linkedin.com/in/mahesh-zinzala>

#MongoDB #Mongoose #NodeJS #ExpressJS #MERNStack
#BackendDevelopment #WebDevelopment #LearningInPublic

Keep
Learning

17. AngularJS CRUD Application

What is CRUD Application ?

- CRUD stands for Create, Read, Update and Delete operations.
- In AngularJS, we can perform CRUD using ng-model, ng-repeat and \$http service.
- We will use a REST API as a backend (JSONPlaceholder in this example).
- This example shows how to integrate AngularJS with a real API.

Technologies Used

- AngularJS 1.x (Frontend)
- \$http service for API calls
- JSONPlaceholder API (<https://jsonplaceholder.typicode.com>)
- ng-model, ng-repeat, ng-click
- Bootstrap for simple UI (optional)

CRUD Flow



1. HTML Structure (index.html)

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
<title>AngularJS CRUD</title>
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/bootstrap@3.4.1/
dist/css/bootstrap.min.css">
<script src="https://ajax.googleapis.com/ajax/libs/
angularjs/1.8.2/angular.min.js"></script>
<script src="app.js"></script>
</head>
<body ng-controller="UserCtrl">
<div class="container">
<h2>AngularJS CRUD Application</h2>
  
```

2. User Interface (HTML)

```

<div class="row">
<div class="col-md-6">
<h3 ng-show="!editing">Add User</h3>
<h3 ng-show="editing">Update User</h3>
<form ng-submit="saveUser()">
<div class="form-group">
<input type="text" class="form-control"
ng-model="user.name" placeholder="Name"
required>
</div>
<div class="form-group">
<input type="email" class="form-control"
ng-model="user.email" placeholder="Email"
required>
</div>
<button type="submit" class="btn btn-primary">
{{ editing ? 'Update User' : 'Add User' }}
</button>
<button type="button" class="btn btn-default"
ng-click="resetForm()" ng-show="editing">
Cancel
</form>
</div>
  
```

Users List

```

<div class="col-md-6">
<h3>Users List</h3>
<table class="table table-bordered
table-striped">
<thead>
<tr>
<th>ID</th>
<th>Name</th>
<th>Email</th>
<th>Actions</th>
</tr>
</thead>
<tbody>
<tr ng-repeat="u in users">
<td>{{ u.id }}</td>
<td>{{ u.name }}</td>
<td>{{ u.email }}</td>
<td>
<button class="btn btn-xs
btn-success" ng-click="editUser(u)">
Edit</button>
<button class="btn btn-xs
btn-danger" ng-click="deleteUser(u.id)">
Delete</button>
</td>
</tr>
</tbody>
</table>
</div>
  
```

3. AngularJS App (app.js - CRUD Logic)

```

var app = angular.module('myApp', []);
app.controller('UserCtrl', function($scope, $http) {
var apiUrl = 'https://jsonplaceholder.typicode.com/users';

$scope.users = [];
$scope.user = { name: '', email: '' };
$scope.editing = false;

// Read - Fetch all users
$scope.getUsers = function() {
$http.get(apiUrl).then(function(response) {
$scope.users = response.data;
}).catch(function(error) {
console.log('Error:', error);
});
};

// Create or Update
$scope.saveUser = function() {
if ($scope.editing) {
// Update existing user (PUT)
$http.put(apiUrl + '/' + $scope.user.id, $scope.user)
.then(function(response) {
alert('User updated successfully!');
$scope.resetForm();
$scope.getUsers();
}).catch(function(error) {
console.log('Error:', error);
});
} else {
// Add new user (POST)
$http.post(apiUrl, $scope.user).then(function(response) {
alert('User added successfully!');
$scope.resetForm();
$scope.getUsers();
}).catch(function(error) {
console.log('Error:', error);
});
}
};

// Edit User
$scope.editUser = function(user) {
$scope.user = angular.copy(user);
$scope.editing = true;
};

// Delete User
$scope.deleteUser = function(id) {
if (confirm('Are you sure to delete this user?')) {
$http.delete(apiUrl + '/' + id).then(function() {
alert('User deleted successfully!');
$scope.getUsers();
}).catch(function(error) {
console.log('Error:', error);
});
}
};

// Reset form
$scope.resetForm = function() {
$scope.user = { name: '', email: '' };
$scope.editing = false;
};

// Load users initially
$scope.getUsers();
});
  
```

4. Output (Browser View)

Initial View (Read Users)

http://localhost/index.html

AngularJS CRUD Application

Add User

Name

Email

Add User

Users List

ID	Name	Email	Actions
1	Leanne Graham	leanne@ex.com	Edit Delete
2	Ervin Howell	ervin@ex.com	Edit Delete
3	Clementine Bauch	clementine@ex.com	Edit Delete

After Add / Update

http://localhost/index.html

AngularJS CRUD Application

Update User

Narayan

narayan@gmail.com

Update User Cancel

Users List

ID	Name	Email	Actions
1	Narayan	narayan@gmail.com	Edit Delete
2	Ervin Howell	ervin@ex.com	Edit Delete
3	Clementine Bauch	clementine@ex.com	Edit Delete

Key Takeaways

- ✓ AngularJS can perform CRUD operations using \$http service.
- ✓ ng-model is used for two-way data binding.
- ✓ REST API is used to store and manage data.
- ✓ ng-repeat is used to display data in a table.
- ✓ CRUD operations include GET, POST, PUT and DELETE.
- ✓ This approach helps to build real-world web applications.

Real-life Use Cases

- User management system (Add, Edit, Delete users)
- Product management in e-commerce applications.
- Employee management in organizations.
- Managing blog posts (articles, comments).
- Any application that needs to create, read, update and delete data.

16. Promises, \$q & Asynchronous Programming

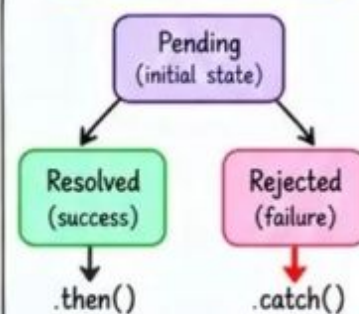
What is Asynchronous Programming ?

- Asynchronous programming means running time-consuming tasks (like API calls, timers, etc.) in the background without blocking the main code.
- It helps the application remain responsive.
- In AngularJS, we handle asynchronous operations using Promises and \$q service.
- Common examples: HTTP requests, time delays, file loading, etc.

What is a Promise ?

- A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.
- A promise has 3 states:
 - Pending - initial state
 - Resolved (Fulfilled) - operation completed successfully
 - Rejected - operation failed with an error

Promise States



1. Using Native Promise in JavaScript (AngularJS)

```

app.js
var app = angular.module('myApp', []);
app.controller('TestCtrl', function($scope) {
  var promise = new Promise(function(resolve, reject) {
    setTimeout(function() {
      var success = true;
      if(success) {
        resolve('Data loaded successfully!');
      } else {
        reject('Something went wrong!');
      }
    }, 2000);
  });
  promise.then(function(data) {
    $scope.message = data;
  }).catch(function(error) {
    $scope.message = error;
  });
});
    
```

Output (Browser)

Data loaded successfully!

Key Points

- new Promise() creates a promise.
- resolve() is called when operation is successful.
- reject() is called when operation fails.
- .then() handles success.
- .catch() handles errors.

2. Using \$q Service in AngularJS

```

app.js
var app = angular.module('myApp', []);
app.controller('TestCtrl', function($scope, $q) {
  // Create a custom promise using $q
  var deferred = $q.defer();
  setTimeout(function() {
    var success = true;
    if(success) {
      deferred.resolve('Data loaded with $q!');
    } else {
      deferred.reject('Error occurred!');
    }
  }, 2000);
  // deferred.promise is returned
  deferred.promise.then(function(data) {
    $scope.message = data;
  }).catch(function(error) {
    $scope.message = error;
  });
});
    
```

Output (Browser)

Data loaded with \$q!

Key Points

- \$q is AngularJS's built-in service for promises.
- \$q.defer() creates a deferred object with 3 parts:
 - promise
 - resolve()
 - reject()
- It is useful when we need to create our own custom asynchronous logic.

3. Real Example: \$http with Promise

```

app.js
var app = angular.module('myApp', []);
app.controller('UserCtrl', function($scope, $http) {
  $http.get('https://jsonplaceholder.typicode.com/users/1')
    .then(function(response) {
      $scope.user = response.data; // success
    })
    .catch(function(error) {
      $scope.error = 'Failed to load data!'; // error
    });
});
    
```

Output (Browser)

User Details:

Name: Leanne Graham
Email: leanne@ex.com
City: Gwenborough

Key Points

- \$http returns a promise.
- Use .then() for success and .catch() for errors.
- response.data contains the actual data.

4. Chaining Multiple Promises

```

app.js
var app = angular.module('myApp', []);
app.controller('ChainCtrl', function($scope, $q) {
  function step1() {
    return $q(function(resolve) {
      setTimeout(function() {
        resolve('Step 1 Done');
      }, 1000);
    });
  }
  function step2(msg) {
    return $q(function(resolve) {
      setTimeout(function() {
        resolve(msg + ' → Step 2 Done');
      }, 1000);
    });
  }
  step1().then(step2).then(function(finalMsg) {
    $scope.result = finalMsg;
  });
});
    
```

Output (Browser)

Step 1 Done →
Step 2 Done

Key Points

- Promises can be chained using multiple .then().
- Each .then() can return another promise.
- Helps in sequential execution of asynchronous tasks.

5. Using \$q.all() for Multiple Requests

```

app.js
var app = angular.module('myApp', []);
app.controller('MultiCtrl', function($scope, $q, $http) {
  var req1 = $http.get('https://jsonplaceholder.typicode.com/users/1');
  var req2 = $http.get('https://jsonplaceholder.typicode.com/posts/1');
  $q.all([req1, req2]).then(function(results) {
    $scope.user = results[0].data;
    $scope.post = results[1].data;
  }).catch(function(error) {
    $scope.error = 'One of the requests failed!';
  });
});
    
```

Output (Browser)

User: Leanne Graham

Post Title: sunt aut fugiat repellat provident occaecati excepturi optio reprehenderit

Key Points

- \$q.all() waits for all promises to complete.
- Returns an array of results.
- Useful for multiple API calls.

Real-life Use Cases

- Calling multiple REST APIs.
- Loading dashboard data in parallel.
- Submitting form and then loading updated data.
- Handling timeouts, delays and animations.
- Sequential operations (e.g., login → get profile → get settings).
- Improving user experience with non-blocking operations.

Key Takeaways

- ✓ Promises handle asynchronous operations.
- ✓ \$q is AngularJS's built-in promise service.
- ✓ Use .then() for success and .catch() for errors.
- ✓ \$q.defer() helps to create custom promises.
- ✓ \$q.all() is used for multiple asynchronous calls.
- ✓ Useful for API calls, delays, and better user experience.

15. Custom Directives & Components

What are Directives in AngularJS ?

- Directives are special markers on a DOM element (like attributes, elements, or classes) that tell AngularJS to do something.
- We can create custom directives to extend HTML with our own behavior.
- Built-in directives: ng-app, ng-model, ng-show, ng-repeat, etc.
- Custom directives: created using directive() method.

What are Components in AngularJS ?

- Components are a special type of directive introduced in AngularJS 1.5+.
- They make the code more modular, reusable and easy to maintain.
- A component has its own template, controller, bindings and is used as a custom HTML element.
- It follows a more structured approach compared to traditional directives.

Directive vs Component

Feature	Directive	Component
Purpose	General purpose	Special type of directive
Syntax	directive()	component()
Usage	Attribute, Element, Class, Comment	Custom HTML element only
Introduced	AngularJS 1.0	AngularJS 1.5+
Structure	Flexible	More structured
Use Case	Small DOM manipulations	Reusable UI components

1. Custom Directive (Attribute Directive)

```
index.html
<div ng-app="myApp" ng-controller="myCtrl">
  <h3 custom-message></h3>
</div>
```

Output :

Hello from Custom Directive!

```
app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  // Controller logic (if needed)
});
app.directive('customMessage', function() {
  return {
    restrict: 'A', // Attribute directive
    template: '<p>Hello from Custom Directive!</p>'
  };
});
```

Key Points

- restrict: 'A' means attribute directive.
- Replaces the element with the template.
- Useful for small DOM enhancements.
- Can also use 'E', 'C', 'M' (element, class, comment).

2. Custom Directive (Element Directive)

```
index.html
<div ng-app="myApp" ng-controller="myCtrl">
  <user-card></user-card>
</div>
```

Output :

User: Narayan
AngularJS Custom Directive

```
app.js
var app = angular.module('myApp', []);
app.directive('userCard', function() {
  return {
    restrict: 'E', // Element directive
    template:
      '<div class="card">
        <h4>User: Narayan</h4> +
        <p>AngularJS Custom Directive</p>
      </div>'
  };
});
```

Key Points

- restrict: 'E' means element directive.
- Creates a custom HTML element.
- Useful for reusable UI parts.
- Can be styled with CSS.

3. AngularJS Component (1.5+)

```
index.html
<div ng-app="myApp">
  <student-info name="Narayan" age="21"></student-info>
</div>
```

Output :

Student Name: Narayan
Age: 21

```
app.js
var app = angular.module('myApp', []);
app.component('studentInfo', {
  bindings: {
    name: '@', // one-way binding (string)
    age: '@'
  },
  template:
    '<div class="student">
      <h4>Student Name: {{$ctrl.name}}</h4>
      <p>Age: {{$ctrl.age}}</p>
    </div>'
});
```

Key Points

- component() is a simplified form of directive.
- Used as a custom HTML element.
- bindings are used to pass data.
- \$ctrl refers to the component controller.
- More modular and reusable code structure.

4. Component with Controller & Bindings

```
index.html
<div ng-app="myApp">
  <product-card product="p1"></product-card>
</div>
```

Output :

Laptop
Price: ₹ 55000

```
app.js
var app = angular.module('myApp', []);
app.controller('MainCtrl', function($scope) {
  $scope.p1 = {name: 'Laptop', price: 55000};
});
app.component('productCard', {
  bindings: { product: '<' }, // one-way binding
  template:
    '<div class="product">
      <h4>{{ $ctrl.product.name }}</h4>
      <p>Price: ₹ {{ $ctrl.product.price }}</p>
    </div>'
});
```

Key Points

- Uses one-way binding <.
- Accepts product object from parent.
- Reusable component for displaying product details.
- Controller logic can be added if needed.

Real-life Use Cases

- Creating reusable UI components (header, footer, cards).
- Building custom form inputs (date picker, file upload, etc.).
- Widget-like features (rating, tabs, modals).
- Component-based structure for large applications.
- Improving code reusability and maintainability.

Key Takeaways

- Directives extend HTML with custom behavior.
- Components are a structured, reusable type of directive (1.5+).
- Use restrict to define directive type (A, E, C, M).
- Use bindings in components to pass data.
- Components make applications modular and easier to maintain.

14. Routing & Single Page Applications

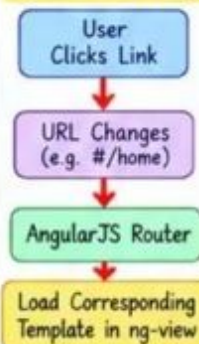
What is Routing in AngularJS ?

- Routing allows us to create multiple views (pages) in a Single Page Application (SPA) without reloading the entire page.
- AngularJS provides routing using `ngRoute` module and `ng-view` directive.
- Based on the URL, different templates and controllers are loaded dynamically.
- It makes the application faster and provides a smooth user experience.

What is Single Page Application (SPA) ?

- SPA is a web application that loads a single HTML page and dynamically updates the content without page reload.
- Navigation between pages happens using routing (URL changes).
- Improves performance and user experience.

How SPA Works ?



- No full page reload
- Only view changes
- Faster and smoother experience
- Uses `ngRoute` and `ng-view`

1. Include ngRoute Module

```

index.html
<html ng-app="myApp">
<head>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-route.min.js"></script>
</head>
<body>
  ...
</body>
</html>
  
```

Why ngRoute ?

- `ngRoute` is a built-in module in AngularJS.
- It provides routing functionality using the `$routeProvider`.

2. Configure Routes (app.js)

```

app.js
var app = angular.module('myApp', ['ngRoute']);
app.config(function($routeProvider) {
  $routeProvider
    .when('/home', {
      templateUrl: 'home.html',
      controller: 'homeCtrl'
    })
    .when('/about', {
      templateUrl: 'about.html',
      controller: 'AboutCtrl'
    })
    .otherwise({
      redirectTo: '/home'
    });
});
  
```

Key Points

- `$routeProvider` is used to define routes.
- `when()` defines a route for a URL.
- `templateUrl` specifies HTML template.
- `controller` specifies the controller.
- `otherwise()` is used for a default route.

3. HTML Structure with ng-view

```

index.html
<body ng-app="myApp">
  <nav>
    <a href="#!/home">Home</a> |
    <a href="#!/about">About</a>
  </nav>
  <div ng-view></div>
</body>
  
```

Explanation

- Links use `#!/` (hashbang) in URL.
- `ng-view` is a placeholder where the template will be loaded based on the route.

4. Create Templates and Controllers

```

home.html
<h2>Welcome to Home Page</h2>
<p>This is Home template.</p>

about.html
<h2>About Us</h2>
<p>This is About template.</p>

app.js (Controllers)
app.controller('HomeCtrl', function($scope) {
  $scope.message = 'Welcome to Home Page!';
});
app.controller('AboutCtrl', function($scope) {
  $scope.message = 'This is About Page!';
});
  
```

5. Output in Browser

http://localhost/index.html

Home | About

Welcome to Home Page

This is Home template.

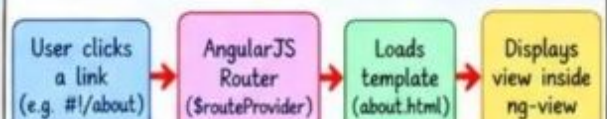
http://localhost/index.html#!/about

Home | About

About Us

This is About template.

6. How Routing Works ?



Key Takeaways

- ✓ Routing helps in building Single Page Applications (SPA).
- ✓ Use `ngRoute` module and `ng-view` directive.
- ✓ `$routeProvider.when()` defines routes.
- ✓ Templates and controllers are loaded dynamically.
- ✓ Improves performance and user experience.
- ✓ Use `otherwise()` for default route.

Real-life Use Cases

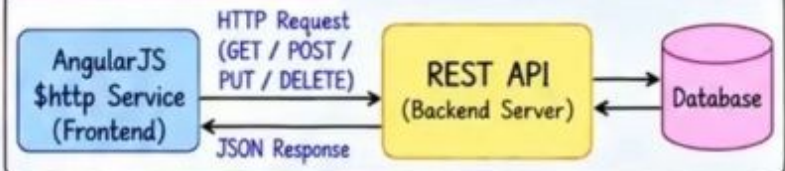
- Dashboard with multiple sections (Home, Profile, Settings).
- Portfolio website with different pages.
- Admin panels with multiple modules.
- E-commerce site (Product List, Product Details, Cart, etc.).
- Any web application where you want fast navigation without page reload.

13. HTTP Service & REST API Integration

What is \$http Service ?

- \$http is a built-in service in AngularJS used to make HTTP requests to a server.
- It is used to consume REST APIs and work with data from backend services.
- It returns a promise, so we can handle success and error using .then() and .catch() (or .error()).
- It supports methods like GET, POST, PUT, DELETE, etc.

How It Works ?



\$http Service Methods

Method	Syntax	Description
GET	<code>\$http.get(url)</code>	Fetch data from server
POST	<code>\$http.post(url, data)</code>	Send data to server
PUT	<code>\$http.put(url, data)</code>	Update existing data
DELETE	<code>\$http.delete(url)</code>	Delete data from server

1. GET Request (Fetch Data)

```

app.js
var app = angular.module('myApp', []);
app.controller('userCtrl', function($scope, $http) {
  $http.get('https://jsonplaceholder.typicode.com/users')
    .then(function(response) {
      $scope.users = response.data;
    })
    .catch(function(error) {
      console.log('Error:', error);
    });
});
  
```

Output (Browser)

User List from API

- Leanne Graham
- Ervin Howell
- Clementine Bauch
- Patricia Lebsack

2. POST Request (Send Data)

```

app.js
$scope.newUser = {
  name: 'Narayan',
  email: 'narayan@gmail.com'
};
$http.post('https://jsonplaceholder.typicode.com/users',
  $scope.newUser)
  .then(function(response) {
    console.log('Data Sent:', response.data);
  })
  .catch(function(error) {
    console.log('Error:', error);
  });
  
```

Output (Console)

Data Sent:

```

{
  "id": 11,
  "name": "Narayan",
  "email": "narayan@gmail.com"
}
  
```

3. PUT Request (Update Data)

```

app.js
var updatedUser = {
  name: 'Narayan Updated',
  email: 'newemail@gmail.com'
};
$http.put('https://jsonplaceholder.typicode.com/users/1',
  updatedUser)
  .then(function(response) {
    console.log('Data Updated:', response.data);
  });
  
```

Output (Console)

Data Updated:

```

{
  "id": 1,
  "name": "Narayan Updated",
  "email": "newemail@gmail.com"
}
  
```

4. DELETE Request (Delete Data)

```

app.js
$http.delete('https://jsonplaceholder.typicode.com/users/1')
  .then(function(response) {
    console.log('User Deleted:', response.data);
  })
  .catch(function(error) {
    console.log('Error:', error);
  });
  
```

Output (Console)

User Deleted:

```

{}

(API returns empty object)
  
```

5. Using \$http with HTML (Display Data)

```

index.html
<div ng-app="myApp" ng-controller="userCtrl">
  <h3>Users List</h3>
  <ul>
    <li ng-repeat="u in users">
      {{ u.name }} - {{ u.email }}
    </li>
  </ul>
</div>
  
```

Output (Browser)

Users List

- Leanne Graham - leanne@example.com
- Ervin Howell - ervin@example.com
- Clementine Bauch - clemetine@example.com

6. Handling API Errors

```

app.js
$http.get('https://invalid-api.com/users')
  .then(function(response) {
    $scope.users = response.data;
  })
  .catch(function(error) {
    $scope.errorMsg = 'Failed to load data. Please try again!';
    console.log('Error:', error);
  });
  
```

Output (Browser)

Failed to load data.
Please try again!

Key Takeaways

- ✓ \$http service is used to make HTTP requests.
- ✓ Supports GET, POST, PUT, DELETE methods.
- ✓ Returns a promise (.then() and .catch()).
- ✓ Used to integrate AngularJS with REST APIs.
- ✓ Response data is available in response.data.
- ✓ We can handle errors using .catch().
- ✓ Useful for real-world web applications.

Real-life Use Cases

- Fetch and display data from a server (e.g., product list, user list).
- Submit form data to backend (e.g., registration).
- Update or delete data (e.g., edit profile, delete user).
- Build CRUD operations in AngularJS.
- Integrate with third-party APIs (e.g., weather API, payment gateway, etc.).

Common APIs for Practice

- JSONPlaceholder (<https://jsonplaceholder.typicode.com>)
- RegRes (<https://reqres.in>)
- FakeStore API (<https://fakestoreapi.com>)
- OpenWeather API (<https://openweathermap.org/api>)

12. Custom Services & Factories

What are Custom Services ?

- Custom services are reusable objects or functions that contain business logic.
- They are used to share data and logic between multiple controllers, directives and components (in AngularJS).
- We can create custom services using `service()`, `factory()`, `provider()` or `value()`.
- Services are singletons, created by AngularJS injector and can be injected where needed.

Service vs Factory vs Other Methods

Method	Returns	Syntax	Use Case
<code>service()</code>	Constructor function (new)	<code>app.service('name', function() { ... })</code>	Simple services with this keyword
<code>factory()</code>	Object (returned manually)	<code>app.factory('name', function() { return (...); })</code>	Most commonly used, flexible
<code>provider()</code>	Provider object (with configure)	<code>app.provider('name', function() { ... })</code>	Advanced configuration
<code>value()</code>	Simple value (string, number, obj)	<code>app.value('name', value)</code>	For constants (simple data)

1. Custom Service using service()

```

app.js
var app = angular.module('myApp', []);
app.service('studentService', function() {
  this.getStudent = function() {
    return { name: 'Narayan', age: 21 };
  };
  this.setStudent = function(name, age) {
    return { name: name, age: age };
  };
});

```

Why service() ?

- Uses a constructor function with this keyword.
- AngularJS creates a new instance (using new).
- Good for simple services.

2. Custom Factory using factory()

```

app.js
var app = angular.module('myApp', []);
app.factory('studentFactory', function() {
  var students = [
    { name: 'Narayan', age: 21 },
    { name: 'Amit', age: 22 }
  ];
  return {
    getStudents: function() {
      return students;
    },
    addStudent: function(student) {
      students.push(student);
    }
  };
});

```

Why factory() ?

- Returns an object explicitly.
- More flexible than service().
- Most commonly used.
- We can return functions, objects, or any data.
- Useful for complex logic.

3. Using Custom Service in a Controller

```

app.js
app.controller('studentCtrl', function($scope, studentService) {
  $scope.student = studentService.getStudent();
  $scope.newStudent = function() {
    $scope.student = studentService.setStudent('Rahul', 25);
  };
});

```

```

index.html
<div ng-app="myApp" ng-controller="studentCtrl">
  <h3>Using Custom Service</h3>
  <p>Name: {{ student.name }}</p>
  <p>Age: {{ student.age }}</p>
  <button ng-click="newStudent()">Change Student
</button>
</div>

```

Output :

Using Custom Service
Name: Narayan
Age: 21

Change Student

Name: Rahul
Age: 25

4. Using Custom Factory in a Controller

```

app.js
app.controller('studentCtrl', function($scope, studentFactory) {
  $scope.students = studentFactory.getStudents();
  $scope.addStudent = function() {
    studentFactory.addStudent({ name: 'Rohit', age: 20 });
    $scope.students = studentFactory.getStudents();
  };
});

```

```

index.html
<div ng-app="myApp" ng-controller="studentCtrl">
  <h3>Using Custom Factory</h3>
  <ul>
    <li ng-repeat="s in students">
      {{ s.name }} - {{ s.age }}
    </li>
  </ul>
  <button ng-click="addStudent()">Add Student</button>
</div>

```

Output :

Using Custom Factory

- Narayan - 21
- Amit - 22

Add Student

- Narayan - 21
- Amit - 22
- Rohit - 20

Real-life Use Cases

- Sharing common data between multiple controllers.
- API calls (using \$http) to fetch data from server.
- Managing user authentication (login/logout).
- Storing application settings or configurations.
- Reusable business logic (validation, calculations, etc.).

Key Takeaways

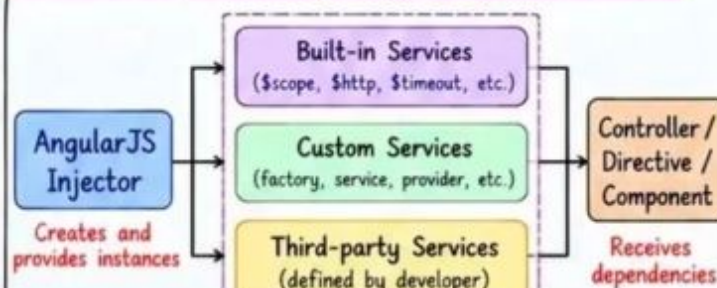
- Custom services are reusable and follow singletons.
- `service()` uses constructor function and this keyword.
- `factory()` returns an object and is more flexible.
- Both can be injected into controllers, directives, etc.
- Use services/factories to keep code modular and clean.
- Helps in building maintainable AngularJS applications.

11. Dependency Injection & Services

What is Dependency Injection ?

- Dependency Injection (DI) is a design pattern where AngularJS provides the required dependencies (like services) to a controller, directive or component.
- We don't create objects using new keyword, AngularJS injects them automatically.
- It makes the code modular, reusable and easy to test.
- AngularJS uses function parameter names for dependency injection.

How Dependency Injection Works ?



Types of Dependency Injection

Type	Syntax	Description
Implicit Injection	function(\$scope, myService)	AngularJS automatically injects based on parameter names.
Explicit Injection (Annotation)	\$inject array	Used when code is minified (to avoid issues with parameter names).
Inline Annotation	\$inject property	Another way for explicit injection.

Creating a Service in AngularJS

```

app.js
var app = angular.module('myApp', []);

// Create a service using service()
app.service('myService', function() {
  this.appName = 'AngularJS Notes';
  this.getName = function() {
    return this.appName;
  };
});

```

Key Points

- Service is a singleton.
- Created using service().
- Uses this keyword to define properties and methods.
- Automatically injected where needed.

Using Service in a Controller

```

app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope, myService) {
  $scope.appName = myService.getName();
});

```

```

index.html
<div ng-app="myApp" ng-controller="myCtrl">
  <h3>App Name: {{ appName }}</h3>
</div>

```

Output :

App Name:
AngularJS Notes

Different Ways to Create a Service

Method	Syntax Example	Use Case
service()	app.service('myService', function() { ... });	Most commonly used.
factory()	app.factory('myService', function() { return { ... }; });	Returns an object.
provider()	app.provider('myService', function() { ... });	Used for advanced configuration.
constant()	app.constant('API_URL', 'https://api.example.com');	Used for constant values.
value()	app.value('appName', 'MyApp');	Used to share simple values.

Example: Using factory()

```

app.js
app.factory('userService', function(){
  var users = [
    { name: 'Narayan', age: 21 },
    { name: 'Amit', age: 22 },
    { name: 'Rohit', age: 20 }
  ];
  return {
    getUsers: function() {
      return users;
    }
  };
});

```

Output :

Student List

- Narayan - 21
- Amit - 22
- Rohit - 20

Using in Controller

```

app.controller('userCtrl',
function($scope, userService) {
  $scope.users = userService.getUsers();
});

```

Explicit Injection (for Minification)

```

app.js
app.controller('myCtrl', ['$scope', 'myService',
function($scope, myService) {
  $scope.appName = myService.getName();
}]);

```

Why use Explicit Injection ?

- When we minify the code, parameter names may change.
- Explicit injection prevents errors by providing dependency names explicitly.

```

index.html
<div ng-app="myApp" ng-controller="userCtrl">
  <h3>Student List</h3>
  <ul>
    <li ng-repeat="u in users">{{ u.name }} - {{ u.age }}</li>
  </ul>
</div>

```

Student List

- Narayan - 21
- Amit - 22
- Rohit - 20

Key Takeaways

- ✓ Dependency Injection provides required services automatically.
- ✓ Services make the application modular and reusable.
- ✓ AngularJS provides multiple ways to create services (service, factory, provider, etc.).
- ✓ Use explicit injection when code is minified.
- ✓ Built-in services like \$scope, \$http, \$timeout can also be injected.

Real-life Use Cases

- Calling REST APIs using \$http service.
- Sharing data between multiple controllers.
- Storing constant values (API URL, configuration).
- Creating reusable utility functions (e.g., date formatter).
- Managing application-wide data (user info, settings).

10. Events & Event Handling

What are Events in AngularJS ?

- Events are user actions like click, change, input, mouseover, etc. that happen on HTML elements.
- AngularJS allows us to handle these events using the `ng-*` directives.
- When an event occurs, an AngularJS expression or function is executed.

Common Event Directives

Directive	Description
<code>ng-click</code>	Executes an expression when element is clicked.
<code>ng-change</code>	Executes when the value of an input field changes.
<code>ng-keyup</code>	Executes when a key is released.
<code>ng-keydown</code>	Executes when a key is pressed.
<code>ng-mouseover</code>	Executes when mouse is moved over an element.
<code>ng-mouseout</code>	Executes when mouse is moved out of an element.
<code>ng-submit</code>	Executes when a form is submitted.

1. ng-click Event (Button Click)

```
index.html
<div ng-app="myApp"
  ng-controller="myCtrl">
  <h3>{{ message }}</h3>
  <button ng-click="showMessage()">
    Click Me
  </button>
</div>
```

```
app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.message = 'Hello AngularJS!';
  $scope.showMessage = function() {
    $scope.message = 'Button Clicked!';
  };
});
```

Output :

Button Clicked!

Click Me

2. ng-change Event (Input Field)

```
index.html
<div ng-app="myApp"
  ng-controller="myCtrl">
  <input type="text"
    ng-model="name"
    ng-change="showName()">
  <p>You typed: {{ name }}</p>
</div>
```

```
app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.name = '';
  $scope.showName = function() {
    console.log('Input changed to: ' + $scope.name);
  };
});
```

Output :

Narayan

You typed: Narayan

3. Keyboard Events (ng-keyup & ng-keydown)

```
index.html
<div ng-app="myApp"
  ng-controller="myCtrl">
  <input type="text" ng-model="text"
    ng-keyup="onKeyUp(text)"
    ng-keydown="onKeyDown(text)"
    placeholder="Type something...">
  <p>Key Down: {{ keyDownMsg }}</p>
  <p>Key Up : {{ keyUpMsg }}</p>
</div>
```

```
app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.keyDownMsg = '';
  $scope.keyUpMsg = '';
  $scope.onKeyDown = function(text) {
    $scope.keyDownMsg = 'Key Down: ' + text;
  };
  $scope.onKeyUp = function(text) {
    $scope.keyUpMsg = 'Key Up: ' + text;
  };
});
```

Output :

AngularJS

Key Down: AngularJ

Key Up: AngularJS

4. Mouse Events (ng-mouseover & ng-mouseout)

```
index.html
<div ng-app="myApp"
  ng-controller="myCtrl">
  <div class="box"
    ng-mouseover="onMouseOver()"
    ng-mouseout="onMouseOut()">
    Hover Me
  </div>
  <p>{{ hoverMsg }}</p>
</div>
```

```
app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.hoverMsg = 'Move your mouse here';
  $scope.onMouseOver = function() {
    $scope.hoverMsg = 'Mouse Over!';
  };
  $scope.onMouseOut = function() {
    $scope.hoverMsg = 'Mouse Out!';
  };
});
```

Output :

Hover Me

Mouse Over!

5. Form Submit Event (ng-submit)

```
index.html
<div ng-app="myApp" ng-controller="myCtrl">
  <form ng-submit="submitForm()">
    <input type="text" ng-model="name" placeholder="Enter name">
    <button type="submit">Submit</button>
  </form>
  <p>{{ formMsg }}</p>
</div>
```

```
app.js
var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.name = '';
  $scope.submitForm = function() {
    if($scope.name) {
      $scope.formMsg = 'Form Submitted!Name: ' + $scope.name;
      $scope.name = '';
    } else {
      $scope.formMsg = 'Please enter a name!';
    }
  };
});
```

Output :

Narayan

Submit

Form Submitted!
Name: Narayan

Key Takeaways

- Events handle user actions in AngularJS.
- Use `ng-*` directives like `ng-click`, `ng-change`, `ng-submit`, etc.
- We can call functions or expressions on a particular event.

Real-life Use Cases

- Handling button clicks (Add, Delete, Save).
- Validating input while typing.
- Form submission with validation.
- Mouse hover effects for menus or tooltips.

9. Forms & Form Validation

What are Forms in AngularJS ?

- Forms in AngularJS are used to collect user input and submit it to the server.
- AngularJS provides built-in directives like **ng-model**, **ng-submit** and form validation.
- We can easily validate input fields using built-in validation properties.
- AngularJS provides **form** and **input** objects to track the form state (valid/invalid, dirty, pristine, etc.).

Form Directives

Directive	Description
ng-form	Defines a form (optional, <form> works too).
ng-model	Binds input field value to a variable.
ng-submit	Specifies a function to call on form submit.
name	Required to access form and input field.
novalidate	Disables browser default validation.

Common Validation Directives

Directive / Attribute	Description
required	Field must not be empty.
ng-minlength	Minimum number of characters.
ng-maxlength	Maximum number of characters.
type="email"	Validates email format.
type="number"	Accepts only numbers.
pattern	Validates input using a regex pattern.

Form State Properties

Property	Description
\$valid	Form is valid.
\$invalid	Form is invalid.
\$dirty	Field value has been changed.
\$pristine	Field value has not been changed.
\$touched	Field has been focused and left.
\$untouched	Field has not been focused yet.

Example: Student Registration Form

```

<div ng-app="myApp" ng-controller="formCtrl">
  <h2>Student Registration</h2>
  <form name="studentForm" ng-submit="submitForm()"
    novalidate>
    <label>Name :</label>
    <input type="text" name="name" ng-model="student.name"
      required>
    <span ng-show="studentForm.name.$touched &&
      studentForm.name.$error.required" class="error">
      Name is required!</span><br><br>
    <label>Email :</label>
    <input type="email" name="email" ng-model="student.email"
      required>
    <span ng-show="studentForm.email.$touched &&
      studentForm.email.$error.email" class="error">
      Enter a valid email!</span><br><br>
    <label>Age :</label>
    <input type="number" name="age" ng-model="student.age"
      min="1" max="100" required>
    <span ng-show="studentForm.age.$touched &&
      studentForm.age.$error.required" class="error">
      Age is required!</span><br><br>
    <button type="submit" ng-disabled="studentForm.$invalid">
      Submit</button>
  </form>
</div>

```

AngularJS Controller (app.js)

```

app.js
var app = angular.module('myApp', []);
app.controller('formCtrl', function($scope) {
  $scope.student = {
    name: '',
    email: '',
    age: null
  };
  $scope.submitForm = function() {
    if ($scope.studentForm.$valid) {
      alert('Form Submitted!\n\nName: ' + $scope.student.name +
        '\nEmail: ' + $scope.student.email +
        '\nAge: ' + $scope.student.age);
    }
  };
});

```

Output in Browser

Student Registration

Name : Narayan

Email : narayan@gmail.com

Age : 21

Submit

Validation Messages

- If name is empty → "Name is required!"
- If email is invalid → "Enter a valid email!"
- If age is empty → "Age is required!"
- Submit button is disabled until the form is valid.

Key Takeaways

- Use **ng-model** to bind input field values.
- Use **ng-submit** to handle form submission.
- AngularJS provides built-in validation directives.
- Form and input objects track the form state.
- We can show custom error messages using **ng-show**.
- Submit button can be disabled using **form.\$invalid**.

Real-life Use Cases

- User registration form with validation.
- Login form with required fields.
- Contact us form with email and message validation.
- Product order form with multiple fields.
- Any web application where user input is required.

8. Filters in AngularJS

What are Filters ?

- Filters in AngularJS are used to format data before displaying it in the view (HTML).
- They are written using the pipe symbol (|).
- Filters can be used with strings, numbers, dates, arrays and objects.
- We can also create custom filters.

Syntax of Using Filter

```
{{ expression | filterName : parameter }}
```

Example :

```
<p>{{ name | uppercase }}</p>
<p>{{ price | currency : '₹' }}</p>
```

Commonly Used Built-in Filters

Filter	Description	Example
uppercase	Converts text to upper case	{{ name uppercase }}
lowercase	Converts text to lower case	{{ name lowercase }}
currency	Formats number as currency	{{ price currency : '₹' }}
number	Formats number with fixed decimal places	{{ amount number : 2 }}
date	Formats date	{{ date date : 'yyyy-MM-dd' }}
json	Converts object to JSON format	{{ user json }}
limitTo	Limits the number of items	{{ items limitTo : 3 }}
orderBy	Sorts an array	{{ users orderBy : 'name' }}
filter	Filters array based on condition	{{ users filter : 'Narayan' }}

1. uppercase & lowercase Filter

```
<p>Original: {{ name }}</p>
<p>Uppercase: {{ name | uppercase }}</p>
<p>Lowercase: {{ name | lowercase }}</p>
```

Output :

```
Original: Narayan
Uppercase: NARAYAN
Lowercase: narayan
```

2. currency & number Filter

```
<p>Price: {{ price | currency : '₹' }}</p>
<p>Amount: {{ amount | number : 2 }}</p>
```

Output :

```
Price: ₹1,250.00
Amount: 1234.57
```

3. date Filter

```
<p>Today: {{ today | date }}</p>
<p>Format 1: {{ today | date : 'dd-MM-yyyy' }}</p>
<p>Format 2: {{ today | date : 'MMM dd, yyyy' }}</p>
```

Output :

```
Today: Sep 26, 2026
Format 1: 26-09-2026
Format 2: Sep 26, 2026
```

4. json Filter

```
<p>Student Object (JSON):</p>
<pre>{{ student | json }}</pre>
```

Output :

```
{
  "name": "Narayan",
  "age": 21,
  "course": "MERN"
}
```

5. limitTo Filter

```
<h3>Student List (First 3)</h3>
<ul>
  <li ng-repeat="s in students | limitTo:3">
    {{ s }}
  </li>
</ul>
```

Output :

```
Student List (First 3)
• Narayan
• Amit
• Rohit
```

6. orderBy Filter

```
<h3>Sorted by Name</h3>
<ul>
  <li ng-repeat="u in users | orderBy:'name'">
    {{ u.name }} - {{ u.age }}
  </li>
</ul>
```

Output :

```
Sorted by Name
• Amit - 20
• Narayan - 21
• Rohit - 23
```

7. filter Filter (Search in Array)

```
<input type="text" ng-model="search"
  placeholder="Search name..." />
<ul>
  <li ng-repeat="u in users | filter:search">
    {{ u.name }} - {{ u.age }}
  </li>
</ul>
```

Output :

```
Narayan
• Narayan - 21
```

Creating a Custom Filter

```
app.filter('reverse', function() {
  return function(text) {
    return text.split('').reverse().join('');
  };
});
```

Using in HTML

```
<p>Original: {{ name }}</p>
<p>Reversed: {{ name | reverse }}</p>
```

Output :

```
Original: Narayan
Reversed: nayaraN
```

Key Takeaways

- ✓ Filters are used to format data in the view.
- ✓ Written using the pipe symbol (|).
- ✓ AngularJS provides many built-in filters.
- ✓ Useful filters: uppercase, lowercase, currency, date, number, json, limitTo, orderBy, filter.
- ✓ We can also create custom filters.

7. Scope & RootScope

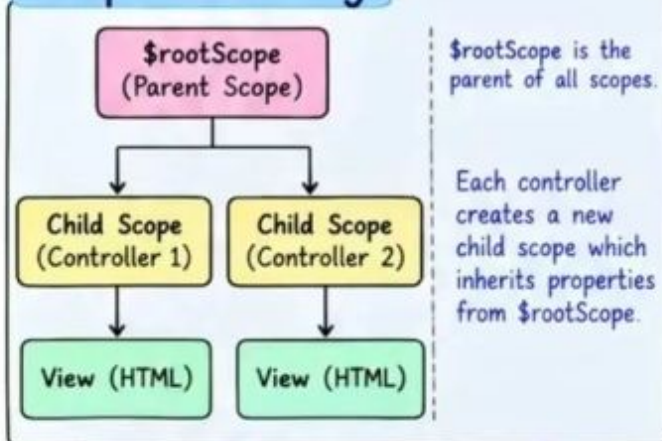
What is Scope ?

- Scope is an object that acts as a bridge between the controller and the view (HTML).
- It contains the application data (variables), methods (functions) and allows data binding with the view.
- Whenever the data in scope changes, the view is automatically updated.
- Each controller has its own scope object.

What is \$rootScope ?

- \$rootScope is the top-level scope of an AngularJS application.
- It is the parent of all other scopes.
- Data added to \$rootScope is available throughout the entire application (in all controllers, services, etc.).
- It is useful for sharing data between different controllers.

Scope Hierarchy



Example 1: Using \$scope (Controller Scope)

index.html

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <script src="angular.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <div ng-controller="myCtrl">
    <h2>Hello {{ name }}!</h2>
    <p>Age : {{ age }}</p>
    <button ng-click="changeName()">
      Change Name
    </button>
  </div>
</body>
</html>

```

app.js

```

var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.name = 'Narayan';
  $scope.age = 21;
  $scope.changeName = function() {
    $scope.name = 'AngularJS';
  };
});

```

Output :

Hello Narayan!
Age : 21
Change Name

→

Hello AngularJS!
Age : 21
Change Name

Scope Inheritance

- Child scopes inherit properties and methods from their parent scope.
- If a property is not found in the child scope, AngularJS looks for it in the parent scope (\$rootScope).
- Changes in child scope do not affect parent scope (for primitives).
- Objects and arrays are shared between parent and child scope (changes are reflected).

Example 2: Using \$rootScope (Global Data)

app.js

```

var app = angular.module('myApp', []);
app.run(function($rootScope) {
  $rootScope.appName = 'My AngularJS App';
});
app.controller('firstCtrl', function($scope) {
  $scope.message = 'Controller 1';
});
app.controller('secondCtrl', function($scope) {
  $scope.message = 'Controller 2';
});

```

index.html

```

<div ng-controller="firstCtrl">
  <h3>{{ appName }}</h3>
  <p>{{ message }}</p>
</div>

<div ng-controller="secondCtrl">
  <h3>{{ appName }}</h3>
  <p>{{ message }}</p>
</div>

```

Output :

My AngularJS App
Controller 1

My AngularJS App
Controller 2

Scope Types

Type	Created By	Description
\$rootScope	AngularJS (automatically)	Top-level scope, parent of all scopes.
Scope (Controller Scope)	Each controller	Child scope, used to share data with its view.
Isolated Scope	Directive (using scope: {})	Separate scope, does not inherit from parent scope.

Key Takeaways

- ✓ \$scope is used in controllers to share data with view.
- ✓ \$rootScope is the parent of all scopes and is global.
- ✓ Child scopes inherit properties from parent scope.
- ✓ Use \$rootScope carefully (avoid overuse).
- ✓ Objects and arrays are shared between scopes.
- ✓ Scope helps in two-way data binding in AngularJS.

Conclusion

- Scope is the heart of AngularJS for data binding.
- Each controller has its own scope, while \$rootScope is the global scope.
- Understanding scope helps to manage data efficiently in AngularJS applications and build scalable projects.

6. Directives in AngularJS

What is a Directive ?

- A directive is a special attribute or element in AngularJS that **extends HTML** with new functionality.
- Directives allow us to create custom HTML elements, attributes and behaviors.
- AngularJS has built-in directives and we can also create custom directives.
- Directives are prefixed with **ng-**.

Types of Directives

Type	Syntax / Example	Description
Element	<directive-name>	Used as a custom HTML element.
Attribute	<div ng-directive>	Used as an attribute in an HTML element.
Class	<div class="ng-directive">	Used as a CSS class in an HTML element.
Comment	<!-- directive: name -->	Used as a comment in HTML.

Built-in Directives (Commonly Used)

Directive	Description	Example
ng-app	Defines an AngularJS application.	<div ng-app>
ng-controller	Defines a controller for the view.	<div ng-controller>
ng-model	Binds input field value.	<input ng-model>
ng-bind	Displays data in HTML (without {{ }}).	<p ng-bind="name">
ng-if	Conditionally includes element in DOM.	<div ng-if="condition">
ng-show	Shows element (CSS based).	<div ng-show="condition">
ng-hide	Hides element (CSS based).	<div ng-hide="condition">
ng-repeat	Repeats element for collections.	<div ng-repeat="item in items">
ng-click	Handles click events.	<button ng-click="function()">

Element Directive Example

```

index.html
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <script src="angular.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <greet></greet> <!-- Custom element directive -->
</body>
</html>

```

app.js (Custom Element Directive)

```

var app = angular.module('myApp', []);
app.directive('greet', function() {
  return {
    template: '<h2>Hello from Custom Element Directive!</h2>'
  };
});

```

Output :

Hello from
Custom Element
Directive!

Attribute Directive Example

```

index.html
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <script src="angular.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <div my-color>AngularJS Attribute Directive</div>
</body>
</html>

```

app.js (Custom Attribute Directive)

```

var app = angular.module('myApp', []);
app.directive('myColor', function() {
  return {
    link: function(scope, element) {
      element.css('color', 'blue');
      element.css('font-size', '20px');
    }
  };
});

```

Output :

AngularJS
Attribute
Directive

Using ng-repeat Directive

```

index.html
<div ng-app="myApp" ng-controller="myCtrl">
  <h3>Student List</h3>
  <ul>
    <li ng-repeat="student in students">
      {{ student.name }} - {{ student.age }}
    </li>
  </ul>
</div>

```

app.js

```

var app = angular.module('myApp', []);
app.controller('myCtrl', function($scope) {
  $scope.students = [
    { name: 'Narayan', age: 21 },
    { name: 'Amit', age: 22 },
    { name: 'Rohit', age: 20 }
  ];
});

```

Output :

Student List

- Narayan - 21
- Amit - 22
- Rohit - 20

Key Points

- ✓ Directives extend HTML with new functionality.
- ✓ AngularJS has built-in and custom directives.
- ✓ Types: Element, Attribute, Class, Comment.
- ✓ ng- directives are commonly used.
- ✓ Custom directives are useful for reusable components.
- ✓ Directives help to make the application more dynamic and interactive.

Conclusion

Directives are one of the most powerful features in AngularJS. They allow us to create custom HTML elements and attributes, and extend the functionality of HTML. Using directives, we can build reusable components and make our AngularJS applications more dynamic and interactive.

5. Expressions & Data Binding

What are Expressions ?

- AngularJS expressions are written inside double curly braces `{{ }}`.
- They are used to bind data from the controller (or `$scope`) to the view (HTML).
- Expressions can contain variables, arithmetic operations and JavaScript expressions.
- AngularJS automatically evaluates the expression and displays the result.

What is Data Binding ?

- Data binding is a mechanism to automatically synchronize data between the model (`$scope`) and the view (HTML).
- When the data in the model changes, the view is updated automatically and vice versa.
- AngularJS uses two-way data binding by default.

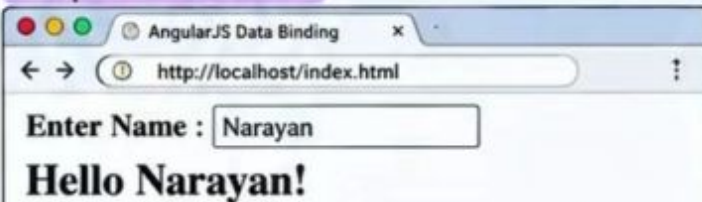
Two-Way Data Binding (ng-model)

- The `ng-model` directive binds the value of an input field to a variable in the controller.
- Any change in the input field is immediately reflected in the variable and the view.

Example:

```
<div ng-app="myApp" ng-controller="myCtrl">
  <p>Enter Name : <input type="text" ng-model="name"></p>
  <p>Hello {{ name }}!</p>
</div>
<script>
  var app = angular.module('myApp', []);
  app.controller('myCtrl', function($scope) {
    $scope.name = 'Narayan';
  });
</script>
```

Output in Browser:



ng-bind-html (Bind HTML Content)

- Used to bind HTML content from a variable.
- It renders the HTML instead of showing it as text.

Example:

```
<div ng-app="myApp" ng-controller="myCtrl">
  <div ng-bind-html="htmlContent"></div>
</div>
<script>
  var app = angular.module('myApp', []);
  app.controller('myCtrl', function($scope) {
    $scope.htmlContent = '<b>This is Bold Text</b><br><i>This is Italic Text</i>';
  });
</script>
```

Output:

This is Bold Text
This is Italic Text

Examples of Expressions

```
<div ng-controller="myCtrl">
  <p>Student Name : {{ name }}</p>
  <p>Age : {{ age }}</p>
  <p>5 + 3 = {{ 5 + 3 }}</p>
  <p>Uppercase : {{ name.toUpperCase() }}</p>
</div>
```

Output :

Student Name : Narayan
Age : 21
5 + 3 = 8
Uppercase : NARAYAN

Types of Data Binding in AngularJS

Type	Syntax	Description
Interpolation	<code>{{ expression }}</code>	Displays data in HTML (view ← model)
ng-model	<code>ng-model="variable"</code>	Two-way data binding (view ↔ model)
ng-bind	<code>ng-bind="variable"</code>	One-way binding (view ← model)
ng-bind-html	<code>ng-bind-html="html"</code>	Binds HTML content (view ← model)

One-Way Binding (ng-bind)

- `ng-bind` directive is used to bind data to HTML without using double curly braces.
- It is useful to prevent flickering of data.

Example:

```
<div ng-app="myApp" ng-controller="myCtrl">
  <p>ng-bind="message"></p>
</div>
<script>
  var app = angular.module('myApp', []);
  app.controller('myCtrl', function($scope) {
    $scope.message = 'Welcome to AngularJS!';
  });
</script>
```

Output:

Welcome to AngularJS!

Key Takeaways

- Expressions use `{{ }}` to display data.
- Data binding keeps model and view in sync.
- `ng-model` provides two-way binding.
- `ng-bind` provides one-way binding.
- `ng-bind-html` is used to bind HTML content.
- AngularJS automatically updates the view when data changes.

Real-life Use Cases

- Displaying user information on a web page.
- Creating forms with real-time updates.
- Showing dynamic content like product lists.
- Building interactive web applications.

4. AngularJS Templates, Views & ng-template

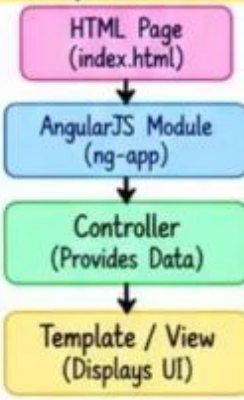
What is Template & View ?

- A template is an HTML file or part of HTML that contains the view (UI) of the application.
- In AngularJS, a template is used with a controller to display data and handle user interaction.
- The template can be a separate HTML file or inline inside the main HTML.
- Views help to create a Single Page Application (SPA) by loading different templates dynamically without reloading the page.

Important Directives

Directive	Purpose
ng-view	Placeholder to load a template dynamically (using routing).
ng-template	Defines an HTML template (not displayed directly).
ng-include	Includes an external HTML file in the current view.
ng-controller	Connects the template with a controller.

Template Flow



1. Basic Template Example (Inline)

```

index.html
<!DOCTYPE html>
<html ng-app="myApp">
<head>
<title>Template Example</title>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
</head>
<body>
<div ng-controller="MyCtrl">
<h2>{{ title }}</h2>
<p>{{ message }}</p>
</div>
<script>
var app = angular.module('myApp', []);
app.controller('MyCtrl', function($scope) {
$scope.title = 'AngularJS Template Example';
$scope.message = 'This is an inline template.';
});
</script>
</body>
</html>
  
```

Output (Browser)

AngularJS
Template Example

This is an inline template.

Key Points

- Template is defined in the same HTML file.
- Controller provides the data.
- {{ }} is used for data binding in template.
- Simple and easy to use for small applications.

2. Using ng-template (Hidden Template)

```

index.html
<div ng-app="myApp" ng-controller="MyCtrl">
<button ng-click="show = !show">
Toggle Template
</button>
<div ng-show="show">
<div ng-template="myTemplate">
<p>This is content inside ng-template.</p>
<p>It will be shown when show is true.</p>
</div>
<div ng-include="myTemplate"></div>
</div>
<script>
var app = angular.module('myApp', []);
app.controller('MyCtrl', function($scope) {
$scope.show = false;
});
</script>
</div>
  
```

Output (Browser)

Toggle Template

This is content inside ng-template.
It will be shown when show is true.

Key Points

- ng-template is used to define a template.
- It is not displayed directly.
- Can be included using ng-include.
- Useful for dynamic and reusable templates.

3. Using ng-include (External Template File)

```

index.html
<div ng-app="myApp" ng-controller="MainCtrl">
<h2>Main Page</h2>
<div ng-include="user.html"></div>
</div>
  
```

Output (Browser)

Main Page

User Information
Name: Narayan
Age: 21

Key Points

- ng-include is used to include an external HTML file.
- Helps to keep code modular and clean.
- Useful for large applications.
- The included file has access to the same scope by default.

```

user.html
<div>
<h3>User Information</h3>
<p>Name: {{ name }}</p>
<p>Age: {{ age }}</p>
</div>
  
```

```

app.js
var app = angular.module('myApp', []);
app.controller('MainCtrl', function($scope) {
$scope.name = 'Narayan';
$scope.age = 21;
});
  
```

4. Using ng-view with Routing (SPA)

```

index.html
<html ng-app="myApp">
<head>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular-route.min.js"></script>
</head>
<body>
<nav>
<a href="#/home">Home</a> |
<a href="#/about">About</a>
</nav>
<div ng-view></div>
<script src="app.js"></script>
</body>
</html>
  
```

```

app.js (Routing)
var app = angular.module('myApp', ['ngRoute']);
app.config(function($routeProvider) {
$routeProvider
.when('/home', {
templateUrl: 'home.html',
controller: 'HomeCtrl'
})
.when('/about', {
templateUrl: 'about.html',
controller: 'AboutCtrl'
})
.otherwise({ redirectTo: '/home' });
});
  
```

Key Takeaways

- Templates define the UI (view) of the application.
- ng-template is used for hidden and reusable templates.
- ng-include helps to include external HTML files.
- ng-view is used with routing to create Single Page Applications (SPAs).
- Templates work with controllers to display dynamic data.

Real-life Use Cases

- Building Single Page Applications (SPAs).
- Loading different views without page reload.
- Reusable templates for header, footer, sidebar.
- Modular and maintainable code structure.
- Dynamic content loading based on user actions.

3. Modules & Controllers

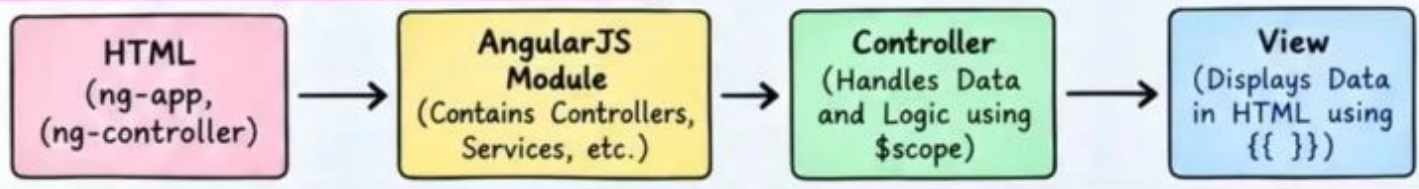
What is a Module ?

- A module is a container for different parts of an AngularJS application like controllers, directives, services, etc.
- It is created using `angular.module()`.
- Every AngularJS application must have at least one module.
- We use `ng-app` directive to connect the module with HTML.

What is a Controller ?

- A controller is a JavaScript function that controls the data and behavior of a view.
- It is defined inside a module using `.controller()`.
- Controller is used to set values, functions and handle user interactions.

How Module and Controller Work ?



Complete Example

index.html

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>AngularJS Module & Controller</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
  <script src="app.js"></script>
</head>
<body>
  <div ng-controller="myController">
    <h2>Hello {{ name }}!</h2>
    <p>{{ message }}</p>
    <button ng-click="showMessage()">Click Me</button>
    <p>{{ result }}</p>
  </div>
</body>
</html>
  
```

app.js

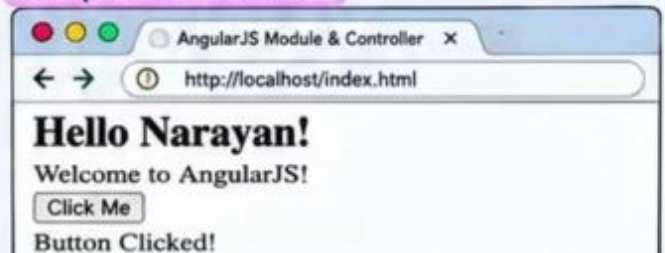
```

var app = angular.module('myApp', []);

app.controller('myController', function($scope) {
  $scope.name = 'Narayan';
  $scope.message = 'Welcome to AngularJS!';

  $scope.showMessage = function() {
    $scope.result = 'Button Clicked!';
  };
});
  
```

Output in Browser



Key Points

- ✓ A module is created using `angular.module()`.
- ✓ A controller is created using `.controller()`.
- ✓ `ng-app` connects the module with HTML.
- ✓ Controller uses `$scope` to share data with the view.
- ✓ We can have multiple controllers in a module.

Important Notes

- Module name should be unique.
- Controller name can be anything (follow meaningful names).
- `$scope` is used to bind data between controller and view.
- We can use multiple controllers in the same module.
- Controller is executed when the `ng-controller` directive is loaded.

2. AngularJS Setup & First Application

Prerequisites

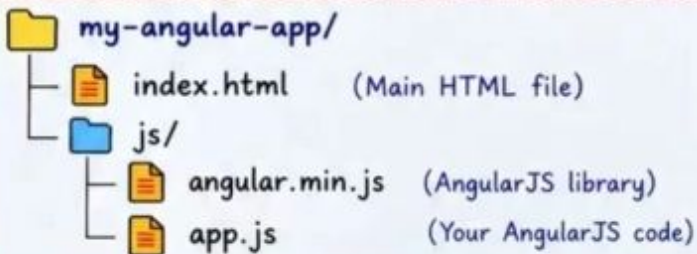
- ✓ Basic knowledge of HTML
- ✓ Basic knowledge of JavaScript
- ✓ A code editor (e.g. VS Code)
- ✓ A web browser (e.g. Chrome)
- ✓ Internet connection (to use CDN)

2. Download and Use Locally

- Go to <https://angularjs.org>
- Download the latest 1.8.3 version.
- Add the angular.min.js file to your project folder.
- Include it in your HTML file:

```
<script src="js/angular.min.js"></script>
```

Project Structure



Example: app.js

```
// Define AngularJS module and controller
var app = angular.module('myApp', []);

app.controller('myController', function($scope) {
  // Define a variable in $scope
  $scope.name = 'Narayan';
});
```

How to Include AngularJS ?

- We can include AngularJS in two ways:
 - ① Using CDN (Recommended for learning)
 - ② Download and Use Locally (for offline use)

1. Using CDN (Recommended)

- Add the AngularJS library link in **<head>** section.
- CDN Link (AngularJS 1.8.3 - last stable version):

```
<script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
```

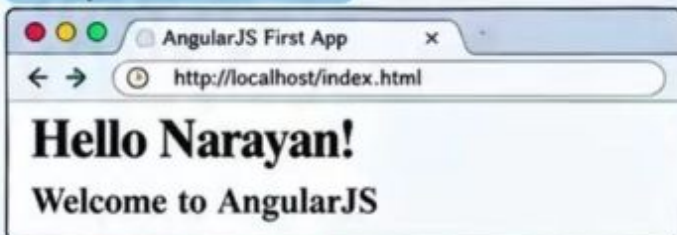
First AngularJS Application

Let's create a simple AngularJS application that displays a message.

Example: index.html

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <title>AngularJS First App</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
  <script src="js/app.js"></script>
</head>
<body>
  <div ng-controller="myController">
    <h1>Hello {{ name }}!</h1>
    <p>Welcome to AngularJS</p>
  </div>
</body>
</html>
```

Output in Browser



How It Works ?

- ① `ng-app="myApp"` → Initializes the AngularJS application.
- ② `ng-controller="myController"` → Defines the controller.
- ③ `{{ name }}` → AngularJS expression to display data from `$scope`.
- ④ `$scope.name` → Binds the JavaScript variable to the view using two-way binding.

Key Takeaways

- ✓ AngularJS can be included using CDN or locally.
- ✓ We create a module using `angular.module()`.
- ✓ Controllers are used to handle application logic.
- ✓ Data is displayed in HTML using `{{ }}` (expression).
- ✓ It is a simple and powerful way to build dynamic web pages.

Next Steps

- Learn more about Modules and Controllers in detail.
- Understand Scope and Data Binding.
- Explore Directives, Filters and Services.
- Build more examples and small projects.

1. AngularJS Introduction & Architecture

What is AngularJS ?

- AngularJS is a JavaScript-based open source front-end framework developed by **Google**.
- It is used to build dynamic and interactive web applications (Single Page Applications - SPA).
- AngularJS extends HTML with additional attributes (directives) and binds data to HTML using **two-way data binding**.

Key Features

- Developed by : Google
- First released : 2010
- Current version : 1.8.3 (last stable)
- Language : JavaScript
- Type : Front-end Framework
- Architecture : MVC (Model-View-Controller)
- Data Binding : Two-way binding
- Directives : Extend HTML functionality
- Runs on : Web Browser
- License : MIT (Open Source)

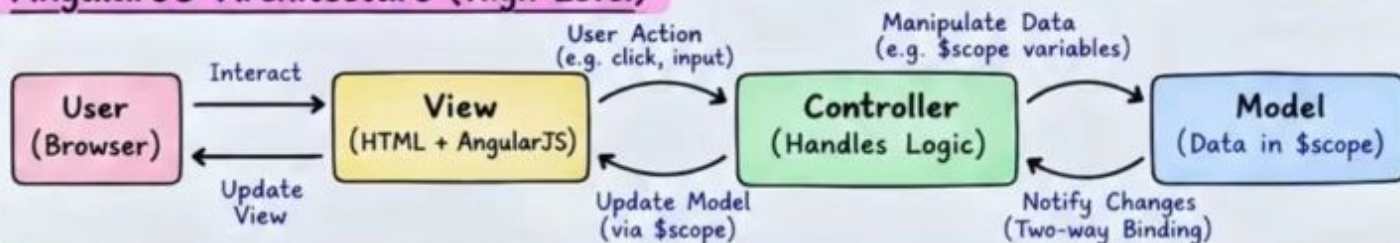
Why Learn AngularJS ?

- ✓ Popular and widely used framework.
- ✓ Helps to build Single Page Applications (SPA).
- ✓ Easy to learn and use.
- ✓ Two-way data binding simplifies development.
- ✓ Good for learning core concepts of modern frameworks (like Angular, React).
- ✓ Large community support and many job opportunities.

Where is AngularJS Used ?

- Single Page Web Applications (SPA).
- Dynamic and interactive web pages
- Enterprise-level web applications
- Admin dashboards
- Real-time web applications
- Used in many popular platforms like YouTube (older version), PayPal (older version), etc.

AngularJS Architecture (High Level)



Key Components

Component	Description
View	HTML with AngularJS directives and bindings.
Model	Data stored in \$scope (JavaScript objects).
Controller	Contains application logic and manipulates data.
Scope (\$scope)	Binds model and view together.
Directive	Extends HTML with new attributes (e.g. ng-app, ng-model).
Service	Provides reusable functionality (e.g. \$http, \$timeout).
Filter	Formats data before displaying (e.g. uppercase, date).

Advantages of AngularJS

- ✓ Easy and fast development
- ✓ Two-way data binding
- ✓ Code reusability with directives and services
- ✓ Reduces the need for manual DOM manipulation
- ✓ Large community and good documentation

Conclusion

AngularJS is a powerful and beginner-friendly framework to build dynamic web applications. It helps in understanding

Simple Example

```

<!DOCTYPE html>
<html ng-app="myApp">
<head>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.3/angular.min.js"></script>
</head>
<body>
  <div ng-controller="myController">
    <h2>Hello {{ name }}!</h2>
  </div>
  <script>
    var app = angular.module('myApp', []);
    app.controller('myController', function($scope) {
      $scope.name = "Narayan";
    });
  </script>
</body>
</html>
  
```

Current Status / Version

- AngularJS is no longer actively developed.
- The last stable version is 1.8.3 (released in 2020).
- It is still used in many legacy applications.
- For new projects, **Angular (2+)** is recommended.

Learn
Practice
Build
Grow

@code.withnarayan

Better
Code
Brighter
Future

AngularJS

COMPLETE HANDWRITTEN NOTES

— Zero to Hero —

- ✓ Easy to Understand
- ✓ Concepts with Examples
- ✓ Diagrams & Visuals
- ✓ Interview Questions
- ✓ Quick Revision
- ✓ Beginner to Advanced



"Small
Steps
Big
Progress"



Modules
Controllers
Directives
Data Binding
Services
Routing
Forms & Validation
HTTP & REST API
Real Projects
= Better You



Write
Test
Debug
Improve

One Framework
Endless Possibilities



10. AUTHENTICATION & AUTHORIZATION

Authentication verifies who you are. Authorization checks what you can do.

★ Authentication Flow



★ JWT (JSON Web Token)

- Compact, URL-safe token
- Contains claims about user
- Signed using secret key
- Consists of 3 parts:

Header.Payload.Signature

Header = { "alg": "HS256", "typ": "JWT" }

Payload = { "id": 1, "name": "Om", "role": "user" }

Signature = HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload), secret)

★ Protecting Routes (Express.js)

```
// Middleware to verify JWT
const jwt = require('jsonwebtoken');
function authenticate(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.status(401).json({
    message: 'Access Denied' });
  try {
    const decoded = jwt.verify(token, 'secretkey');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(403).json({ message: 'Invalid Token' });
  }
}
```

Use this middleware on protected routes.

★ Role Based Access Control (RBAC)

Role	Permissions
Admin	Create, Read, Update, Delete
Editor	Create, Read, Update
User	Read
Guest	Read (Limited)



Always assign minimum required permissions to roles.

★ Password Best Practices

- ✓ Use strong passwords
- ✓ Hash passwords (bcrypt)
- ✓ Never store plain passwords
- ✓ Add salt for extra security

```
const bcrypt = require('bcrypt');
// Hash Password
const hash = await bcrypt.hash(password, 10);
// Compare Password
const match = await bcrypt.compare(
password, hash);
```

★ Authorization Example

```
function authorize(roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        message: "Not Allowed"
      });
    }
    next();
  }
}
```

Use after authentication middleware.
Example: `app.get('/admin', authenticate, authorize(['admin']));`

★ OAuth 2.0 (Social Login)

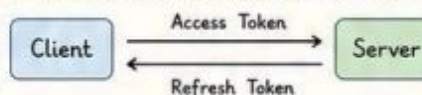
Allows users to login using:



Flow: User → Provider → Your App

★ Refresh Token Flow

1. User logs in → Access Token + Refresh Token
2. Access Token expires
3. Use Refresh Token to get new Access Token
4. Keep Refresh Token secure (HttpOnly Cookie)



★ Common Status Codes

- 200 OK
- 401 Unauthorized (Login Required)
- 403 Forbidden (Not Allowed)
- 404 Not Found
- 500 Internal Server Error

★ Security Tips



Always use HTTPS



Store secrets in .env file



Validate & sanitize user input



Limit login attempts



Use HttpOnly cookies



Keep dependencies updated



Remember :

- ✓ Authenticate first, then authorize.
- ✓ Never trust user input.
- ✓ Tokens make apps stateless & scalable.
- ✓ Security is an important part of every app.

Example in Real Life :

- Instagram → Login & See Your Posts
- YouTube → Login & Upload Videos
- Amazon → Login & Place Orders
- Bank Apps → Secure Access to Accounts



9. APIs, GIT & DEPLOYMENT

APIs connect your app with the world. Git helps in version control.

Deployment makes your project live for everyone.

★ REST API Basics

- REST = Representational State Transfer
- Uses HTTP methods.
- Data in JSON format.
- Stateless communication.
- Common methods:
 - GET (Read)
 - POST (Create)
 - PUT / PATCH (Update)
 - DELETE (Delete)

★ Example API Call (Fetch)

```
fetch('https://api.example.com/data')
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log(err));
```

Always handle errors using .catch()

★ JSON Example

```
{
  "id": 1,
  "name": "Om",
  "email": "om@example.com",
  "skills": ["HTML", "CSS", "JS"],
  "active": true
}
```

JSON = Lightweight data format

★ Git Basics

- Git is a distributed version control system.
- Tracks changes in code.
- Works locally and in cloud (GitHub, GitLab).



★ Important Git Commands

Command	Description
git init	Initialize new repo
git clone <url>	Clone a repository
git status	Check status
git add .	Stage all changes
git commit -m "msg"	Commit changes
git push origin main	Push to remote repo
git pull origin main	Pull latest changes
git branch	List all branches
git checkout <branch>	Switch branch
git merge <branch>	Merge branches

★ Git Workflow

1. Modify Files
2. git add .
3. git commit -m "msg"
4. git push origin main

Commit small changes regularly with meaningful messages.

★ Deployment Options

Frontend (Static)	Backend (API)	Database	Full Stack
- Netlify - Vercel - GitHub Pages	- Render - Railway - Heroku	- MongoDB Atlas (NoSQL) - PlanetScale (MySQL) - Supabase (PostgreSQL)	- AWS - DigitalOcean - Google Cloud

★ Deployment Steps (General)

1. Build your project (if required)
2. Push code to GitHub
3. Connect repo to hosting platform
4. Set environment variables
5. Deploy
6. Share your live URL

★ Project Folder Structure (MERN Stack Example)

```
my-fullstack-app/
├── frontend/ (React)
│   ├── public/
│   ├── src/
│   └── package.json
├── backend/ (Node.js + Express)
│   ├── config/
│   ├── models/
│   ├── routes/
│   ├── controllers/
│   ├── server.js
│   ├── .env
│   └── package.json
└── README.md
```

Keep frontend and backend separate for better structure.



★ Security Best Practices

- ✓ Always use HTTPS
- ✓ Store secrets in .env file
- ✓ Never expose API keys
- ✓ Validate user input
- ✓ Use JWT for authentication
- ✓ Keep dependencies updated



Remember :

- APIs connect frontend with backend.
- Git helps you track and manage code.

Example in Real Life :

Instagram → APIs for data
GitHub → Git for collaboration





8. MONGODB

MongoDB is a NoSQL database that stores data in flexible, JSON-like documents.

★ Key Concepts

- Database → Container for collections
- Collection → Group of documents
- Document → JSON-like data
- Field → Key-value pair in document
- ObjectId → Unique ID for documents

★ Basic Commands

Command	Description
<code>show dbs</code>	Show all databases
<code>use <db_name></code>	Switch to a database
<code>show collections</code>	Show all collections
<code>db</code>	Show current database
<code>db.dropDatabase()</code>	Delete current database

★ CRUD Operations

Collection: `users`

- Create → `db.users.insertOne({...})`
- Read → `db.users.find({...})`
- Update → `db.users.updateOne(filter, {...})`
- Delete → `db.users.deleteOne({...})`

★ Insert Documents

```
// Insert One Document
db.users.insertOne({
  name: "Om",
  age: 22,
  city: "Nagpur"
});

// Insert Many Documents
db.users.insertMany([
  { name: "Aman", age: 21 },
  { name: "Riya", age: 20 }
]);
```

★ Find Documents

```
// Find All
db.users.find();

// Find with Condition
db.users.find({ age: { $gt: 20 } });

// Find One
db.users.findOne({ name: "Om" });

// Select Specific Fields
db.users.find({}, { name: 1, age: 1, _id: 0 });

// Sort Results
db.users.find().sort({ age: -1 });
```

★ Update Documents

```
// Update One
db.users.updateOne(
  { name: "Om" },
  { $set: { age: 23 } }
);

// Update Many
db.users.updateMany(
  { city: "Nagpur" },
  { $set: { active: true } }
);
```

★ Delete Documents

```
// Delete One
db.users.deleteOne({ name: "Riya" });

// Delete Many
db.users.deleteMany({ age: { $lt: 20 } });

// Delete All Documents
db.users.deleteMany({});
```

★ Operators

Operator	Description
<code>\$eq</code>	Equal
<code>\$ne</code>	Not Equal
<code>\$gt</code>	Greater Than
<code>\$gte</code>	Greater Than or Equal
<code>\$lt</code>	Less Than
<code>\$lte</code>	Less Than or Equal
<code>\$in</code>	In Array
<code>\$nin</code>	Not In Array
<code>\$and</code>	Logical AND
<code>\$or</code>	Logical OR

★ Logical Operators (Examples)

```
// $and
db.users.find({ $and: [ { age: { $gt: 20 } },
  { city: "Nagpur" } ] });

// $or
db.users.find({ $or: [ { age: { $lt: 20 } },
  { city: "Pune" } ] });

// $in
db.users.find({ city: { $in: ["Nagpur", "Pune"] } });
```

★ Aggregation (Basic)

```
db.orders.aggregate([
  { $match: { status: "Completed" } },
  { $group: { _id: "$userId", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } }
]);
```

★ Data Types

- String → "Hello"
- Integer → 42
- Double → 3.14
- Boolean → true / false
- Array → [1, 2, 3]
- Object → { key: "value" }
- Date → `ISODate("2024-01-01T00:00:00Z")`
- ObjectId → Automatically generated unique id

Example Document

```
{
  "_id": ObjectId("64f8a1b2c3d4e5f6g7h8i9j"),
  "name": "Om",
  "age": 22,
  "city": "Nagpur",
  "skills": ["JavaScript", "Node.js"],
  "active": true,
  "createdAt": ISODate("2024-05-12T10:30:00Z")
}
```



Remember :

- ✓ MongoDB stores data in JSON-like documents.
- ✓ Use `find()` to read data and queries.
- ✓ Always use proper indexes in production.
- ✓ Keep your schema flexible but consistent.

Example in Real Life :

MongoDB is used in many real-world apps like:



Pinterest



Instagram



Uber



E-commerce



Gaming



Chat Apps



7. NODE.JS & EXPRESS.JS

Node.js is a JavaScript runtime built on Chrome's V8 engine.

Express.js is a minimal and flexible Node.js web application framework.

★ What is Node.js ?

- JavaScript runtime environment.
- Event-driven, non-blocking I/O model.
- Built-in modules (fs, http, path, etc.)
- NPM - Node Package Manager.

★ Creating a Server (HTTP Module)

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello from Node.js');
});
server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

★ Common Built-in Modules

Module	Description
fs	File system module
http	Create HTTP server
path	Utilities for file paths
url	URL utilities
os	Operating system info
events	Event handling
util	Utility functions

★ NPM Commands

```
npm init      → Initialize a new project
npm install <package> → Install a package
npm install   → Install from package.json
npm uninstall <package> → Uninstall a package
npm start     → Run the project
```

★ What is Express.js ?

- Web framework for Node.js
- Simplifies routing and middleware
- Helps in building APIs quickly
- Minimal and unopinionated

★ Hello World in Express

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello Express!');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

★ Routing in Express

```
app.get('/', (req, res) => res.send('Home Page'));
app.get('/about', (req, res) => res.send('About Page'));
app.post('/login', (req, res) => res.send('Login Page'));
app.put('/update', (req, res) => res.send('Update Page'));
app.delete('/delete', (req, res) => res.send('Delete Page'));
```

★ Route Parameters

```
app.get('/user/:id', (req, res) => {
  const id = req.params.id;
  res.send('User ID: ' + id);
});
```

Example URL
/user/123
↓
req.params.id = 123

★ Query Parameters

```
app.get('/search', (req, res) => {
  const q = req.query.q;
  res.send('Search: ' + q);
});
```

Example URL
/search?q=nodejs
↓
req.query.q = nodejs

★ Middleware

```
app.use((req, res, next) => {
  console.log('Request Time:', Date.now());
  next(); // Pass control to next middleware
});
```

Middleware is a function that has access to req, res, and next middleware.

★ Serving Static Files

```
app.use(express.static('public'));
```

Serves all files inside 'public' folder as static assets.

★ Common Response Methods

Method	Description
res.send()	Send response of any type
res.json()	Send JSON response
res.status()	Set status code
res.redirect()	Redirect to another URL

★ Body Parsing (JSON)

```
const express = require('express');
const app = express();
app.use(express.json()); // Parse JSON body

app.post('/api/data', (req, res) => {
  console.log(req.body); // Access JSON data
  res.send('Data received');
});
```

Allows us to read JSON data from request body.



Remember :

- ☑ Node.js runs JS on the server.
- ☑ NPM helps in managing packages.
- ☑ Express.js makes routing & APIs easy.
- ☑ Middleware = Functions that run before the route.

Example in Real Life :

Used in real-world applications like:



6. REACT JS

React is a JavaScript library for building user interfaces, especially single-page applications.

★ What is React ?

- Library for building UI.
- Component-based architecture.
- Uses Virtual DOM for performance.
- Declarative and efficient.
- Maintained by Meta (Facebook).

★ Key Features

- Component Based
- JSX Syntax
- Virtual DOM
- One-way Data Binding
- Reusable Components
- Hooks (useState, useEffect, etc.)

★ JSX (JavaScript XML)

JSX allows us to write HTML inside JavaScript.

```
const element = (
  <h1 className="title">Hello React!</h1>
);
```

★ Component (Functional)

```
function Welcome(props) {
  return <h1>Hello, {props.name}!</h1>;
}
export default Welcome;
```

Components must return a single parent element.

★ Using Component

```
import Welcome from "../Welcome";
function App() {
  return <Welcome name="Om" />;
}
```

★ Props (Properties)

Props are used to pass data from one component to another.

```
function User(props) {
  return <p>Name: {props.name}, Age: {props.age}</p>;
}
<User name="Om" age={22} />
```

★ State (useState Hook)

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

★ useEffect Hook

```
import { useEffect, useState } from "react";
function Example() {
  const [data, setData] = useState([]);
  useEffect(() => {
    fetch("https://api.example.com/data")
      .then(res => res.json())
      .then(setData);
  }, []); // Runs only once on mount
  return <p>Data Loaded!</p>;
}
```

★ List Rendering

```
const items = ["HTML", "CSS", "JS"];
return (
  <ul>
    {items.map((item, index) => (
      <li key={index}>{item}</li>
    ))}
  </ul>
);
```

Always use "key" in lists for better performance.

★ Conditional Rendering

```
function Greeting(props) {
  return (
    <div>
      {props.isLoggedIn ? (
        <p>Welcome Back!</p>
      ) : (
        <p>Please Login.</p>
      )}
    </div>
  );
}
```



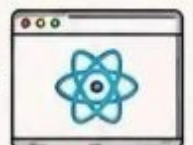
Remember :

- ☑ Components start with a capital letter.
- ☑ JSX must return a single parent element.
- ☑ Use hooks inside functional components.
- ☑ Keep components small and reusable.
- ☑ Always add key in list rendering.

Example in Real Life :

React is used in real projects like:

• Facebook • Instagram • Netflix • Airbnb • Reddit
React helps in building fast, scalable and interactive web applications.





5. BOOTSTRAP 5

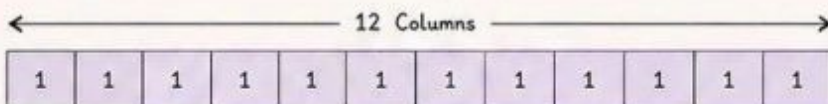
Bootstrap is a popular CSS framework that helps to build responsive and mobile-first websites quickly.

★ Why Use Bootstrap?

- Responsive and mobile-first.
- Pre-built components and utilities.
- Saves time and ensures consistency.
- Easy customization.
- Cross-browser compatible.

★ Grid System

Bootstrap uses a 12-column responsive grid system.



Breakpoints:

xs <576px | sm ≥576px | md ≥768px | lg ≥992px | xl ≥1200px | xxl ≥1400px

★ Container

`.container` → Fixed width, changes at breakpoints

`.container-fluid` → Full width

★ Grid Classes

Class	Extra Small (<576px)	Small (≥576px)	Medium (≥768px)	Large (≥992px)	Extra Large (≥1200px)	XX Large (≥1400px)
<code>.col-*</code>	100%	Auto	Auto	Auto	Auto	Auto
<code>.col-sm-*</code>	-	100%	Auto	Auto	Auto	Auto
<code>.col-md-*</code>	-	-	100%	Auto	Auto	Auto
<code>.col-lg-*</code>	-	-	-	100%	Auto	Auto
<code>.col-xl-*</code>	-	-	-	-	100%	Auto
<code>.col-xxl-*</code>	-	-	-	-	-	100%

★ Buttons

Use `.btn` class with contextual classes.

```
<button class="btn btn-primary">Primary</button>
<button class="btn btn-success">Success</button>
<button class="btn btn-danger">Danger</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-info">Info</button>
<button class="btn btn-secondary">Secondary</button>
```

★ Navigation Bar

Use `.navbar` class.

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <a class="navbar-brand" href="#">Brand</a>
  <button class="navbar-toggler"
    type="button" data-bs-toggle="collapse"
    data-bs-target="#navMenu">
    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse"
    id="navMenu">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link"
        href="#">Home</a></li>
    </ul>
  </div>
</nav>
```

★ Cards

```
<div class="card" style="width: 18rem;">
  
  <div class="card-body">
    <h5 class="card-title">Card Title</h5>
    <p class="card-text">Some quick example</p>
    <a href="#" class="btn btn-primary">
      Read More
    </a>
  </div>
</div>
```

★ Utility Classes (Examples)

```
.text-center → Center align text
.mt-3 → Margin top (1rem)
.p-3 → Padding (1rem)
.bg-light → Light background
.d-flex → Display flex
.justify-content-between → Space between
.align-items-center → Vertical center
```

★ Alerts

Use `.alert` class.

```
<div class="alert alert-success">Success</div>
<div class="alert alert-info">Info</div>
<div class="alert alert-warning">Warning</div>
<div class="alert alert-danger">Danger</div>
<div class="alert alert-secondary">Note</div>
```

★ Forms

```
<form>
  <div class="mb-3">
    <label class="form-label">Name</label>
    <input type="text" class="form-control"
      placeholder="Enter name">
  </div>
  <button type="submit" class="btn btn-primary">
    Submit
  </button>
</form>
```



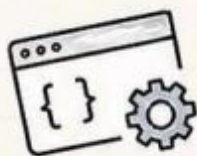
Remember :

- ✓ Bootstrap is mobile-first.
- ✓ Use containers and grid system for layout.
- ✓ Use utility classes to save time.
- ✓ Customize Bootstrap using your own CSS.

Example in Real Life :

Bootstrap is used in many real-world websites like Spotify, LinkedIn, and many admin dashboards for responsive design.





4. RESPONSIVE WEB DESIGN

Responsive Web Design makes your website look great on all devices (mobile, tablet, laptop, desktop).

★ Media Queries

Use media queries to apply CSS based on device width.

```
/* Mobile (max width 600px) */
@media (max-width: 600px) {
  body { background-color: #f8f8f8; }
  .container { padding: 10px; }
}

/* Tablet (601px - 1024px) */
@media (min-width: 601px) and (max-width: 1024px) {
  .container { padding: 20px; }
}

/* Desktop (min width 1025px) */
@media (min-width: 1025px) {
  .container { padding: 30px; }
}
```

★ Common Breakpoints

Device	Screen Size	Breakpoint
 Mobile (Portrait)	Up to 600px	max-width: 600px
 Mobile (Landscape)	601px - 768px	max-width: 768px
 Tablet	769px - 1024px	max-width: 1024px
 Laptop	1025px - 1440px	max-width: 1440px
 Desktop	1441px and above	min-width: 1441px

★ Flexible Layouts

- Use relative units like %, em, rem, vw, vh.
- Use Flexbox and Grid for layouts.
- Use max-width for containers.

```
.container {
  width: 90%;           /* Responsive width */
  max-width: 1200px;    /* Prevents too wide content */
  margin: 0 auto;       /* Center align */
}
```

★ Images & Media

- Images should scale with the container.
- Use max-width: 100%.
- Use object-fit for better control.

```
img {
  max-width: 100%;
  height: auto;
  display: block;
}

.cover {
  width: 100%;
  height: 100%;
  object-fit: cover;
}
```

★ Viewport Meta Tag

Add this in <head> to control layout on mobile browsers.

```
<meta name="viewport"
      content="width=device-width,
              initial-scale=1.0">
```



Mobile Tablet Laptop Desktop

★ Mobile First Approach

- Design for mobile first, then scale up for larger screens.
- Write base styles for mobile.
- Use media queries for larger screens.

```
/* Base styles for mobile */
.navbar {
  flex-direction: column;
}

/* Styles for larger screens */
@media (min-width: 768px) {
  .navbar { flex-direction: row; }
}
```

★ Units for Responsive Design

Unit	Description
%	Relative to parent element
em	Relative to parent font-size
rem	Relative to root font-size
vw	1% of viewport width
vh	1% of viewport height
vmin	1% of smaller viewport dimension
vmax	1% of larger viewport dimension

★ Best Practices

- ✓ Use semantic HTML.
- ✓ Use Flexbox / Grid for layouts.
- ✓ Use relative units, not fixed pixels.
- ✓ Optimize images for all devices.
- ✓ Test on real devices.
- ✓ Keep it simple and user-friendly.



Remember :

- ✓ Responsive design = Better User Experience (UX)
- ✓ Media queries help in adapting to different devices.
- ✓ Always test your website on multiple screen sizes.
- ✓ Mobile first approach is the best practice.

Tools for Testing :

- Chrome DevTools (Toggle Device Toolbar)
- Responsive Design Checker
- Real Devices
- Online Tools (browserling.com, responsivetesttool.com)



3. JAVASCRIPT (Contd.)

★ DOM (Document Object Model)

- JavaScript can access and manipulate HTML elements using DOM.
- Every HTML element is a node in a tree structure.
- JavaScript can change content, style, attributes and more.

★ Selecting Elements

Method	Example	Selects
<code>getElementById()</code>	<code>document.getElementById("id")</code>	Element with specific id
<code>getElementsByName()</code>	<code>document.getElementsByName("box")</code>	All elements with class name
<code>getElementsByTagName()</code>	<code>document.getElementsByTagName("p")</code>	All elements with tag name
<code>querySelector()</code>	<code>document.querySelector(".box")</code>	First element matching CSS selector
<code>querySelectorAll()</code>	<code>document.querySelectorAll(".box")</code>	All elements matching CSS selector

★ Changing Content & Style

- Changing Content**
`element.innerHTML = "<b>Hello</b>";`
`element.textContent = "Hello";`
`element.innerText = "Hello";`
- Changing Style**
`element.style.color = "red";`
`element.style.fontSize = "20px";`
`element.style.backgroundColor = "yellow";`

★ Events

- Events are actions or occurrences that happen in the browser.
- JavaScript can "listen" and "react" to these events.
- Common Events:

<code>click</code>	Mouse click
<code>dblclick</code>	Double click
<code>mouseover</code>	Mouse over element
<code>mouseout</code>	Mouse out of element
<code>keydown</code>	Key is pressed
<code>submit</code>	Form submitted
<code>change</code>	Value changed
<code>load</code>	Page or resource loaded

★ Event Example

```
<button id="btn">Click Me</button>
<p id="msg"></p>
<script>
  document.getElementById("btn").addEventListener("click", function(){
    document.getElementById("msg").innerText = "Button Clicked!";
  });
</script>
```

★ Array in JavaScript

```
let fruits = ["Apple", "Banana", "Mango"];
console.log(fruits[0]); // Apple
fruits.push("Orange"); // Add at end
fruits.pop(); // Remove last
console.log(fruits.length); // Length of array
for (let i = 0; i < fruits.length; i++){
  console.log(fruits[i]);
}
```

★ Conditional Statements

```
if (condition) {
  // code
} else if (condition) {
  // code
} else {
  // code
}
```

★ Switch Statement

```
switch (variable) {
  case value1:
    // code
    break;
  case value2:
    // code
    break;
  default:
    // code
}
```

★ Ternary Operator

```
condition ? expressionIfTrue : expressionIfFalse;
let age = 18;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result);
```

★ Template Literals (ES6)

```
let name = "Om";
let msg = `Hello, ${name}!`;
console.log(msg); // Hello, Om!
```



Remember :

- ✓ JavaScript makes web pages interactive.
- ✓ Use meaningful variable and function names.
- ✓ DOM lets you manipulate HTML with JS.
- ✓ Practice more to master JavaScript!

Mini Project Idea 💡

Create a To-Do List App:

- Add tasks
- Mark as completed
- Delete tasks





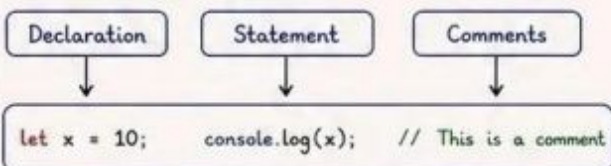
3. JAVASCRIPT

JavaScript is used to add interactivity and logic to the webpage.

★ What is JavaScript?

- JavaScript is a programming language of the web.
- It makes webpages interactive and dynamic.
- It can change HTML and CSS.
- It runs in the browser.

★ JavaScript Syntax



- Declaration - Creating variables.
- Statement - Instructs to perform an action.
- Comments - Notes for code (ignored by browser).

★ Data Types

Type	Description	Example
String	Textual data	"Hello"
Number	Numeric data	42, 3.14
Boolean	True or False	true, false
Undefined	Declared but not assigned	let a;
Null	Intentional absence of value	let a = null;
Object	Collection of key-value pairs	{ name: "Om" }
Array	Ordered list of values	[1, 2, 3]

★ Control Statements

1. if - else

```
if (x > 10) {
  console.log("x is greater");
} else {
  console.log("x is smaller");
}
```

2. for Loop

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

3. while Loop

```
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

★ Functions

```
function greet(name) {
  return "Hello, " + name;
}

let msg = greet("Om");
console.log(msg); // Hello, Om
```

★ Ways to Add JavaScript

1. Internal JavaScript - Using `<script>` in `<head>` or `<body>`.
`<script>`
 `console.log("Hello, JS!");`
`</script>`
2. External JavaScript - Using external .js file.
`<script src="script.js"></script>`
3. Inline JavaScript - Using in HTML tag.
`<button onclick="alert('Hi!')">Click Me</button>`

★ Variables in JavaScript

Keyword	Scope	Can Re-declare	Can Re-assign	Example
var	Function Scoped	Yes	Yes	var a = 5;
let	Block Scoped	No	Yes	let b = 10;
const	Block Scoped	No	No	const c = 20;

★ Operators

Arithmetic	+ - * / % **	(add, sub, mul, div, mod, power)
Assignment	= += -= *= /= %= **=	(assign values)
Comparison	== === != !== > < >= <=	(compare values)
Logical	&& !	(and, or, not)
Ternary	condition ? x : y	(if-else shortcut)



Remember :

- JavaScript adds behavior to webpages.
- Use meaningful variable names.
- Keep your code clean and well organized.
- Test your code in browser console (F12).

Mini Example (HTML + JS)

```
<button onclick="sayHi()">Say Hi</button>
<script>
  function sayHi() {
    alert("Hello from JavaScript!");
  }
</script>
```

2. CSS

CSS (Cascading Style Sheets) is used to style and layout the webpage.

★ What is CSS ?

- CSS stands for Cascading Style Sheets.
- It controls the look and feel of HTML elements.
- It separates content (HTML) from design.
- It makes websites beautiful and responsive.
- It follows a cascading order (priority).

★ Ways to Add CSS

1. Inline CSS - Using style attribute inside an element.

```
<p style="color: red;">This is red text.</p>
```

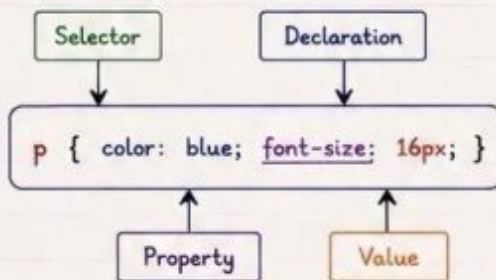
2. Internal CSS - Using <style> tag in <head>.

```
<style>
p { color: blue; }
</style>
```

3. External CSS - Using separate .css file.

```
<link rel="stylesheet" href="style.css">
```

★ CSS Syntax



- **Selector**: selects the HTML element.
- **Property**: what you want to change.
- **Value**: the value of the property.
- **Declaration**: property and value together.

★ CSS Selectors

Selector	Example	Meaning
Element Selector	p { ... }	Selects all <p> elements
Class Selector	.box { ... }	Selects all elements with class="box"
ID Selector	#main { ... }	Selects element with id="main"
Universal Selector	* { ... }	Selects all elements
Group Selector	h1, p { ... }	Selects all <h1> and <p> elements
Descendant Selector	div p { ... }	Selects <p> inside <div>
Child Selector	ul > li { ... }	Selects direct child of
Adjacent Sibling	h1 + p { ... }	Selects <p> next to <h1>

★ CSS Comments

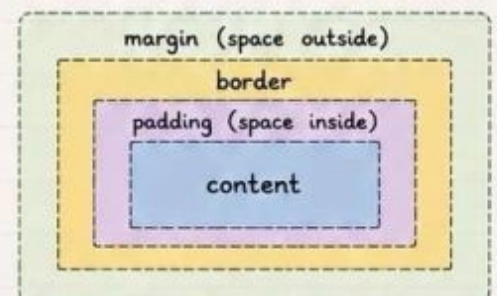
```
/* This is a
   multi-line comment */

/* Single line comment */
```

★ CSS Colors

- By Color Name: red, blue, green
- By Hex: #ff0000, #00ff00, #0000ff
- By RGB: rgb(255, 0, 0)
- By HSL: hsl(0, 100%, 50%)
- By RGBA: rgba(0, 0, 255, 0.5)
(last value is opacity 0 to 1)

★ CSS Box Model



★ Example (in CSS)

```
p {
  color: darkblue;
  font-size: 18px;
  background-color: #f0f0f0;
  padding: 10px;
  border: 1px solid #ccc;
  margin: 20px;
  text-align: center;
}
```



This is a paragraph styled with CSS.

★ Remember

- CSS controls presentation, HTML controls structure.
- Use external CSS for better performance.
- Keep your CSS clean and organized!





FULL STACK WEB DEVELOPMENT



CHEATSHEET - PAGE 1

1. HTML

HTML (HyperText Markup Language) is used to create the structure of a webpage.

★ What is HTML?

- HTML stands for HyperText Markup Language.
- It is the standard language to create web pages.
- HTML describes the structure of a webpage.
- HTML elements are the building blocks of a webpage.
- It works with CSS (for styling) and JavaScript (for behavior).

★ Basic HTML Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Page Title</title>
</head>
<body>
  <!-- Visible content goes here -->
</body>
</html>
```

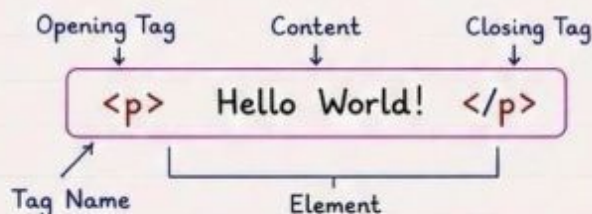
★ Common HTML Tags

Tag	Description	Example
<h1> to <h6>	Headings	Heading 1
<p>	Paragraph	This is a paragraph.
<a>	Link	Google
	Image	
/	Unordered / Ordered List	Item
<div>	Division / Container	Content
	Inline Container	Text
<form>	Form	<form>...</form>
<input>	Input Field	<input type="text">
<button>	Button	<button>Click Me</button>

★ Attributes Example

- Google
→ Opens link in new tab
-
→ Image with size and alternative text
- <input type="email" placeholder="Enter email" required>
→ Email input field (required)
- <button type="submit">Submit</button>
→ Submit button in form

★ HTML Elements



★ Void Elements (No Closing Tag)

These elements do not have a closing tag.

Examples:

```

<br> <hr> <input> <meta> <link>
```



Remember :

- HTML creates the structure of the webpage.
- Use semantic tags (like <header>, <nav>, <main>, <footer>) for better SEO and readability.
- Always validate your HTML code.

Tip :

Write clean, organized HTML for better maintenance!





FULL STACK WEB DEVELOPMENT



CHEATSHEET - PAGE 1

1. HTML

HTML (HyperText Markup Language) is used to create the structure of a webpage.

★ What is HTML?

- HTML stands for HyperText Markup Language.
- It is the standard language to create web pages.
- HTML describes the structure of a webpage.
- HTML elements are the building blocks of a webpage.
- It works with CSS (for styling) and JavaScript (for behavior).

★ Basic HTML Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Page Title</title>
</head>
<body>
  <!-- Visible content goes here -->
</body>
</html>
```

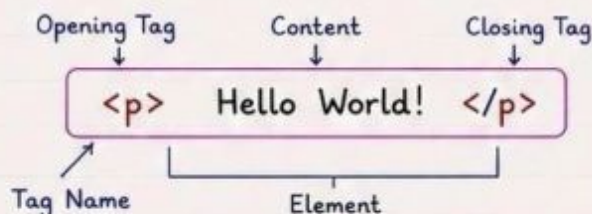
★ Common HTML Tags

Tag	Description	Example
<h1> to <h6>	Headings	Heading 1
<p>	Paragraph	This is a paragraph.
<a>	Link	Google
	Image	
/	Unordered / Ordered List	Item
<div>	Division / Container	Content
	Inline Container	Text
<form>	Form	<form>...</form>
<input>	Input Field	<input type="text">
<button>	Button	<button>Click Me</button>

★ Attributes Example

- Google
→ Opens link in new tab
-
→ Image with size and alternative text
- <input type="email" placeholder="Enter email" required>
→ Email input field (required)
- <button type="submit">Submit</button>
→ Submit button in form

★ HTML Elements



★ Void Elements (No Closing Tag)

These elements do not have a closing tag.

Examples:

```

<br> <hr> <input> <meta> <link>
```



Remember :

- HTML creates the structure of the webpage.
- Use semantic tags (like <header>, <nav>, <main>, <footer>) for better SEO and readability.
- Always validate your HTML code.

Tip :

Write clean, organized HTML for better maintenance!



★ 14. MASTER NODE.JS — COMPLETE ROADMAP ★

Core Concepts

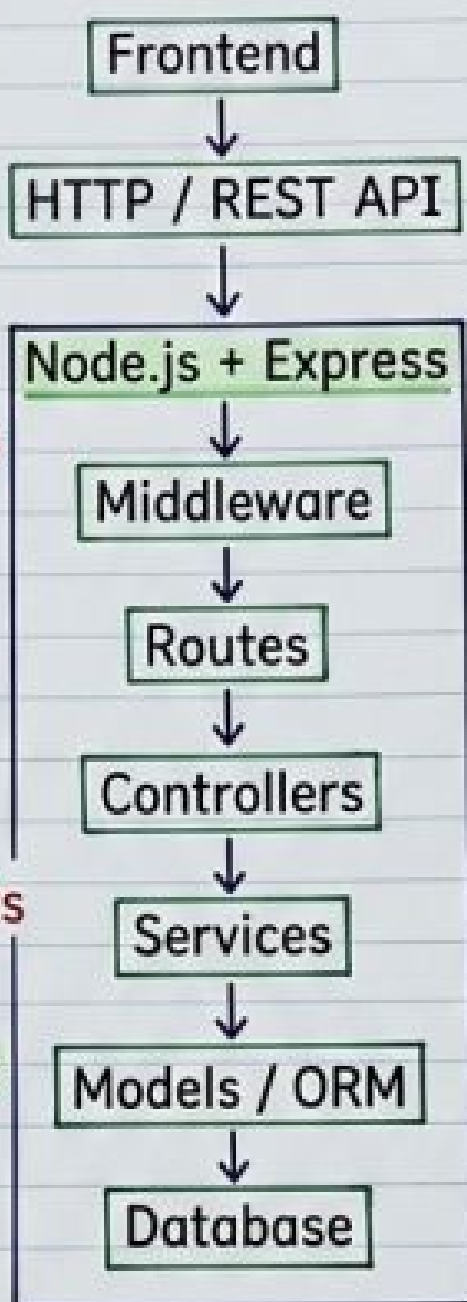
- JavaScript
- Modules
- npm
- Async/Await
- Promises
- Event Loop
- EventEmitter

Backend Concepts

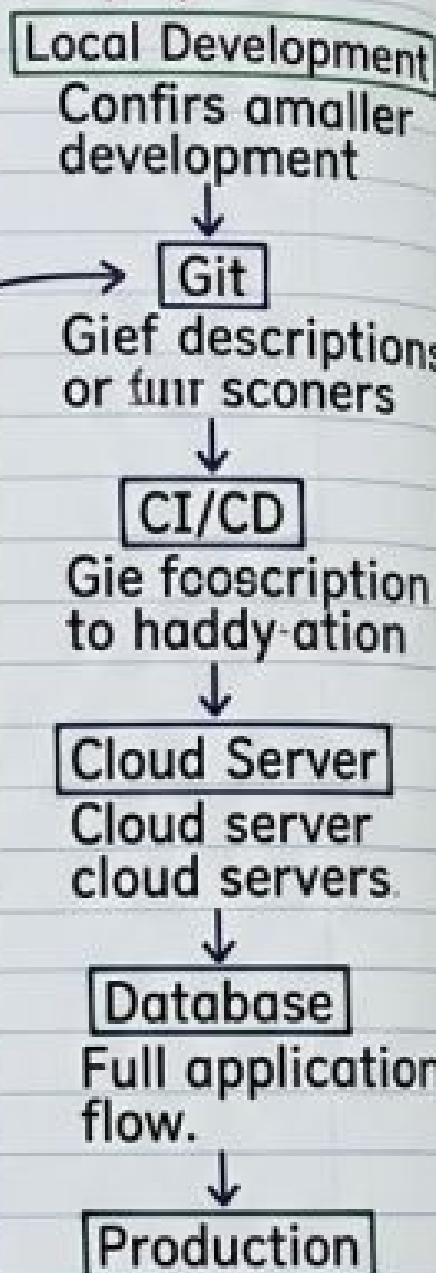
- HTTP
- REST
- Express
- Middleware
- CRUD
- Authentication
- Authorization

Advanced Concepts

- Streams
- Buffers
- Worker Threads
- Caching
- Performance
- Logging
- Testing
- Security



Deployment



Node.js Learning Roadmap

- ↓ [JavaScript]
- ↓ [Node.js Basics]
- ↓ [Core Modules]
- ↓ [Async + Event Loop]
- ↓ [HTTP]
- ↓ [REST APIs]
- ↓ [Express]
- ↓ [Database]
- ↓ [Authentication]
- ↓ [Security]
- ↓ [Testing]
- ↓ [Performance]
- ↓ [Deployment]

BUILD REAL BACKEND SYSTEMS

★ Learn → Build → Debug ★
Secure → Test → Deploy

Node.js Interview Quick Revision

1. What is Node.js?
2. What is npm?
3. What is the event loop?
4. What is middleware?
5. What is REST?
6. What is Express?
7. What is JWT?
8. What is a stream?
9. What is authentication?
10. How do you deploy a Node.js API?

★ 13. NODE.JS SECURITY + TESTING + DEBUGGING ★

1. Common Security Risks

- SQL/NoSQL injection
- XSS (Cross-Site Scripting)
- CSRF (Cross-Site Request Forgery)
- broken authentication
- insecure secrets
- excessive requests

2. Security Practices

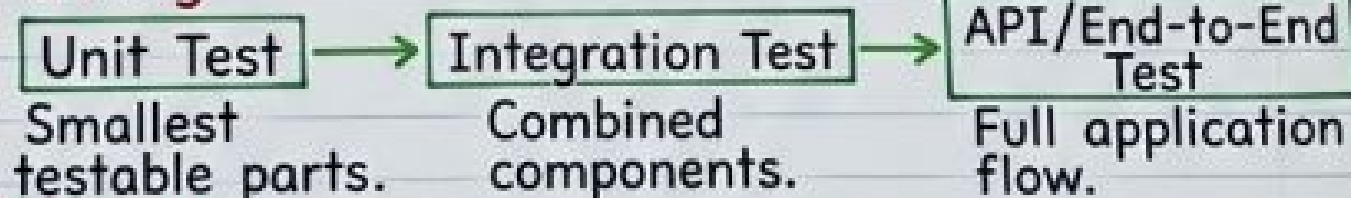
- validate input
- sanitize where appropriate
- hash passwords
- use HTTPS
- protect secrets with environment variables
- rate limiting
- security headers

3. Environment Variables

```
.env
PORT=3000
DATABASE_URL=...
```

→ Why '.env' should not be committed to Git. Avoid exposing sensitive info in version control.

4. Testing



5. Debugging

- console.log()
- breakpoints
- stack traces
- Node debugger

6. Error Logging

Why production applications need structured logs. Helps trace issues, monitor application health, and understand user behavior.

Structured logs are easier to search and analyze.

7. Production Checklist

- | | |
|-------------------------|-------------------------|
| ✓ <u>validation</u> | ✓ logging |
| ✓ <u>authentication</u> | ✓ environment variables |
| ✓ error handling | ✓ testing |
| ✓ logging | ✓ <u>HTTPS</u> |

★ 12. ADVANCED NODE.JS – STREAMS + BUFFERS ★

In this page, we will learn some advanced Node.js concepts like buffers, streams, file handling and performance tips. These help us build faster and more efficient applications.

① What is a Buffer?

- A **buffer** is a temporary memory area in Node.js.
- It stores binary or raw data (like images, files, audio, video, etc.).
- It is not a normal string. It stores data in bytes.

Example:

```
const buf = Buffer.from([0x48, 0x65, 0x6c, 0x6c, 0x66]);  
// This stores 'Hello' in binary format
```

- Buffers are useful when working with files, network data and streams.

③ Types of Streams

Type	Direction	Example	Use Case
Readable	Read data	fs.createReadStream()	Reading from file, network, etc.
Writable	Write data	fs.createWriteStream()	Writing to file, response, etc.
Duplex	Read + Write	net.Socket	Chat apps, TCP connections, etc.
Transform	Read + Modify + Write	zlib.createGzip()	Compress, convert, process data.

⑤ Real-world Uses

- Video streaming (YouTube, Netflix, live streaming).
- Large file handling (logs, backups, datasets).
- Uploads / downloads (cloud storage, websites).
- Data processing (CSV, JSON, image processing).

⑥ pipe()

- The `pipe()` method is used to connect readable and writable streams easily.

Example:

```
fs.createReadStream('file.txt')  
  .pipe(fs.createWriteStream('copy.txt'))  
  .on('finish', () => console.log('Done!'));
```

It automatically handles data flow and errors.

⑦ Performance

To build a fast and reliable app, follow these tips:

- ① Avoid blocking operations (like heavy loops or sync code).
- ② Use streams for large data (instead of loading all in memory).
- ③ Cache data when appropriate (like API responses, DB queries).
- ④ Monitor memory usage (to avoid memory leaks).

② What is a Stream?

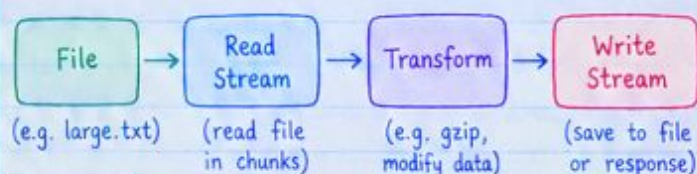
- A **stream** lets us process data piece by piece instead of loading the whole data into memory.
- It is more memory efficient, especially for large files or continuous data (like video or logs).

Example:

Instead of reading a 1GB file completely, we read small chunks (like 64KB) and process them one by one.

④ Stream Pipeline

We can connect multiple streams together using pipeline to process data step by step.



Example:

```
const { pipeline } = require('stream');  
const fs = require('fs');  
const zlib = require('zlib');  
  
pipeline(  
  fs.createReadStream('input.txt'),  
  zlib.createGzip(),  
  fs.createWriteStream('output.gz'),  
  (err) => { if (err) console.error('Pipeline failed', err); }  
);
```

⑧ Worker Threads

Node.js is single-threaded. If you do CPU-heavy work (like image processing, data encryption, or complex calculations), it can block the main event loop.

To solve this, we can use worker threads to move heavy tasks to a separate thread.

When to use?

- Image/video processing
- Cryptography
- Heavy calculations
- Machine learning tasks



Example:

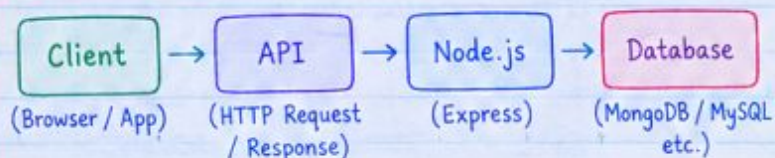
```
const { Worker } = require('worker_threads');  
const worker = new Worker('./heavyTask.js');  
worker.on('message', (msg) => console.log('Result:', msg));  
worker.postMessage(data);
```

This keeps your main thread free and your app responsive.

★ 11. DATABASE + AUTHENTICATION ★

This page explains how Node.js applications communicate with databases and secure users.

① Node.js + Database



- Client sends a request to the API (like a website or mobile app).
- API (Express) in Node.js handles the request.
- Node.js communicates with the database.
- Database returns the data back to Node.js, and then to the client.

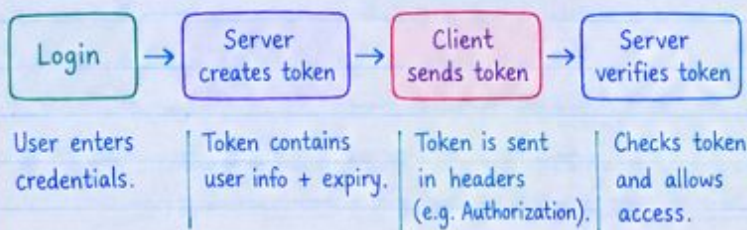
③ CRUD with Database

CRUD means Create, Read, Update, Delete.

- ① Create (POST) → Add new data.
Example: create a new user.
- ② Read (GET) → Get data from database.
Example: get user details.
- ③ Update (PUT/PATCH) → Modify existing data.
Example: update user profile.
- ④ Delete (DELETE) → Remove data.
Example: delete a user.

⑥ JWT (JSON Web Token)

JWT is used to keep the user logged in without storing data on the server.



⑦ Authentication vs Authorization

Point	Authentication	Authorization
Meaning	Verifies who you are	Decides what you can do
Question	"Are you the user?"	"What can you access?"
Example	Login with password	Accessing admin panel
Occurs	At login	After login
Handled by	Auth system	Permission / role system

Both are important. First authenticate, then authorize.

② SQL vs NoSQL

Feature	SQL	NoSQL
Data format	Tables	Documents
Structure	Fixed structure	Flexible structure
Data access	SQL (query language)	NoSQL query / APIs
Relationships	Strong (foreign keys)	Flexible (embedded / reference)
Examples	MySQL, PostgreSQL, SQLite	MongoDB, Firebase, CouchDB

④ Authentication

Authentication means verifying who the user is.

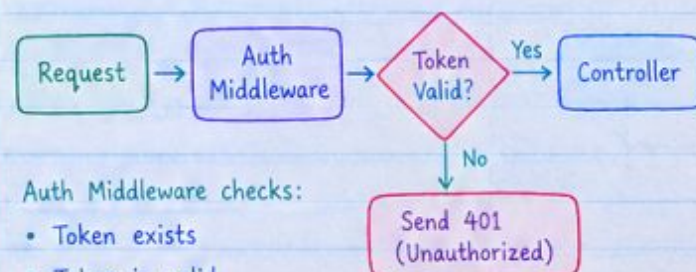


⑤ Password Security

- Passwords should be hashed, not stored as plain text.
- Hashing converts the password into a fixed-length string.
- Even if the database is leaked, the real password cannot be seen.
- Use bcrypt, Argon2 or similar strong hashing methods.

⑧ Protected Route

Only logged-in users can access this route.



Auth Middleware checks:

- Token exists
- Token is valid
- User is logged in



Security Warning

Never store passwords or secret keys directly in source code.

Use environment variables (.env) and keep sensitive data safe.

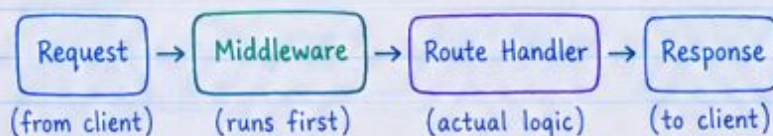
★ 10. MIDDLEWARE + CONTROLLERS + ERROR HANDLING ★

This page explains how a professional Express backend is organized and how request flows through different parts.

① What is Middleware?

Middleware is a function that runs between the incoming request and the final response. It can do things like logging, authentication, validation, or modifying the request/response.

Flow:



② Types of Middleware

1. **Application middleware**
 - Used for the whole app (`app.use()`).
2. **Router middleware**
 - Used for a specific route or router.
3. **Built-in middleware**
 - Provided by Express (like `express.json()`).
4. **Third-party middleware**
 - Made by other developers (like `cors`, `morgan`).
5. **Error-handling middleware**
 - Handles errors (4 arguments).

③ `express.json()`

This middleware parses incoming JSON data from the request body and makes it available in `req.body`.

Example:

```
app.use(express.json());
// Now we can access data like:
// req.body.name
```

Why?

- Without it, `req.body` will be undefined.
- It helps us work with JSON data easily.

④ Custom Middleware

We can create our own middleware for tasks like logging, authentication, etc.

Example (Logging Middleware):

```
const logger = (req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
  next(); // move to next step
};
```

Use it like:

```
app.use(logger); // runs for every request
```

⑤ Controllers

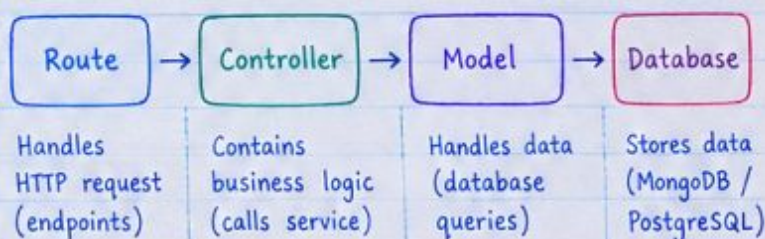
- Routes should only handle HTTP requests.
- Business logic should be separated into controllers. This keeps the code clean, readable and easy to maintain.

Example:

routes → controllers → models → database
(not all logic in routes)

⑥ MVC Architecture

In Express, we can follow MVC pattern.



⑦ Error Handling

- Use `try/catch` for async code.
- Use `next(error)` to pass error to error middleware.
- Use a centralized error middleware to handle all errors in one place.

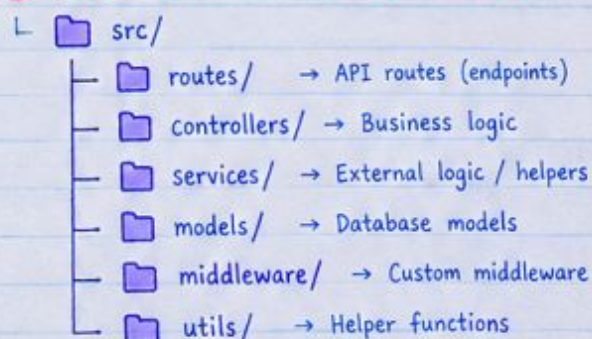
Example:

```
try {
  const user = await User.findById(id);
  res.json(user);
} catch (error) {
  next(error); // pass to error middleware
}
```

Error Middleware (at the end):

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({
    message: err.message || 'Something went wrong'
  });
});
```

⑧ Professional Folder Structure



This structure keeps the project organized and scalable.

★ 9. EXPRESS.JS — NODE.JS WEB FRAMEWORK ★

Express.js is a fast, simple and flexible web framework for Node.js. It helps us build web servers and APIs easily.

① What is Express.js?

Express.js is a minimal and flexible web framework for Node.js. It provides a simple way to build web applications and APIs using JavaScript.

② Why Express?

Express is commonly used with Node.js because:

- Routing — handles different URLs.
- Middleware — add extra features (like auth).
- Request handling — works with incoming data.
- Response handling — sends data back to client.
- API development — easy to build REST APIs.

③ Install Express

We can install Express using npm:

```
npm install express
```

④ First Express Server

A simple server to show Express is working:

```
const express = require("express");
const app = express();

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

This will start a server on port 3000.

⑤ Routes

Express provides methods to handle different HTTP requests:

Method	Use
app.get()	Get data (read)
app.post()	Create data
app.put()	Update data
app.patch()	Partially update data
app.delete()	Delete data

Example:

```
app.get("/users", (req, res) => {
  res.send("Get users");
});
```

⑥ Request & Response

Inside a route, we get data from request (req) and send data using response (res).

Object	Description
req	Contains request data (params, query, body, headers etc).
res	Used to send response (json, text, status etc).

Example:

```
app.get("/users", (req, res) => {
  const id = req.params.id;
  res.json({ message: "User " + id });
});
```

⑦ Router

We can split routes into separate files using Express Router.

routes/users.js

```
const express = require("express");
const router = express.Router();

router.get("/", (req, res) => {
  res.json({ message: "Users route" });
});

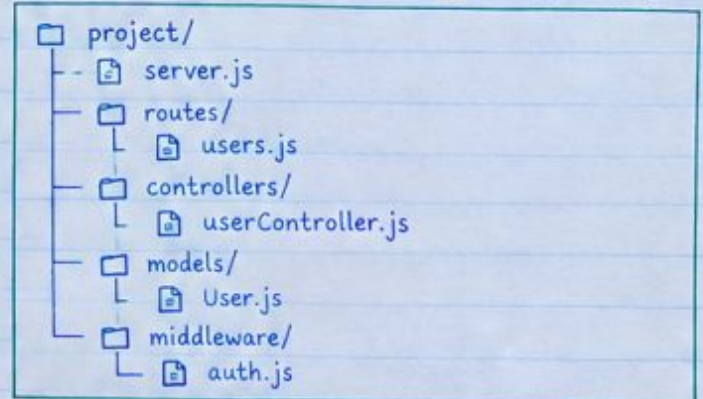
module.exports = router;
```

Then use it in server.js:

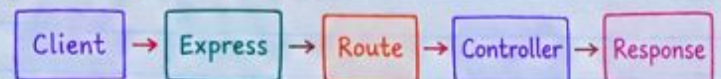
```
const usersRouter = require("./routes/users");
app.use("/users", usersRouter);
```

⑧ Basic Project Structure

A simple structure for a Node.js + Express project:



Flow of a Request



Client sends a request → Express handles it → Route matches → Controller processes → Response goes back.

★ 8. BUILDING REST APIS ★

REST APIs help us to make data available over the internet. We use Node.js to create and handle these APIs.

① What is REST API?

REST API is a way to communicate with a server using standard HTTP methods like GET, POST, PUT, DELETE. It follows a simple and fixed structure.

② Resource-Based Design

In REST, we work with resources. Each resource has a unique URL.

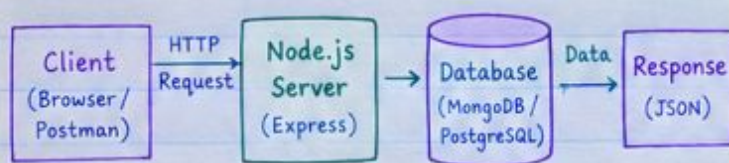
Examples:

- /users → all users
- /users/10 → user with id 10
- /products → all products
- /orders/25 → order with id 25

③ CRUD (HTTP Methods)

Action	HTTP Method
Create	POST
Read	GET
Update	PUT / PATCH
Delete	DELETE

④ Request → Server → Database → Response



Client sends a request → Server processes it → Fetches/updates data from database → Sends back response.

⑤ JSON

JSON (JavaScript Object Notation) is a lightweight format to store and exchange data.

Example:

```
{
  "id": 1,
  "name": "Vijay"
}
```

It is easy to read and write for both humans and machines.

⑥ REST API Response

The server sends data in JSON format.

Example response:

```
{
  "id": 1,
  "name": "Vijay"
}
```

⑦ API Status Codes

Code	Meaning	When to use
200	OK	Request successful
201	Created	Resource created
400	Bad Request	Invalid input
401	Unauthorized	Not logged in
403	Forbidden	No permission
404	Not Found	Resource not found
500	Internal Server Error	Server error

⑧ Good REST Practices

- Use meaningful URLs (e.g. /users/10).
- Use correct HTTP methods (GET, POST, PUT, DELETE).
- Return proper status codes.
- Send consistent JSON responses.
- Validate input data.
- Handle errors properly.

Example REST Endpoints

Method	Endpoint	Description
GET	/users	Get all users
GET	/users/10	Get user by id
POST	/users	Create a new user
PUT	/users/10	Update user (full)
PATCH	/users/10	Partially update user
DELETE	/users/10	Delete user

★ REST APIs make it easy to build, connect and manage data between different applications using standard rules.

★ 6. EVENTS + EVENTEMITTER + TIMERS ★

Node.js uses an event-driven model. It means instead of waiting for a task to finish, we listen for events and then run a function when that event happens.

① What is an Event?

- An event is a signal that something happened in our application.
- It can be a user action, a system event or a custom event.
- We can listen to that event and run a function when it occurs.

Example: A user clicks a button → "click" event is fired.

③ EventEmitter

EventEmitter is a built-in class in Node.js that helps us handle events.

Main methods:

Method	Purpose
on()	Listen for an event (multiple times)
emit()	Trigger an event
once()	Listen for an event (only once)
off()	Remove a listener

Example:

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

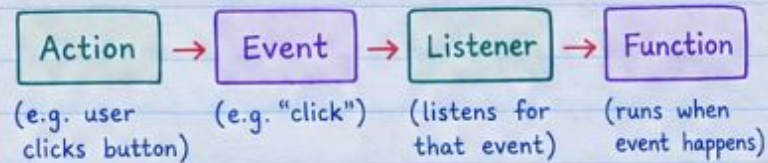
emitter.on("message", (data) => {
  console.log("Message received:", data);
});
```

⑥ Real-world Uses

- Notifications (e.g. new message, email)
- Logging (e.g. log when something happens)
- Server events (e.g. connection, disconnect)
- Authentication events (e.g. login, logout)
- Background tasks (e.g. sending emails, cleanup)

② Event-Driven Architecture

In Node.js, the flow is like this:



④ Custom Events

We can create our own events and handle them using EventEmitter.

Example:

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("login", (user) => {
  console.log("User logged in:", user);
});

// Trigger the event
emitter.emit("login", "Hitesh");
```

⑤ Timers

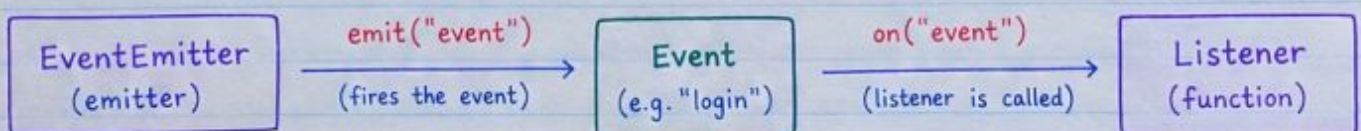
Node.js also provides timers to run code after a delay or repeatedly.

Method	Use
setTimeout()	Run once after a delay
setInterval()	Run repeatedly after a delay
setImmediate()	Run after I/O events (next tick)

Example:

```
setTimeout(() => {
  console.log("This runs after 2 seconds");
}, 2000);
```

★ Event Flow (Emitter → Listener) ★



This is how an event travels from the emitter to the listener in Node.js.

★ 6. EVENTS + EVENTEMITTER + TIMERS ★

Node.js uses an event-driven model. It means instead of waiting for a task to finish, we listen for events and then run a function when that event happens.

① What is an Event ?

- An event is a signal that something happened in our application.
- It can be a user action, a system event or a custom event.
- We can listen to that event and run a function when it occurs.

Example : A user clicks a button → "click" event is fired.

③ EventEmitter

EventEmitter is a built-in class in Node.js that helps us handle events.

Main methods :

Method	Purpose
on()	Listen for an event (multiple times)
emit()	Trigger an event
once()	Listen for an event (only once)
off()	Remove a listener

Example :

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

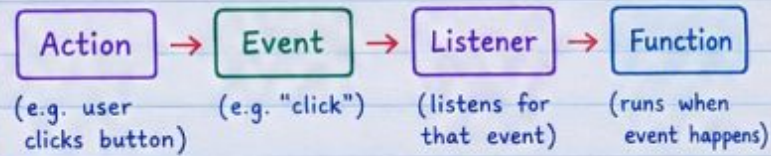
emitter.on("message", (data) => {
  console.log("Message received:", data);
});
```

⑥ Real-world Uses

- Notifications (e.g. new message, email)
- Logging (e.g. log when something happens)
- Server events (e.g. connection, disconnect)
- Authentication events (e.g. login, logout)
- Background tasks (e.g. sending emails, cleanup)

② Event-Driven Architecture

In Node.js, the flow is like this :



④ Custom Events

We can create our own events and handle them using EventEmitter.

Example :

```
const EventEmitter = require("events");
const emitter = new EventEmitter();

emitter.on("login", (user) => {
  console.log("User logged in:", user);
});

// Trigger the event
emitter.emit("login", "Hitesh");
```

⑤ Timers

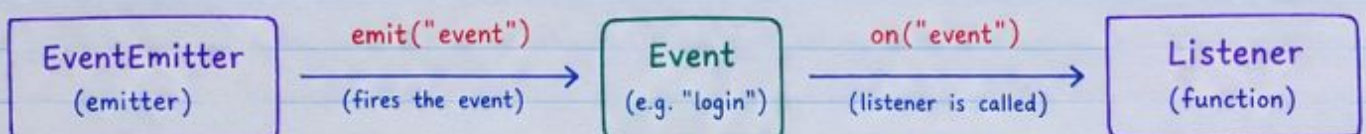
Node.js also provides timers to run code after a delay or repeatedly.

Method	Use
setTimeout()	Run once after a delay
setInterval()	Run repeatedly after a delay
setImmediate()	Run after I/O events (next tick)

Example :

```
setTimeout(() => {
  console.log("This runs after 2 seconds");
}, 2000);
```

★ Event Flow (Emitter → Listener) ★



This is how an event travels from the emitter to the listener in Node.js.

★ 5. ASYNC NODE.JS + EVENT LOOP ★

1. Synchronous vs Asynchronous

Feature	Synchronous	Asynchronous
Execution	One by one	Parallel (non-blocking)
Waiting	Waits for result	Does not wait
Performance	Slower	Faster
Example	readFileSync()	readFile()

2. Blocking vs Non-Blocking

Blocking

- Stops the next task
- Waits for completion
- Example: readFileSync()

Like standing in one queue and waiting.

Non-Blocking

- Gives task and continues
- Uses callbacks / promises
- Example: readFile()

Like giving task and moving ahead.

3. Callback

A callback is a function that runs after a task is completed.

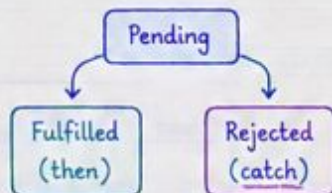
```
fs.readFile("file.txt", (err, data) => {  
  if (err) return console.error(err);  
  console.log(data);  
});
```

4. Promise

A promise represents a value that will be available in the future.

Promise States :

- Pending (initial state)
- Fulfilled (success)
- Rejected (error)



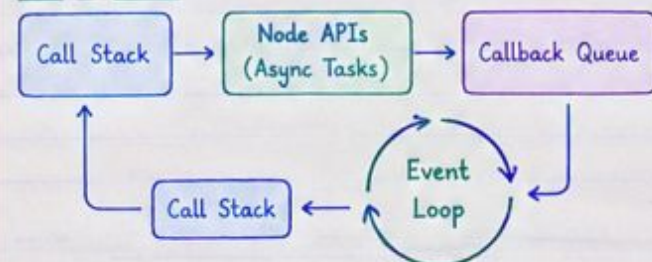
5. async / await

async and await make asynchronous code look like synchronous code.

```
async function getData() {  
  const data = await fetchData();  
  console.log(data);  
}
```

6. Event Loop

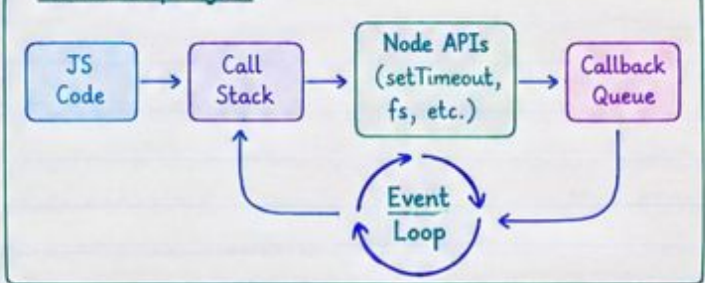
How it works:



7. Event Loop Flow (Step by Step)

1. JS code runs in Call Stack.
2. If there is an async task, it goes to Node APIs.
3. When the task is completed, its callback goes to Callback Queue.
4. Event Loop checks if Call Stack is empty.
5. It moves the callback from queue to Call Stack.
6. The callback runs.

Event Loop Cycle



8. Important Concept

- ★ Node.js uses a single JavaScript thread, but can handle many I/O operations efficiently through asynchronous mechanisms.

9. Common Mistake

CPU-heavy synchronous work can block the event loop.
Example: heavy calculation, large loops, file processing.

▲ Result:

- Other requests will wait.
- Server becomes slow.
- Not a good practice.

Quick Summary

- Use async/await or promises.
- Avoid blocking operations.
- Event loop keeps your app responsive.
- I/O tasks are handled outside the main thread.
- CPU-heavy work can block the event loop.

★ Remember :

Asynchronous code helps Node.js handle many users at the same time without blocking the server.



★ 4. NODE.JS CORE MODULES ★

1. File System - fs

The `fs` module is used to work with files and directories. It helps us read, write and modify files.

Common methods:

- `fs.readFile()` - read a file
- `fs.writeFile()` - write a file
- `fs.appendFile()` - add data to a file

Sync vs Async:

<u>Synchronous (Sync)</u>	<u>Asynchronous (Async)</u>
• blocks the thread • simple to use • not recommended for large files	• non-blocking • better performance • uses callbacks / promises

2. Path Module

The `path` module helps to work with file and directory paths in a platform-independent way.

Common methods:

- `path.join()` - join paths
- `path.basename()` - get file name
- `path.dirname()` - get directory name
- `path.extname()` - get file extension

3. OS Module

The `os` module provides information about the operating system and hardware.

Common methods:

- `os.platform()` - platform type
- `os.cpus()` - CPU info
- `os.freemem()` - free memory
- `os.totalmem()` - total memory

4. URL Module

The `url` module helps to parse and work with URLs.

It provides utilities to handle URL components.

Example:

```
- const url = new URL("https://example.com:3000/api?id=10#section");
  console.log(url.hostname); // example.com
  console.log(url.pathname); // /api
  console.log(url.search); // ?id=10
```

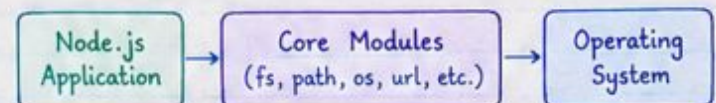
5. Why Core Modules Matter?

- Built into Node.js (no npm installation needed).
- Faster and more reliable.
- Used for essential system operations.

6. Summary Table

<u>Module</u>	<u>Purpose</u>	<u>Example</u>
<code>fs</code>	Files	<code>readFile()</code>
<code>path</code>	File paths	<code>join()</code>
<code>os</code>	System information	<code>cpus()</code>
<code>url</code>	URLs	<code>URL()</code>

7. Node.js → Core Modules → Operating System



Core modules provide the bridge between your application and the operating system.

8. Quick Code Examples

1. Read a file (`fs`):

```
const fs = require('fs');
fs.readFile('./file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

2. Join paths (`path`):

```
const path = require('path');
console.log(path.join(__dirname, 'data', 'file.txt'));
```

3. Get system info (`os`):

```
const os = require('os');
console.log(os.platform());
console.log(os.cpus().length);
```

4. Parse URL (`url`):

```
const { URL } = require('url');
const myURL = new URL("https://example.com");
console.log(myURL.hostname);
```

9. Important Note

★ These core modules are always available in Node.js. You just need to `require()` them and start using them! 😊

★ 3. MODULES + NPM ECOSYSTEM ★

1. What is a Module ?

A module is a reusable piece of code. It helps us organise code into small files so that it is easy to manage and use again.

Example : We can create our own modules or use built-in modules and npm packages.

2. Core Modules

Node.js comes with many built-in (core) modules.

Common Core Modules

- fs (file system)
- http (web server)
- path (file paths)
- events (event handling)
- os (system info)
- url (URL parsing)

3. CommonJS (require)

In older Node.js versions, we use require() to import modules and module.exports to export.

```
const fs = require("fs");
const data = "Hello";
module.exports = data;
```

4. ES Modules (import)

In modern Node.js, we can use import and export.

```
import fs from "node:fs";
const data = "Hello";
export default data;
```

CommonJS vs ES Modules

Feature	CommonJS	ES Modules
Syntax	<code>require()</code>	<code>import</code>
Export	<code>module.exports</code>	<code>export</code>
File extension	<code>.js</code>	<code>.mjs</code> or <code>.js</code> (with <code>type: module</code>)
Support	Older Node.js	Modern Node.js

5. package.json type

If we want to use ES modules, we can set type in `package.json`.

```
{
  "type": "module"
}
```

This tells Node.js to treat `.js` files as ES modules.

6. Creating Your Own Module

We can create our own module and use it in other files.

math.js

```
export function add(a,b){
  return a + b;
}
```

app.js

```
import { add } from "./math.js";
console.log(add(2,3));
```

7. npm Packages

npm is the default package manager for Node.js.

- Dependency : Used in production.
- DevDependency : Used only for development.
- Package version : Defines which version to use.
- Semantic versioning : MAJOR.MINOR.PATCH (e.g. 1.2.3)

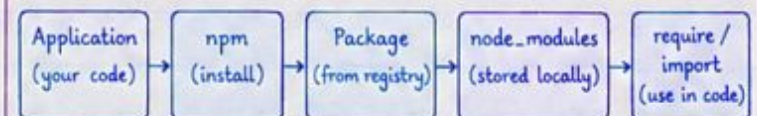
8. Useful npm Commands

<code>npm init</code>	Create package.json (interactive)
<code>npm init -y</code>	Create package.json (default)
<code>npm install express</code>	Install a package
<code>npm uninstall express</code>	Remove a package

npm Workflow



Application → npm → Package → node_modules → require/import



★ Remember :

Modules help us keep our code organized, npm gives us access to thousands of useful packages, and together they make Node.js powerful !!!

★ 2. NODE.JS SETUP + CORE CONCEPTS ★

1. Installing Node.js

Node.js can be installed from the official website (<https://nodejs.org>). It comes with npm (Node Package Manager). After installation, we can check the version using terminal commands.

Check version:

```
> node -v    # shows Node.js version
> npm -v     # shows npm version
```

2. Running JavaScript

We can run a JavaScript file using Node.js.

```
> node app.js # runs the file
```

This helps us execute JavaScript code outside the browser.

3. Node REPL

REPL stands for Read - Evaluate - Print - Loop. It is an interactive shell where we can write and execute JavaScript code line by line.

Steps:

1. Read - take input
2. Evaluate - execute the code
3. Print - show the result
4. Loop - wait for next input

Start with
"node" in
terminal.

4. Node.js Project

A basic Node.js project has some default files and folders.

Project Structure:

```
my-app/
├── package.json
├── package-lock.json
├── node_modules/
└── app.js
```

Important:

- package.json stores project details.
- node_modules/ contains installed packages.
- app.js is the main file.

5. package.json

This file manages the project and its dependencies.

Field	Purpose
name	Project name
version	Project version
scripts	Custom commands (e.g. start)
dependencies	Production dependencies
devDependencies	Development dependencies

6. npm Basics

npm is the package manager for Node.js.

```
npm init          # create package.json (interactive)
npm init -y       # create package.json (default)
npm install express # install a package
npm uninstall express # remove a package
```

7. npm scripts

We can define custom scripts in package.json.

```
{
  "scripts": {
    "start": "node app.js"
  }
}
```

Now we can run:

```
npm start
```

runs node app.js

8. Flow: From Project to Node.js Application



9. Quick Comparison

Feature	Node.js	Browser JavaScript
Where it runs	Server / local	Browser
APIs	File system, network, OS, etc.	DOM, BOM
File access	Yes	No
Main purpose	Backend, servers, CLI tools	Web pages, UI, user interaction

Key Takeaways

- ✓ Node.js lets you run JavaScript outside the browser.
- ✓ It is built on Chrome's V8 engine.
- ✓ npm helps manage packages and dependencies.
- ✓ You can build servers, APIs, real-time apps and more with Node.js.

Browser JavaScript vs Node.js

Browser JavaScript

- Runs in browser
- Has DOM & BOM
- No file system access
- Used for web pages

Node.js

- Runs on server
- Has file system, network, OS APIs
- No DOM access
- Used for backend, CLI tools

★ 1. INTRODUCTION TO NODE.JS ★

1. What is Node.js ?

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser. It is built on Chrome's V8 engine and provides a set of APIs to work with the system, files, network and more.

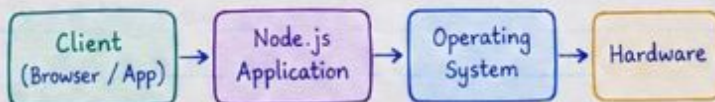
2. Why was Node.js created ?

- Browser JavaScript : Runs in the browser and mainly works with the DOM (Document Object Model).
- Server-side JavaScript : Node.js was created to run JavaScript on the server, so we can build web servers, APIs and backend applications.

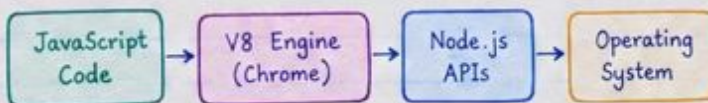
3. Node.js vs Browser JavaScript

Feature	Node.js	Browser JavaScript
Where it runs	Server / local machine	Web browser
APIs available	File system, network, OS, etc.	DOM, BOM, browser APIs
DOM access	No	Yes
File-system access	Yes	No
Main purpose	Backend, servers, CLI tools	Web pages, UI, user interaction

4. Node.js Architecture



5. Node.js Runtime



6. Important Features

- Fast : Uses V8 engine for high performance.
- Asynchronous : Handles many operations at once.
- Event-driven : Responds to events.
- Non-blocking I/O : Does not block the main thread.
- Cross-platform : Works on Windows, macOS, Linux.
- Large npm ecosystem : Lots of ready-made packages.

7. Where Node.js is Used ?

- REST APIs : Build backend APIs for web and mobile.
- Web servers : Serve static and dynamic content.
- Real-time apps : Chat apps, live updates, notifications.
- Backend systems : Business logic and data processing.
- CLI tools : Command line applications.
- Streaming applications : Video, audio, file streaming.

8. First Node.js Program

Create a file "app.js" and write :

```
1 console.log("Hello Node.js");
3
```

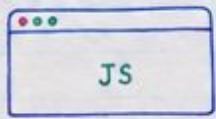
Run it with :

```
node app.js
```

Output :

```
Hello Node.js
```

Browser JavaScript vs Node.js (Quick View)

Browser JavaScript	Node.js
	
• Runs in browser • Has DOM & BOM • No file system access • Used for web pages	• Runs on server • Has file system, network, OS APIs • No DOM access • Used for backend, CLI, tools

Quick Summary

- Node.js = JavaScript outside browser
- Built on V8 engine
- Uses APIs to access system resources
- Fast, asynchronous, event-driven
- Used for APIs, servers, real-time apps
- Has a large npm ecosystem

➤ Node.js gives JavaScript the power to run the internet. ⚡